

Shape calculus and automatic differentiation for multi-phase level-set topology optimisation with unfitted finite elements

Zachary J. Wegert^{a,*}, Martin Berggren^b, Vivien J. Challis^a

^a*School of Mathematical Sciences, Queensland University of Technology, 2 George St, Brisbane, 4000, Queensland, Australia*

^b*Department of Computing Science, Umeå University, SE-901 87 Umeå, Sweden*

Abstract

We present shape calculus techniques and a scalable automatic shape differentiation framework for multi-phase topology optimisation on unfitted discretisations defined by several level-set functions. First we establish general, exact shape calculus expressions in the discrete case for multi-phase systems by leveraging concepts from convex geometry. To complement this theoretical foundation, we introduce an open-source multi-phase automatic shape differentiation framework based on polytopal cutting. This computational framework is validated against both finite differences and our established exact expressions, matching the latter to near machine precision. Furthermore, the proposed automatic shape differentiation is scalable across distributed computing environments, demonstrating near ideal weak scaling up to 1.65 billion finite elements across 13,824 computer cores. We demonstrate our implementation by solving unfitted multi-phase topology optimisation problems for anisotropic diffusion, linear elasticity, and fluid–structure interaction. Together, these theoretical and computational contributions provide a robust and accessible foundation for advancing multi-phase topology optimisation using unfitted finite element methods. In particular, the methods enable the solution of topology optimisation problems involving multi-phase and multi-physics systems with non-trivial boundary conditions. The open-source software is available at <https://github.com/zjwegert/GridapTopOpt.jl>.

Keywords: Level-set method, Multi-phase topology optimisation, Shape calculus, Automatic shape differentiation, Unfitted finite element methods, CutFEM

1. Introduction

New additive manufacturing technologies are enabling the fabrication of complex multi-material designs that were previously unattainable [1]. Across industries such as aviation and aerospace, these advances create significant opportunities for lead-time and cost reduction, for instance through weight reduction, without sacrificing design performance [2]. Owing in part to the considerable growth in additive manufacturing, topology optimisation has become a key design tool in several engineering fields such as structural design, aerospace, and robotics [3, 4]. Topology optimisation enables the optimisation of material layouts to maximise performance, while satisfying physics constraints formulated as partial differential equations (PDEs). Broadly speaking, topology optimisation methods fall into two categories: density-based (also

*Corresponding author

Email addresses: wegert@qut.edu.au (Zachary J. Wegert), martin.berggren@cs.umu.se (Martin Berggren), vivien.challis@qut.edu.au (Vivien J. Challis)

called material distribution) approaches [5, 6] and level-set approaches [7, 8]. In density methods, the design variables are typically material densities assigned to the elements or nodes of a mesh, whereas in level-set methods, the domain boundary is represented implicitly by a level-set function that is updated, for instance, according to an evolution equation.

Multi-phase topology optimisation facilitates the design of components with more than two distinct regions that have different material properties. In this context, both density and level-set approaches have been developed and studied. The predominant approach in the construction of these methods is to smoothly interpolate material properties between regions [4]. In some cases, such as for functionally graded materials [9], this interpolation of material properties is physically realistic. However, when modelling the situation of material properties that change sharply at the interface between the two phases, interpolating the material properties only approximates the interface conditions. Furthermore, the imposition of more complicated interface conditions in the case of multi-physics problems may be intractable. For this reason, several authors have recently developed level set-based body-fitted mesh approaches for multi-phase topology optimisation. Allaire et al. [10] considered the sensitivity analysis for two-phase problems involving a boundary-conforming mesh. Liu et al. [11] developed a body-fitted level-set method for two-dimensional bi-material elastic structures. Nguyen-Tran et al. [12] combined a two-dimensional body-fitted ‘multi-trimmed’ mesh technique with a reaction-diffusion-based level-set formulation for elastic structures. Kosta and Amir [13] and Kosta et al. [14] developed a multi-material isogeometric analysis method for two-dimensional elastic structures. These works are restricted to two-dimensional systems, and significant challenges persist in the tractable generation of more than one connected boundary-conforming mesh for multi-phase three-dimensional problems. Furthermore, the shape calculus is problem specific, poses significant difficulty, and often requires approximation. A versatile and systematic approach for the topology optimisation of general sharp-interface multi-material problems is currently absent from the literature.

Unfitted finite element methods are powerful techniques for solving PDEs over complicated physical domains that overcome the need to generate a boundary-conforming mesh [15, 16]. For domains defined using a level-set function, this typically works by computing a non-conforming triangulation of the boundary by intersecting the zero level curve with the background triangulation. Integration of the variational form of a PDE over the interface or cut cells can then be carried out using the resulting cut-cell triangulation. Ill-conditioned linear systems can arise due to arbitrarily small cuts [17], and therefore several schemes have been developed to address this issue. The approaches include ghost penalty stabilisation (CutFEM, [15]) and discrete extension operators (AgFEM, [16]). Unfitted finite element methods have been applied to solve a wide range of PDE problems, including modelling fluid–structure interaction [18], acoustics [19], and electrophysiology [20]. The extension of unfitted finite element methods to multi-phase PDE problems typically relies on recursively cutting the background mesh and adding appropriate jump conditions on the distinct phase interfaces [15, 21, 22, 23].

Unfitted finite element methods have been used for solving single-phase level-set topology optimisation problems, including those involving linear elasticity [24, 25], fluid flow [26, 25], and acoustics [19]. However, for multi-phase systems, a general framework for sensitivity analysis to enable gradient-based optimisation is lacking. Allaire et al. [10] considered the sensitivity analysis of two-phase problems using Hadamard’s domain transformation approach. This provides a clear framework for the case of two-phase problems. However, generalising the approach to an arbitrary number of phases defined by several level-set functions remains a difficult issue. As a result, the use of multi-phase unfitted finite element methods for topology optimisation has not been considered in the literature.

In recent work by Berggren [27] and the extension by Wegert et al. [25], a ‘discretise-then-differentiate’ shape calculus approach was proposed for single-phase problems based on the concept of Delfour dilations

[28]. This can be regarded as a ‘discretise-then-differentiate’ approach because it directly relies on locally parametrising the dilated region under a perturbation of a level-set function defined on a fixed mesh. Here we generalise the results of Berggren [27] and Wegert et al. [25] to arbitrary convex polytopes that arise in the case of multiple intersecting domains defined by level-set functions. The resulting shape calculus results make it possible to use gradient-based optimisation with multiple level-set functions.

It is also possible to compute shape derivatives using automatic differentiation. Automatic differentiation enables the exact recovery of derivatives to machine precision by applying the chain rule, in which derivatives are propagated from inputs to outputs (forward-mode) or from outputs to inputs (reverse-mode) [29]. In the context of the single-phase shape calculus of Berggren [27], automatic shape differentiation was proposed by Wegert et al. [25]. In that work, it was shown that automatic shape differentiation techniques were able to recover the shape calculus results of Berggren [27] to machine precision. This significantly reduces the burden associated with computing shape derivatives, particularly for complicated multi-physics problems. Here we consider the extension of the automatic shape differentiation techniques of Wegert et al. [25] to the case of several intersecting domains defined by level-set functions.

We implement multi-phase automatic shape differentiation for unfitted discretisations in the Julia package GridapTopOpt [30] and the wider Gridap-package ecosystem [31, 32]. Our implementation utilises GridapDistributed [33] so that it can be leveraged in both serial and distributed computing frameworks on central processing units (CPUs). Like Gridap, our implementation aims for a clear and intuitive application programming interface (API). Furthermore, our automatic differentiation implementation is fully compatible with GridapTopOpt’s automatic differentiation of arbitrary PDE-constrained maps. Providing this open-source software will facilitate the solution of complex multi-physics and multi-phase topology optimisation problems by the research community.

The remainder of the article is as follows. In Section 2, we introduce the reader to the concept of multi-phase unfitted finite elements to provide context for our results. In Section 3 we extend the shape calculus of Berggren [27] and Wegert et al. [25] to the case of intersecting domains defined by multiple level-set functions. The key mathematical results are given in Theorems 3.10, 3.15, and 3.18. In Section 4, we discuss automatic differentiation for multi-phase unfitted discretisations and provide scalability benchmarks for our automatic differentiation implementation. In Section 5, we briefly discuss our implementation of multi-phase unfitted level-set topology optimisation. In Section 6, we consider exemplar topology optimisation problems that showcase the capabilities provided by our approach. Finally, in Section 7 we present our concluding remarks.

2. Multi-phase unfitted finite elements

To illustrate the concept of multi-phase unfitted finite element methods, we will consider the construction of a variational formulation of a multi-phase diffusion problem on which the discretisation will be based, and for which we will parametrise the subdomains using level-set functions.

2.1. A multi-phase diffusion boundary-value problem

Consider a hold-all *background* domain $D \subset \mathbb{R}^d$, in which we consider an open and bounded domain $\Omega \subset D$ that can be subdivided into two non-overlapping subdomains Ω_1 and Ω_2 , that is, $\bar{\Omega} = \bar{\Omega}_1 \cup \bar{\Omega}_2$, and denote the nonempty interface between these by $\Gamma_{12} = \partial\Omega_1 \cap \partial\Omega_2$. On Ω , consider the scalar field u and the vector flux function $\mathbf{q}(u)$, defined locally on Ω_i , $i = 1, 2$, and denote $\mathbf{q}_i(u_i) := \mathbf{q}(u)|_{\Omega_i}$, where $u_i := u|_{\Omega_i}$. These fields

satisfy the boundary-value problem

$$-\nabla \cdot \mathbf{q}(u) = 0 \quad \text{in } \Omega, \quad (1a)$$

$$\mathbf{q}(u) \cdot \mathbf{n} = g \quad \text{on } \Gamma_N, \quad (1b)$$

$$\mathbf{q}(u) \cdot \mathbf{n} = 0 \quad \text{on } \partial\Omega \setminus (\Gamma_N \cup \Gamma_D), \quad (1c)$$

$$u = 0 \quad \text{on } \Gamma_D, \quad (1d)$$

where a homogeneous Dirichlet condition for u is given on one part of the boundary and homogeneous and inhomogeneous fluxes are defined on the rest of the boundary. In this formulation, we assume that $\Gamma_N \subset \partial D$ and $\Gamma_D \subset \partial D$. On the phase interface, we impose the transmission conditions

$$[[u]] = u_1 \mathbf{n}_1 + u_2 \mathbf{n}_2 = \mathbf{0} \quad \text{on } \Gamma_{12}, \quad (2a)$$

$$[[\mathbf{q}(u)]] = \mathbf{q}_1(u_1) \cdot \mathbf{n}_1 + \mathbf{q}_2(u_2) \cdot \mathbf{n}_2 = 0 \quad \text{on } \Gamma_{12}, \quad (2b)$$

where $\mathbf{n}_i$, $i = 1, 2$, is the outward-directed normal field on $\partial\Omega_i \cap \Gamma_{12}$ satisfying $\mathbf{n}_1 = -\mathbf{n}_2$. Here, we have adopted the jump convention of Arnold et al. [34, § 3.1]. Note that under this convention, the jump of a scalar function is a vector and the jump of a vector function is a scalar. The advantage with this notation is that no unique orientation needs to be assigned to Γ_{12} to define the jumps.

In this general discussion, the relation between the scalar unknown u and the flux function $\mathbf{q}(u)$ is unspecified. Later we will consider the case of anisotropic multi-phase diffusion with $\mathbf{q}_i = \mathbf{A}_i \nabla u_i$ where $\mathbf{A}_i$ are symmetric diffusion tensors.

To derive the variational formulation, we begin by defining the spaces $\mathcal{V}_i = \{v \in H^1(\Omega_i) : v|_{\Gamma_D} = 0\}$. Next, separate (1a) over each partition Ω_i , multiply by a test function $v_i \in \mathcal{V}_i$, sum the contributions, and integrate by parts. This gives

$$0 = \sum_i (-\nabla \cdot \mathbf{q}_i(u_i), v_i)_{\Omega_i} = \sum_i (\mathbf{q}_i(u_i), \nabla v_i)_{\Omega_i} - \sum_i (\mathbf{q}_i(u_i) \cdot \mathbf{n}_i, v_i)_{\partial\Omega_i}, \quad (3)$$

where we use the notation $(u, v)_\omega = \int_\omega u \cdot v \, d\omega$ for integrals over domains, boundaries, or interfaces ω . Note that the integrand includes a dot product for vector arguments u, v . Substituting boundary conditions (1b)–(1d) yields that

$$0 = \sum_i (\mathbf{q}_i(u_i), \nabla v_i)_{\Omega_i} - \sum_i (g, v_i)_{\Gamma_N} - (\mathbf{q}_1 \cdot \mathbf{n}_1, v_1)_{\Gamma_{12}} - (\mathbf{q}_2 \cdot \mathbf{n}_2, v_2)_{\Gamma_{12}}. \quad (4)$$

Defining the weighted mean operators $\{u\} = \kappa_1 u_1 + \kappa_2 u_2$ and $\langle u \rangle = \kappa_2 u_1 + \kappa_1 u_2$ with $\kappa_1 + \kappa_2 = 1$, a substitution shows that $[[\mathbf{q}v]] = [[v]] \cdot \{\mathbf{q}\} + \langle v \rangle [[\mathbf{q}]]$. Thus, (4) can be rewritten as

$$0 = \sum_i (\mathbf{q}_i(u_i), \nabla v_i)_{\Omega_i} - \sum_i (g, v_i)_{\Gamma_N} - (\{\mathbf{q}(u)\}, [[v]])_{\Gamma_{12}} - ([[\mathbf{q}(u)]], \langle v \rangle)_{\Gamma_{12}}, \quad (5)$$

and using the interface jump condition (2b), it follows that

$$0 = \sum_i (\mathbf{q}_i(u_i), \nabla v_i)_{\Omega_i} - \sum_i (g, v_i)_{\Gamma_N} - (\{\mathbf{q}(u)\}, [[v]])_{\Gamma_{12}}. \quad (6)$$

From here, we enforce the continuity condition in (2a) using a symmetric Nitsche method [15]. This method relies on adding, to the variational form, a symmetry term $(\{\mathbf{q}(v)\}, [[u]])_{\Gamma_{12}}$ and a penalty term $(\mu [[u]], [[v]])_{\Gamma_{12}}$, where μ is a penalty parameter. These are consistent additions in the sense that $[[u]] = 0$ in the case of u being a solution to boundary value problem (1) subject to transmission conditions (2).

Adding the the symmetry and penalty term to variational expression (6) yields the following variational formulation of the boundary value problem (1) under jump conditions (2):

$$\begin{aligned} \text{Find } (u_1, u_2) \in \mathcal{V}_1 \times \mathcal{V}_2 \text{ such that} \\ a([u_1, u_2], [v_1, v_2]) = l([v_1, v_2]) \quad \forall (v_1, v_2) \in \mathcal{V}_1 \times \mathcal{V}_2, \end{aligned} \quad (7)$$

where

$$a([u_1, u_2], [v_1, v_2]) = \sum_i (\mathbf{q}_i(u_i), \nabla v_i)_{\Omega_i} - (\{\mathbf{q}(u)\}, \llbracket v \rrbracket)_{\Gamma_{12}} - (\{\mathbf{q}(v)\}, \llbracket u \rrbracket)_{\Gamma_{12}} + (\mu \llbracket u \rrbracket, \llbracket v \rrbracket)_{\Gamma_{12}}, \quad (8)$$

and

$$l([v_1, v_2]) = \sum_i (g, v_i)_{\Gamma_N}. \quad (9)$$

2.2. Discrete formulation

Suppose that we partition the background domain D by cells $K \in \mathcal{T}_h$ where $\mathcal{T}_h$ is the set of cells. We denote the facets of $\mathcal{T}_h$ to be $\mathcal{F}_h$. Suppose that D is partitioned by two level-set functions $(\phi_1, \phi_2) \in \mathcal{V}_h^2$ such that

$$D_{\phi_i} = \{\mathbf{x} \in D : \phi_i(\mathbf{x}) < 0\}, \quad D \setminus \overline{D}_{\phi_i} = \{\mathbf{x} \in D : \phi_i(\mathbf{x}) > 0\}, \quad \partial D_{\phi_i} = \{\mathbf{x} \in D : \phi_i(\mathbf{x}) = 0\}, \quad (10)$$

for $i = 1, 2$, where $\mathcal{V}_h$ is the space of continuous, piecewise linear functions on $\mathcal{T}_h$:

$$\mathcal{V}_h = \{v \in C^0(\mathcal{T}_h) : v|_K \in \mathbb{P}_1(K) \quad \forall K \in \mathcal{T}_h\}. \quad (11)$$

Using set operations on D_{ϕ_1} and D_{ϕ_2} , we can define four non-overlapping subdomains. Figure 1 shows an example of this.

For the purpose of this demonstration, we consider parametrising Ω_1 and Ω_2 as $\Omega_1(\phi_1, \phi_2) = D_{\phi_1} \cap \overline{D}_{\phi_2}^c$ and $\Omega_2(\phi_1, \phi_2) = D_{\phi_1} \cap D_{\phi_2}$, respectively. As a result, Γ_{12} can be written as $\Gamma_{12}(\phi_1, \phi_2) = D_{\phi_1} \cap \partial D_{\phi_2}$.

Suppose that we wish to discretise using CutFEM over the mesh shown in Figure 1. The CutFEM is based on variational form (7) to which is added ghost penalty terms to control the condition number of the system matrix [35]. In this case, we add a ghost penalty term over the *ghost skeleton* $\mathcal{F}_{G_i}$ of each of Ω_i , where $\mathcal{F}_{G_i}$ is defined as follows: for distinct cells $K_1 \in \mathcal{T}_h$ and $K_2 \in \mathcal{T}_h$, a facet $F \in \mathcal{F}_{G_i}$ is given by $F = K_1 \cap K_2$ where at least one of K_1 or K_2 intersect the interface $\partial \Omega_i$ [15]. In particular, we add terms of the form

$$j([u_1, u_2], [v_1, v_2]) = \sum_i \sum_{F \in \mathcal{F}_{G_i}} (\eta_i(h) \llbracket \partial_{\mathbf{n}_F} u_i \rrbracket, \llbracket \partial_{\mathbf{n}_F} v_i \rrbracket)_F, \quad (12)$$

where $\eta_i(h)$ are penalty parameters.

The CutFEM approximation of boundary-value problem (1)–(2) is then:

$$\begin{aligned} \text{Find } (u_1, u_2) \in \mathcal{V}_{1,h} \times \mathcal{V}_{2,h} \text{ such that} \\ A([u_1, u_2], [v_1, v_2]) = l([v_1, v_2]) \quad \forall (v_1, v_2) \in \mathcal{V}_{1,h} \times \mathcal{V}_{2,h}, \end{aligned} \quad (13)$$

with

$$A([u_1, u_2], [v_1, v_2]) = a([u_1, u_2], [v_1, v_2]) + j([u_1, u_2], [v_1, v_2]), \quad (14)$$

where a and l are defined in 8 and (9), respectively, and $\mathcal{V}_{i,h} = \{v \in C^0(\mathcal{T}_{i,h}) : v|_K \in \mathbb{P}_1(K) \quad \forall K \in \mathcal{T}_h, v|_{\Gamma_D} = 0\}$, in which $\mathcal{T}_{i,h}$ are the active elements associated with phase i given by

$$\mathcal{T}_{i,h} = \{K \in \mathcal{T}_h : \Omega_i \cap K \neq \emptyset\}. \quad (15)$$

Now that we have constructed a discrete forward problem, we will in the next section consider how to compute derivative information under perturbations of ϕ_1 and ϕ_2 using level-set shape calculus.

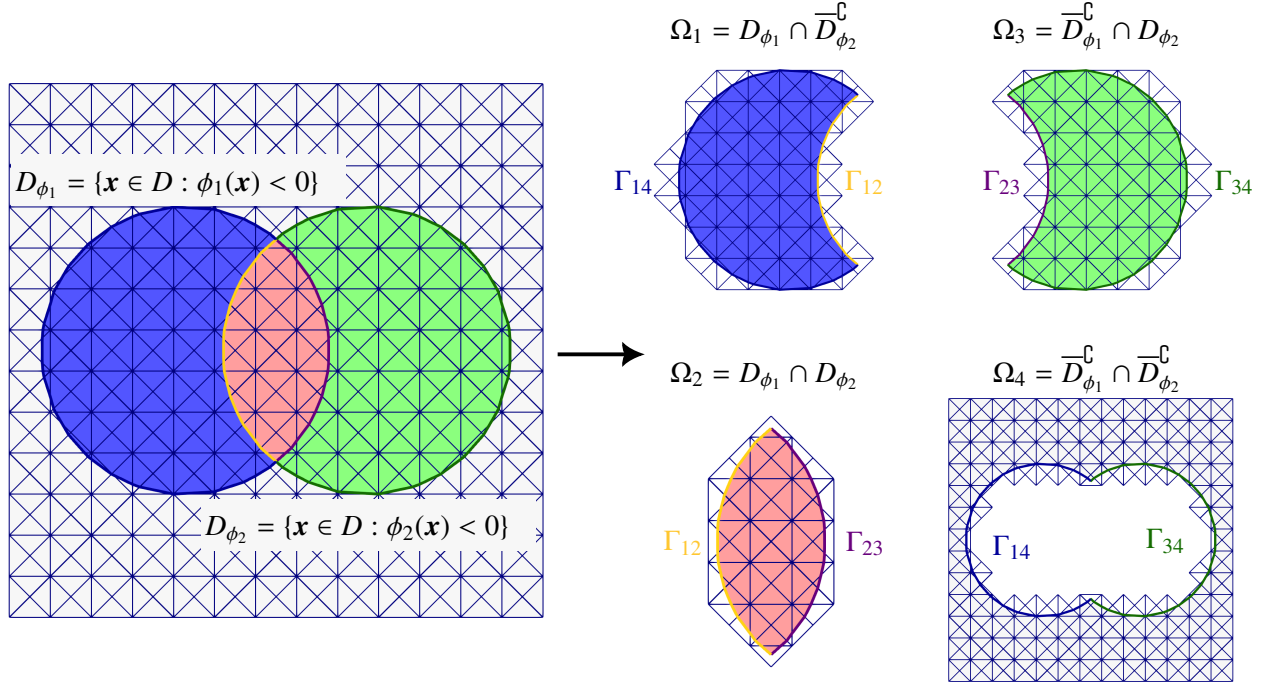


Figure 1 An example of non-overlapping subdomains Ω_j defined by set operations on the domains D_{ϕ_i} that are generated by two level-set functions ϕ_i . The interfaces between each subdomain Ω_i can be written as $\Gamma_{ij} = \partial\Omega_i \cap \partial\Omega_j$ for $i \neq j$.

3. Shape differentiation via level-set dilation

In the following, we extend the shape calculus of Berggren [27] to the case of intersecting domains defined by level-set functions. There are three types of directional derivatives under a perturbation of a level-set function that must be considered:

- (i) *Domain integrals* (Theorem 3.10), that is, the directional derivative of a functional defined over the intersection of several domains, where the level-set function being perturbed defines one of these domains. An example is a functional defined on $\Omega_1 = D_{\phi_1} \cap \overline{D}_{\phi_2}^c$ in Figure 1 under a perturbation of ϕ_1 or ϕ_2 .
- (ii) *Boundary integrals truncated by a perturbed domain* (Theorem 3.15), that is, the directional derivative of a functional defined over the boundary of a domain that is intersected with several other domains, and where the level-set function that is being perturbed defines one of the domains intersecting the boundary. An example is a functional defined on $\Gamma_{23} = \partial D_{\phi_1} \cap D_{\phi_2}$ in Figure 1 under a perturbation of ϕ_2 .
- (iii) *Boundary integrals* (Theorem 3.18). Here, the functional is of the same type as for (ii); however the level-set function that is being perturbed to compute the directional derivative is the one that defines the boundary. An example is a functional defined on $\Gamma_{23} = \partial D_{\phi_1} \cap D_{\phi_2}$ in Figure 1 under a perturbation of ϕ_1 .

Our results for Cases (i) and (iii) reduce to those of Berggren [27] and Wegert et al. [25] for the case of a single level-set function. Case (ii) is new and appears only in the case of two or more intersecting level-set functions. We restrict the scope to simplex-type meshes in $\mathbb{R}^d$.

3.1. Preliminaries

For clarity, we restate some of the previously used notation. Let $D \subset \mathbb{R}^d$ be a hold-all domain, to be referred to as the background domain. We partition D by simplices $K \in \mathcal{T}_h$, where $\mathcal{T}_h$ is the set of cells, and denote the facets of $\mathcal{T}_h$ to be $\mathcal{F}_h$. Note that the facets are of codimension one. We consider the following space of level-set functions that partition D :

$$\mathcal{V}_h = \{v \in C^0(\mathcal{T}_h) : v|_K \in \mathbb{P}_1(K) \ \forall K \in \mathcal{T}_h\}. \quad (16)$$

Assume now that D is partitioned by a level-set function $\phi_i \in \mathcal{V}_h$ such that

$$D_{\phi_i} = \{\mathbf{x} \in D : \phi_i(\mathbf{x}) < 0\}, \quad D \setminus \overline{D_{\phi_i}} = \{\mathbf{x} \in D : \phi_i(\mathbf{x}) > 0\}, \quad \partial D_{\phi_i} = \{\mathbf{x} \in D : \phi_i(\mathbf{x}) = 0\}, \quad (17)$$

and consider perturbations of the form

$$\phi_{i,t}(\mathbf{x}) = \phi_i(\mathbf{x}) + tw(\mathbf{x}), \quad (18)$$

where $t \geq 0$ and w is a linear Lagrangian basis function of $\mathcal{V}_h$. We define the associated perturbed domains

$$D_{\phi_i,t} = \{\mathbf{x} \in D : \phi_{i,t}(\mathbf{x}) < 0\}. \quad (19)$$

Note that by construction $D_{\phi_i,t} \subseteq D_{\phi_i,t'}$ for $t' \leq t$, since $w \geq 0$ by convention. In other words, because of our choice of perturbation and how the level-set function defines D_{ϕ_i} , the perturbed domain is always shrinking for increased t [27].

Before proceeding, we recall the following assumptions from Berggren [27]:

Assumption 3.1. There is a $t_{\max} > 0$ such that for each $K \in \mathcal{T}_h$, if $\partial D_{\phi_i,t} \cap K$ is either empty or non-empty for some $t \in (0, t_{\max}]$, it has the same property for each $t \in (0, t_{\max}]$.

This is an assumption on the smallness of the perturbation, motivated by the fact that we will in the end be taking limits as $t \rightarrow 0^+$.

Assumption 3.2. The boundary ∂D_{ϕ_i} does not intersect any mesh points of the triangulation $\mathcal{T}_h$.

Under this assumption, the semiderivatives will agree in the limits $t \rightarrow 0^+$ and $t \rightarrow 0^-$, respectively.

3.2. The conceptual approach

To illustrate the conceptual approach of level-set shape differentiation for multiple level-set functions, suppose we consider a functional of the form

$$J_1(\phi_1) = \int_{D_{\phi_1} \cap D_{\phi_2}} f(\mathbf{x}) \, d\mathbf{x} \quad (20)$$

where ϕ_2 is fixed. For $t > 0$ sufficiently small, it holds that

$$\frac{1}{t}(J_1(\phi_{1,t}) - J_1(\phi_1)) = \frac{1}{t} \int_{D_{\phi_{1,t}} \cap D_{\phi_2}} f(\mathbf{x}) \, d\mathbf{x} - \frac{1}{t} \int_{D_{\phi_1} \cap D_{\phi_2}} f(\mathbf{x}) \, d\mathbf{x} = -\frac{1}{t} \int_{E_t} f(\mathbf{x}) \, d\mathbf{x}, \quad (21)$$

where, since $(D_{\phi_{1,t}} \cap D_{\phi_2}) \subset (D_{\phi_1} \cap D_{\phi_2})$, $E_t = (D_{\phi_1} \cap D_{\phi_2}) \setminus (D_{\phi_{1,t}} \cap D_{\phi_2})$. Moreover, in the algebra of subsets A, B, C of a universe D , it holds that $(A \cap B) \setminus (C \cap B) = (A \setminus C) \cap B$. Thus, $E_t = (D_{\phi_1} \cap D_{\phi_2}) \setminus (D_{\phi_{1,t}} \cap D_{\phi_2}) = (D_{\phi_1} \setminus D_{\phi_{1,t}}) \cap D_{\phi_2}$. Figure 2 shows an example region E_t in two dimensions. Broadly speaking, an approach to compute the directional derivative $dJ_1(\phi_1)(w)$ is to: (i) parametrise E_t ; (ii) express the final term in (21) as a kind of coarea formula (that is, an integral of surface integrals); and (iii) take the limit as $t \rightarrow 0$.

Without additional mathematical machinery, the number of cases needed to derive the parametrisation of E_t and the corresponding coarea formula is combinatorial in the number of level-set functions. In the below we therefore borrow some concepts from convex geometry to generalise the approach of Berggren [27] to arbitrary convex polytopes.

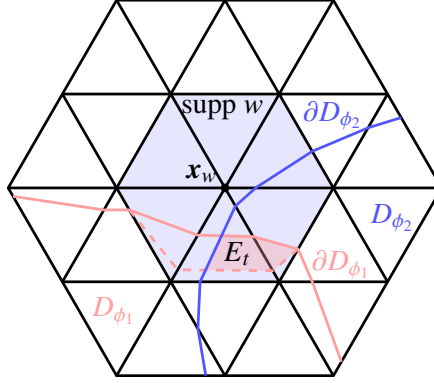


Figure 2 An example region $E_t = (D_{\phi_1} \setminus D_{\phi_{1,t}}) \cap D_{\phi_2}$ under a perturbation $\phi_{1,t} = \phi_1 + tw$. We denote the support of a linear Lagrangian basis w located at x_w by the shaded blue region. Here we only depict two domains defined by level-set functions. In the rest of the article, we consider the case of an arbitrary number of domains.

3.3. Domain integrals

In order to prove a general result for several intersecting domains, we will consider the generalisation of the results of Berggren [27] to arbitrary convex polytopes $\mathcal{P} \subseteq K \in \mathcal{T}_h$. Let $V(\mathcal{P})$ denote the extreme points of convex polytope $\mathcal{P}$. We begin with the following results based on concepts in Gruber [36].

Proposition 3.3. *An intersection plane Π cuts a convex polytope $\mathcal{P}$ into two convex polytopes.*

Proof. Suppose we have a convex polytope $\mathcal{P}$ generated by a finite set of vertices $V(\mathcal{P})$. By the Minkowski–Weyl theorem [Theorem 14.2, 36], $\mathcal{P}$ has a halfspace representation

$$\mathcal{P} = \bigcap_{i=1}^m H_i, \quad (22)$$

where $\{H_i\}_{i=1}^m$ is a finite set of halfspaces. Note that halfspaces are defined as one of the two convex sets generated by partitioning $\mathbb{R}^d$ with a hyperplane, that is,

$$H_i = \{\mathbf{x} \in \mathbb{R}^d : \mathbf{x} \cdot \boldsymbol{\alpha}_i < c_i\}. \quad (23)$$

Now, suppose that Π divides $\mathbb{R}^d$ into halfspaces H_{Π}^+ and H_{Π}^- . Then, the resulting polytopes are

$$\mathcal{P}_{\pm} = \mathcal{P} \cap H_{\Pi}^{\pm}. \quad (24)$$

Since $\mathcal{P}$ and $H_{\Pi}^{\pm}$ are convex, their intersection is convex. $\square$

Lemma 3.4. *Given a convex polytope $\mathcal{P}$ and halfspaces $H^{\pm}$ generated by the plane Π cutting $\mathcal{P}$, there exists $\hat{\mathbf{x}}_w \in V(\mathcal{P} \cap H^{\pm})$ and $S \subset \partial(\mathcal{P} \cap H^{\mp})$ such that all points $\mathbf{x} \in \mathcal{P} \cap H^{\mp}$ can be parametrised by $\mathbf{x} = \hat{\mathbf{x}}_w + \sigma(\mathbf{x}_S - \hat{\mathbf{x}}_w)$ for $\sigma \in (0, 1)$ and $\mathbf{x}_S \in S$.*

Proof. The proof follows by convexity of $\mathcal{P} \cap H^{\pm}$ from Proposition 3.3. In particular, by convexity, all points $\mathbf{x} \in \mathcal{P} \cap H^{\mp}$ can be connected by a ray to a point $\hat{\mathbf{x}}_w \in V(\mathcal{P} \cap H^{\pm})$. Extending this ray to the boundary $\partial(\mathcal{P} \cap H^{\mp})$ recovers the set S . $\square$

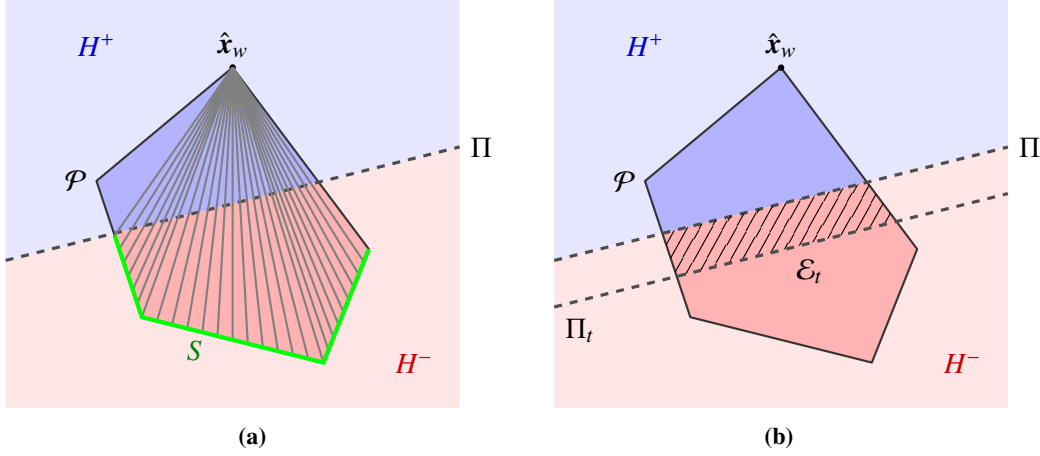


Figure 3 (a) A conceptual picture of the statement in Lemma 3.4. (b) A visualisation of the quantities Π_t and $\mathcal{E}_t = (H^- \setminus \overline{H_t^-}) \cap \mathcal{P}$ where Π_t satisfies $\Pi_t|_K = \{\mathbf{x} \in K : \phi_t(\mathbf{x}) = 0\}$.

A conceptual picture of the statement in Lemma 3.4 is shown in Figure 3a for the case of $\hat{\mathbf{x}}_w \in V(\mathcal{P} \cap H^+)$ and $S \subset \partial(\mathcal{P} \cap H^-)$.

Suppose that we now specify Π *locally* in a polytope $\mathcal{P} \subseteq K \in \mathcal{T}_h$ using a level-set function $\phi(\mathbf{x}) = 0$. Without lack of generality, we assume that Π_t , defined such that $\Pi_t|_K = \{\mathbf{x} \in K : \phi_t(\mathbf{x}) = 0\}$, is *pushed* into the halfspace H^- generated by Π for $t \geq 0$. We can then define the perturbed region to be $\mathcal{E}_t = (H^- \setminus \overline{H_t^-}) \cap \mathcal{P}$, which is itself convex. Figure 3b shows a conceptual picture of this setup.

We require extensions of Assumption 3.1 and 3.2 because we are working on a general polytope $\mathcal{P} \subseteq K$. In the first, we restrict the perturbation to be sufficiently small such that the following condition holds.

Assumption 3.5. There is a $t_{\max} > 0$ such that, for the convex polytope $\mathcal{P} \subseteq K \in \mathcal{T}_h$, the geometric elements (that is, facets, edges, and vertices for $d = 3$) of $H^\pm \cap \mathcal{P}$ are topologically equivalent to those of $H_t^\pm \cap \mathcal{P}$ for all $t \in (0, t_{\max}]$.

Next, we introduce an assumption on the intersection of multiple domain boundaries.

Assumption 3.6. For a subfacet of dimension $d - 2$ (that is, a point for $d = 2$ or an edge for $d = 3$) and where level-set function ϕ_k is perturbed, ∂D_{ϕ_k} intersects at most one other ∂D_{ϕ_i} along the subfacet, and any intersection does not occur on the boundary of a cell ∂K .

We can now establish the existence of a parametrisation of $\mathcal{E}_t$.

Lemma 3.7. Consider a moving plane Π_t that is locally parametrised by $\phi_t(\mathbf{x}) = \phi(\mathbf{x}) + t w(\mathbf{x}) = 0$ for some $t \in (0, t_{\max}]$, according to Assumption 3.5, and where w is a linear Lagrangian basis function. Suppose $K \in \mathcal{T}_h$ is a cell in the support of the basis function w and that Π_t cuts a convex polytope $\mathcal{P} \subseteq K$ into two sub-polytopes $\mathcal{P} \cap H_t^\pm$, where $H_t^\pm$ are the halfspaces defined by Π_t . Then, there exists a set S and a diffeomorphism $\hat{X} : S \times (0, t) \rightarrow \mathcal{E}_t$ such that $\hat{X}(S, \tau) = \Pi_\tau \cap \mathcal{P}$ for each $\tau \in (0, t)$ where $\mathcal{E}_t = (H^- \setminus \overline{H_t^-}) \cap \mathcal{P}$.

Proof. By Lemma 3.4, there exists $\hat{\mathbf{x}}_w \in V(\mathcal{P} \cap H_0^+)$ and $S \subset \partial(\mathcal{P} \cap H_0^-)$ such that all points $\mathbf{x}_\tau \in \Pi_\tau \cap \mathcal{P}$ can be parametrised by

$$\mathbf{x}_\tau = \hat{\mathbf{x}}_w + \sigma(\mathbf{x}_S - \hat{\mathbf{x}}_w) \quad (25)$$

for $\sigma \in (0, 1)$ and $\mathbf{x}_S \in S$. Note that $\hat{\mathbf{x}}_w$ is fixed for all $t \in (0, t_{\max})$ by Assumption 3.5.

We may then proceed in a similar way to Berggren [27, Lemma 6.5]. Since ϕ_τ vanishes on Π_τ we have

$$0 = \phi_\tau(\mathbf{x}_\tau) = \phi_\tau(\hat{\mathbf{x}}_w + \sigma(\mathbf{x}_S - \hat{\mathbf{x}}_w)) = \phi_\tau(\hat{\mathbf{x}}_w) + \sigma \nabla \phi_\tau|_K \cdot (\mathbf{x}_S - \hat{\mathbf{x}}_w) \quad (26)$$

where we have used the fact that $\phi_\tau|_K$ is linear. Since $\hat{\mathbf{x}}_w$ and each $\mathbf{x}_S \in S$ are on opposite sides of Π_τ , the last term in (26) cannot vanish. Thus, for each $\mathbf{x}_S \in S$, we can solve (26) for σ , and define $\hat{X} : S \times (0, t) \rightarrow \mathcal{E}_t$ by

$$\hat{X}(\mathbf{x}_S, \tau) = \hat{\mathbf{x}}_w - \frac{\phi_\tau(\hat{\mathbf{x}}_w)}{\nabla \phi_\tau|_K \cdot (\mathbf{x}_S - \hat{\mathbf{x}}_w)} (\mathbf{x}_S - \hat{\mathbf{x}}_w), \quad (27)$$

which satisfies $\hat{X}(S, \tau) = \Pi_\tau \cap \mathcal{P}$. Thus, $\hat{X}$ is surjective. Furthermore, $\hat{X}$ is injective because $\Pi_\tau \cap \mathcal{P}$ and $\Pi_{\tau'} \cap \mathcal{P}$ nowhere intersect for $\tau \neq \tau'$, and because of uniqueness of (26) with respect to σ and $\mathbf{x}_S$. Finally, $\hat{X}$ is smooth because it is rational and non-singular. $\square$

Using Lemma 3.7 we can now parametrise the integral over $\mathcal{E}_t$ where $\mathcal{P}$ is a convex polytope equal to, or a subset of, $K \in \mathcal{T}_h$.

Lemma 3.8. *Consider a perturbation $\phi_t(\mathbf{x}) = \phi(\mathbf{x}) + t w(\mathbf{x})$ for some $t \in (0, t_{\max}]$, according to Assumption 3.5, and where w is a linear Lagrangian basis function. Let $\mathcal{P} \subseteq K \in \mathcal{T}_h$ be a convex polytope where K is in the support of w and Π_t be a moving plane parametrised locally by $\Pi_t|_K = \{\mathbf{x} \in K : \phi_t(\mathbf{x}) = 0\}$. For $f \in C^0(\mathcal{E}_t)$ where $\mathcal{E}_t = (H^- \setminus \overline{H_t^-}) \cap \mathcal{P}$, it holds that*

$$\int_{\mathcal{E}_t} f \, d\mathbf{x} = \int_0^t \int_{\Pi_\tau \cap \mathcal{P}} f \frac{w}{|\partial_{\mathbf{n}_\tau} \phi_\tau|} \, dS \, d\tau. \quad (28)$$

Proof. Suppose $\hat{X}$ is the mapping from Lemma 3.7 and $\mathcal{U} \subset \mathbb{R}^{d-1}$ is the domain of a piecewise smooth parametrisation of S , again from Lemma 3.7. We define a parametrisation of $\mathcal{E}_t$ by $X : \mathcal{U} \times (0, t) \rightarrow \mathcal{E}_t$ as

$$X(\mathbf{u}, \tau) = \hat{X}(\mathbf{x}_S(\mathbf{u}), \tau), \quad (29)$$

where $\tau \in (0, t)$ and the mapping $\mathbf{u} \mapsto \mathbf{x}_S(\mathbf{u})$ is piecewise affine such that $X(\mathcal{U}, \tau) = \Pi_\tau \cap \mathcal{P}$. The Jacobian of X is given by

$$\det DX = \pm \frac{\partial X}{\partial \tau} \cdot N_\tau \sigma, \quad (30)$$

where, in the general case for $d > 2$, using the notation of Spivak [37, Ch. 4],

$$N_\tau \sigma = \frac{\partial X}{\partial u_1} \times \cdots \times \frac{\partial X}{\partial u_{d-1}}, \quad \sigma = \left\| \frac{\partial X}{\partial u_1} \times \cdots \times \frac{\partial X}{\partial u_{d-1}} \right\|, \quad (31)$$

in which $\times$ is the generalised cross product defined such that $N_\tau \sigma$ is the vector perpendicular to each of the tangent vectors $\partial X / \partial u_i$, and the plus/minus depends on the orientation of the parametrisation. For the case of $d = 2$, we can define

$$N_\tau \sigma = \left(-\frac{\partial X_2}{\partial u_1}, \frac{\partial X_1}{\partial u_1} \right)^\top \quad (32)$$

By these definitions, N_τ will be the unit normal vector to the hyperplane $\Pi_\tau \cap \mathcal{P}$ outward with respect to H_τ^- .

We may now follow similar steps to those in Berggren [27]. Namely, since $X(\mathcal{U}, \tau) = \Pi_\tau \cap \mathcal{P}$ and $\phi_\tau|_{\Pi_\tau \cap \mathcal{P}} = 0$ we have

$$\phi_\tau \circ X = \phi \circ X + \tau w \circ X = 0. \quad (33)$$

Differentiating with respect to τ , it follows that

$$\begin{aligned} 0 &= \frac{\partial}{\partial \tau}(\phi \circ X + \tau w \circ X) = (\nabla \phi \circ X) \cdot \frac{\partial X}{\partial \tau} + w \circ X + \tau(\nabla w \circ X) \cdot \frac{\partial X}{\partial \tau} \\ &= w \circ X + (\nabla \phi_\tau \circ X) \cdot \frac{\partial X}{\partial \tau} = w \circ X + \|\nabla \phi_\tau \circ X\| N_\tau \cdot \frac{\partial X}{\partial \tau}, \end{aligned} \quad (34)$$

where, in the fourth equality, we have used the property of the level-set function that

$$N_\tau = \frac{\nabla \phi_\tau \circ X}{\|\nabla \phi_\tau \circ X\|}. \quad (35)$$

Combining (30) and (34) and taking the absolute value then gives

$$|\det DX| = \frac{w \circ X}{\|\nabla \phi_\tau \circ X\|} \sigma, \quad (36)$$

where we have used that $w \geq 0$. We can now parametrise the integral in terms of X as follows,

$$\int_{\mathcal{E}_t} f \, d\mathbf{x} = \int_0^t \int_{\mathcal{U}} f \circ X |\det DX| \, du \, d\tau = \int_0^t \int_{\mathcal{U}} f \circ X \frac{w \circ X}{\|\nabla \phi_\tau \circ X\|} \sigma \, du \, d\tau. \quad (37)$$

Thus a change of variables and using the fact that the tangential derivative of ϕ_τ vanishes on $\Pi_\tau \cap \mathcal{P}$ yields

$$\int_0^t \int_{\mathcal{U}} f \circ X \frac{w \circ X}{\|\nabla \phi_\tau \circ X\|} \sigma \, du \, d\tau = \int_0^t \int_{\Pi_\tau \cap \mathcal{P}} f \frac{w}{|\partial_{n_\tau} \phi_\tau|} \, dS \, d\tau, \quad (38)$$

which completes the proof. $\square$

Now having a result for a generic polytope, we need to relate this back to the case of several intersecting domains. To do so, we introduce some additional notation as follows.

For n level-set functions, there are 2^n different unique subdomains that combinatorically can be defined by the signs of each level-set function. For example, for the case of two level-set functions as in Figure 1, there are four subdomains $\Omega_1, \dots, \Omega_4$ defined as

$$D_{\phi_1} \cap \overline{D}_{\phi_2}^c, D_{\phi_1} \cap D_{\phi_2}, \overline{D}_{\phi_1}^c \cap D_{\phi_2}, \text{ and } \overline{D}_{\phi_1}^c \cap \overline{D}_{\phi_2}^c. \quad (39)$$

More generally, the 2^n subdomain can be enumerated as [10]

$$\left\{ \begin{aligned} \Omega_1 &= D_{\phi_1} \cap D_{\phi_2} \cap \dots \cap D_{\phi_n} \\ \Omega_2 &= \overline{D}_{\phi_1}^c \cap D_{\phi_2} \cap \dots \cap D_{\phi_n} \\ &\vdots \\ \Omega_{2^n} &= \overline{D}_{\phi_1}^c \cap \overline{D}_{\phi_2}^c \cap \dots \cap \overline{D}_{\phi_n}^c \end{aligned} \right., \quad (40)$$

where the order can be arbitrarily chosen. Taking inspiration from this representation, we define F to be any one of the subdomains $\Omega_1, \Omega_2, \dots, \Omega_{2^n}$. We then define F_k to be the region with the domain defined by ϕ_k , which will be either D_{ϕ_k} or $\overline{D}_{\phi_k}^c$, removed from the construction of F . Analogously, also define F_{kl} to be the region with both of the domains defined by ϕ_k and ϕ_l removed from the construction of F . For

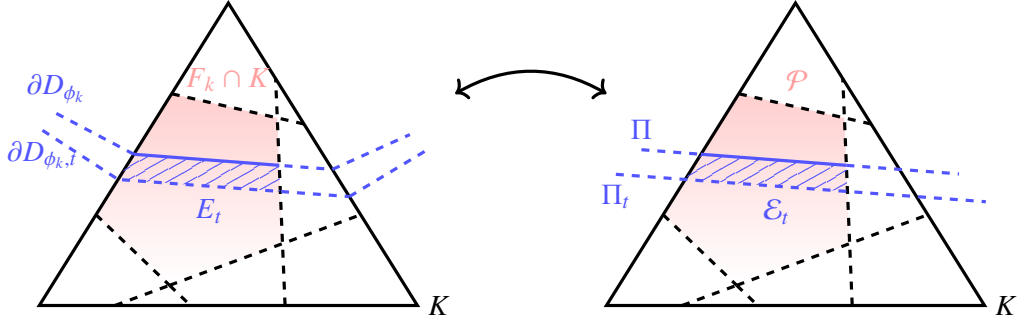


Figure 4 An illustration of the two equivalent representations of a domain being cut by several level-set functions. The left figure shows the domain $F_k \cap K$ cut by a boundary $\partial D_{\phi_k,t}$ for some $t \in (0, t_{\max}]$ satisfying Assumption 3.5 and the resulting perturbed domain $E_t = (D_{\phi_k} \setminus \overline{D_{\phi_k,t}}) \cap F_k$. Note that the dashed black lines represent fixed boundaries of other domains D_{ϕ_i} for $i \neq k$. The right figure shows the equivalent representation in terms of a polytope $\mathcal{P}$ cut by a plane Π_t , again for some $t \in (0, t_{\max}]$. In this case, the perturbed region is defined using the halfspaces for Π and Π_t as $\mathcal{E}_t = (H^- \setminus \overline{H_t^-}) \cap \mathcal{P}$. Since the $\partial D_{\phi_k,t} \cap K$ is planar, it holds that inside K we have $\partial D_{\phi_k,t} \cap K \equiv \Pi_t$. An immediate corollary of this is that we also have $E_t \cap K \equiv \mathcal{E}_t$.

example, if $F = \Omega_2 = \overline{D_{\phi_1}}^c \cap D_{\phi_2} \cap \dots \cap D_{\phi_n}$, then F_1 and F_{1n} are given by $F_1 = D_{\phi_2} \cap \dots \cap D_{\phi_n}$ and $F_{1n} = D_{\phi_2} \cap \dots \cap D_{\phi_{n-1}}$, respectively.

Finally, we need a way to keep track of which domains are and are not complemented in a particular F . To do so, we denote the set of all domains $\mathcal{D} = \{D_{\phi_1}, D_{\phi_2}, \dots, D_{\phi_n}\}$ and define $\mathcal{D}_c \subset \mathcal{D}$ and $\mathcal{D}_p \subset \mathcal{D}$ to be the sets of domains that appear with and without complements in F , respectively. By definition, these sets of domains should satisfy $\mathcal{D}_c \cap \mathcal{D}_p = \emptyset$ and $\mathcal{D}_c \cup \mathcal{D}_p = \mathcal{D}$. In terms of our previous example for $F = \Omega_2 = \overline{D_{\phi_1}}^c \cap D_{\phi_2} \cap \dots \cap D_{\phi_n}$, we have $\mathcal{D}_c = \{D_{\phi_1}\}$ and $\mathcal{D}_p = \{D_{\phi_2}, \dots, D_{\phi_n}\}$.

Remark 3.9. Using the sets of domains $\mathcal{D}_c$ and $\mathcal{D}_p$, we can alternatively write F as

$$F = \left[\bigcap_{A \in \mathcal{D}_p \cup \{D\}} A \right] \cap \left[\bigcap_{B \in \mathcal{D}_c} B^c \right], \quad (41)$$

where $D^c = \emptyset$ because D is the hold-all domain. Similar expressions can be written for F_k and F_{kl} , however for brevity we do not write them here.

The above notation is convenient, because when we perturb the k^{th} level-set function, we are actually cutting a fixed domain F_k with $\partial D_{\phi_k,t}$. As such, the perturbed domain E_t can be redefined as $E_t = (D_{\phi_k} \setminus \overline{D_{\phi_k,t}}) \cap F_k$, assuming $D_{\phi_k} \in \mathcal{D}_p$. We can then establish equivalence to the polytopal representation as follows: given $K \in \mathcal{T}_h$, we choose

$$\mathcal{P} \equiv F_k \cap K. \quad (42)$$

We then use that $\partial D_{\phi_k,t} \cap K$ is planar, and we can therefore choose $\Pi_t \equiv \partial D_{\phi_k,t} \cap K$. Finally, by construction

$$E_t \cap K \equiv \mathcal{E}_t \quad (43)$$

inside K . Figure 4 shows a visualisation of this.

In Theorem 3.10 below, as well as in Theorems 3.15, and 3.18, we use the notation $C^k(\overline{\mathcal{T}_h})$, where $k \in \mathbb{N}$, for the space of functions f such that $f|_K \in C^k(\overline{K})$ for each $K \in \mathcal{T}_h$. Note that such functions may possess jump discontinuities between elements, a generality that is needed for some functions emanating from a finite-element discretisation.

Theorem 3.10. *Let F be any one of the subdomains (40) defined by n level set functions. Given a perturbation $\phi_{k,t}(\mathbf{x}) = \phi_k(\mathbf{x}) + tw(\mathbf{x})$, where w is a linear Lagrangian basis function, the directional semiderivative of the integral*

$$J(\phi_k) = \int_F f \, d\mathbf{x}, \quad (44)$$

with $D_{\phi_k} \in \mathcal{D}_p$ and $f \in C^0(\overline{\mathcal{T}_h})$, satisfies

$$dJ(\phi_k)(w) = \lim_{t \rightarrow 0^+} \frac{1}{t} (J(\phi_{k,t}) - J(\phi_k)) = - \int_{\partial D_{\phi_k} \cap F_k} f \frac{w}{|\partial_n \phi_k|} \, dS, \quad (45)$$

where $\mathbf{n}$ is the unit normal to ∂D_{ϕ_k} . If $D_{\phi_k} \in \mathcal{D}_c$, the right-hand side is of opposite sign.

(i) *If Assumption 3.2 is violated, then f and $\partial_n \phi$ are the limits of these functions from the interior of F , whenever these quantities possess jump discontinuities on $\partial D_{\phi_k} \cap F_k$.*

(ii) *If Assumption 3.2 is satisfied, the semiderivatives $t \rightarrow 0^-$ and $t \rightarrow 0^+$ agree.*

Proof. Suppose $D_{\phi_k} \in \mathcal{D}_p$ and let $t \in (0, t_{\max}]$ according to Assumption 3.5. Then we have

$$\frac{1}{t} (J(\phi_{k,t}) - J(\phi_k)) = \frac{1}{t} \left(\int_{D_{\phi_{k,t}} \cap F_k} f \, d\mathbf{x} - \int_{D_{\phi_k} \cap F_k} f \, d\mathbf{x} \right) = -\frac{1}{t} \sum_{K \in \mathcal{T}_h} \int_{E_t \cap K} f \, d\mathbf{x}, \quad (46)$$

where, using the same arguments as used for (21), $E_t = (D_{\phi_k} \setminus \overline{D_{\phi_{k,t}}}) \cap F_k$. By the equivalence of the domain representation and the polytopal representation inside a cell K , we can apply Lemma 3.8 to find

$$\frac{1}{t} (J(\phi_{k,t}) - J(\phi_k)) = -\frac{1}{t} \sum_{K \in \mathcal{T}_h} \int_0^t \int_{\partial D_{\phi_k, \tau} \cap F_k \cap K} f \frac{w}{|\partial_n \phi_\tau|} \, dS \, d\tau, \quad (47)$$

where we have identified $\Pi_t \equiv \partial D_{\phi_{k,t}} \cap K$. Taking the limit as then yields (45). The case for $D_{\phi_k} \in \mathcal{D}_c$ follows similarly with the addition of a chain-rule computation because $D_{\phi_k}^c = D_{-\phi_k}$.

Results (i) and (ii) follow by similar arguments to those of Berggren [27]. Namely, (i) follows from the fact that $\partial D_{\phi_{k,\tau}} \cap F_k$ is interior to F for each $\tau \in (0, t)$. Furthermore, an analogous analysis can be carried out for $t \leq 0$ that would also lead to (45) for $t \rightarrow 0^-$, but with f and $\partial_n \phi$ being the limits from the exterior of F . Finally, (ii) follows from the fact that f and $\partial_n \phi$ are continuous almost everywhere on $\partial D_{\phi_{k,\tau}} \cap F_k$ when Assumption 3.2 is satisfied. Note that in this case, the violation of Assumption 3.6 has no effect by the same continuity on $\partial D_{\phi_{k,\tau}} \cap F_k$. $\square$

Remark 3.11. It is worth noting that in reality, provided that the domains are not initialised with such an intersection, the likelihood that Assumption 3.2 or Assumption 3.6 are violated when the level-set function is updated by an optimisation algorithm is very close to zero. Put in measure-theoretic terms, Assumptions 3.2 and 3.6 are satisfied for almost all level-set functions.

3.4. Boundary truncated by perturbed domains

It turns out that in the case of several intersecting domains, there is a new case to consider — a case not occurring with a single level-set function — in which a boundary is truncated by a perturbed domain.

This situation naturally arises when integrating on interfaces between two regions. One example is when integrating on Γ_{23} in Figure 1 and computing a derivative of

$$F(\phi_2) = \int_{\partial D_{\phi_1} \cap D_{\phi_2}} f \, d\mathbf{x} \quad (48)$$

under a perturbation of the level-set function ϕ_2 . Generally, we will consider the case shown in Figure 5, integrating over a part of Π_f . Note that we allow Π_f to align with an edge $e \in E(\mathcal{P})$, which turns out to be useful later.

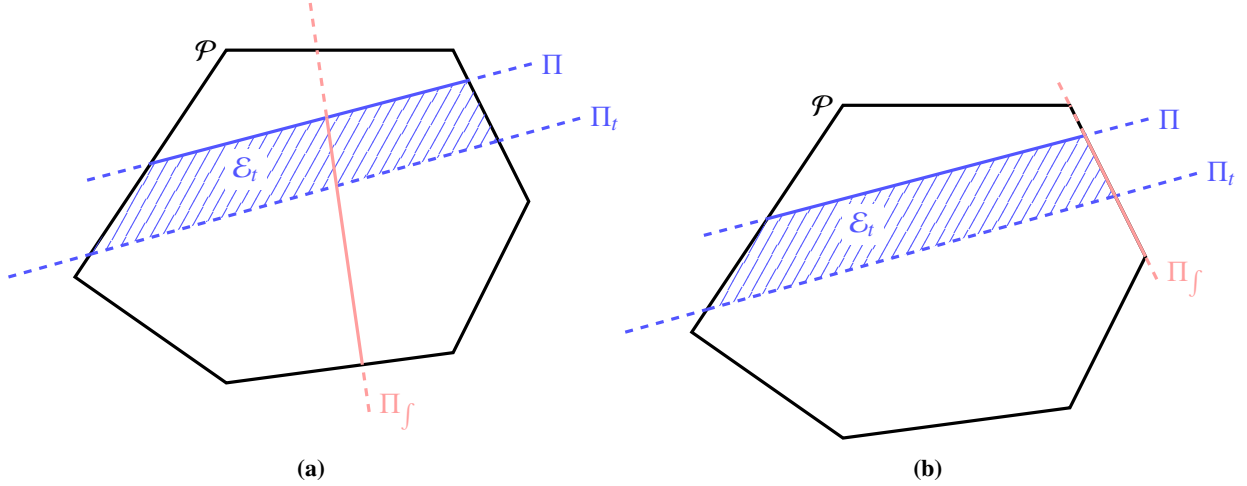


Figure 5 An illustration of the case of a plane Π_f truncated by a moving plane Π_t with: (a) Π_f inside a convex polytope $\mathcal{P}$; or (b) Π_f on an edge of $\mathcal{P}$. Conceptually, we can think of $\Pi_f \cap \mathcal{P}$ as a $(d-1)$ -dimensional convex polytope that is being cut by the plane Π_t .

Lemma 3.12. *Consider a moving plane Π_t that is locally parametrised by $\phi_t(\mathbf{x}) = \phi(\mathbf{x}) + tw(\mathbf{x}) = 0$ for some $t \in (0, t_{\max}]$, according to Assumption 3.5, and where w is a linear Lagrangian basis function. Suppose $K \in \mathcal{T}_h$ is a cell in the support of the basis function w , $\mathcal{P} \subseteq K$ is a d -dimensional convex polytope, Π_f is a plane that has a non-empty intersection with $\overline{\mathcal{P}}$, and Π_t cuts the $(d-1)$ -dimensional convex polytope $\Pi_f \cap \overline{\mathcal{P}}$ into two sub-polytopes $\Pi_f \cap \overline{\mathcal{P}} \cap H_t^\pm$, where $H_t^\pm$ are the halfspaces defined by Π_t . Then, there exists a set S and a diffeomorphism $\hat{X} : S \times (0, t) \rightarrow \mathcal{E}_t \cap \Pi_f$ such that $\hat{X}(S, \tau) = \Pi_\tau \cap \Pi_f$ for each $\tau \in (0, t)$.*

Proof. We can apply Lemma 3.4 to the case of a $(d-1)$ -dimensional convex polytope $\Pi_f \cap \overline{\mathcal{P}}$ that is cut by a plane Π_t . We thus establish the existence of $\hat{\mathbf{x}}_w \in V(\Pi_f \cap \overline{\mathcal{P}} \cap H^+)$ and $S \subset \partial(\Pi_f \cap \overline{\mathcal{P}} \cap H_\tau^-)$ such that all points $\mathbf{x}_\tau \in \Pi_\tau \cap \Pi_f$ can be parametrised by

$$\mathbf{x}_\tau = \hat{\mathbf{x}}_w + \sigma(\mathbf{x}_S - \hat{\mathbf{x}}_w) \quad (49)$$

for $\sigma \in (0, 1)$ and $\mathbf{x}_S \in S$. Note that $\hat{\mathbf{x}}_w$ is fixed for all $t \in (0, t_{\max})$ by Assumption 3.5. We may then restrict ourselves to working on the plane Π_f , i.e., the gradient operator ∇ is the intrinsic derivative on Π_f , which is given extrinsically as the surface gradient ∇_{Π_f} . From here, the proof follows similarly to Lemma 3.7. $\square$

Figure 6 shows a conceptual picture of the parametrisation $\hat{X}$ in two and three dimensions. Using Lemma 3.12 we can now parametrise the integral over $\mathcal{E}_t \cap \Pi_f$.

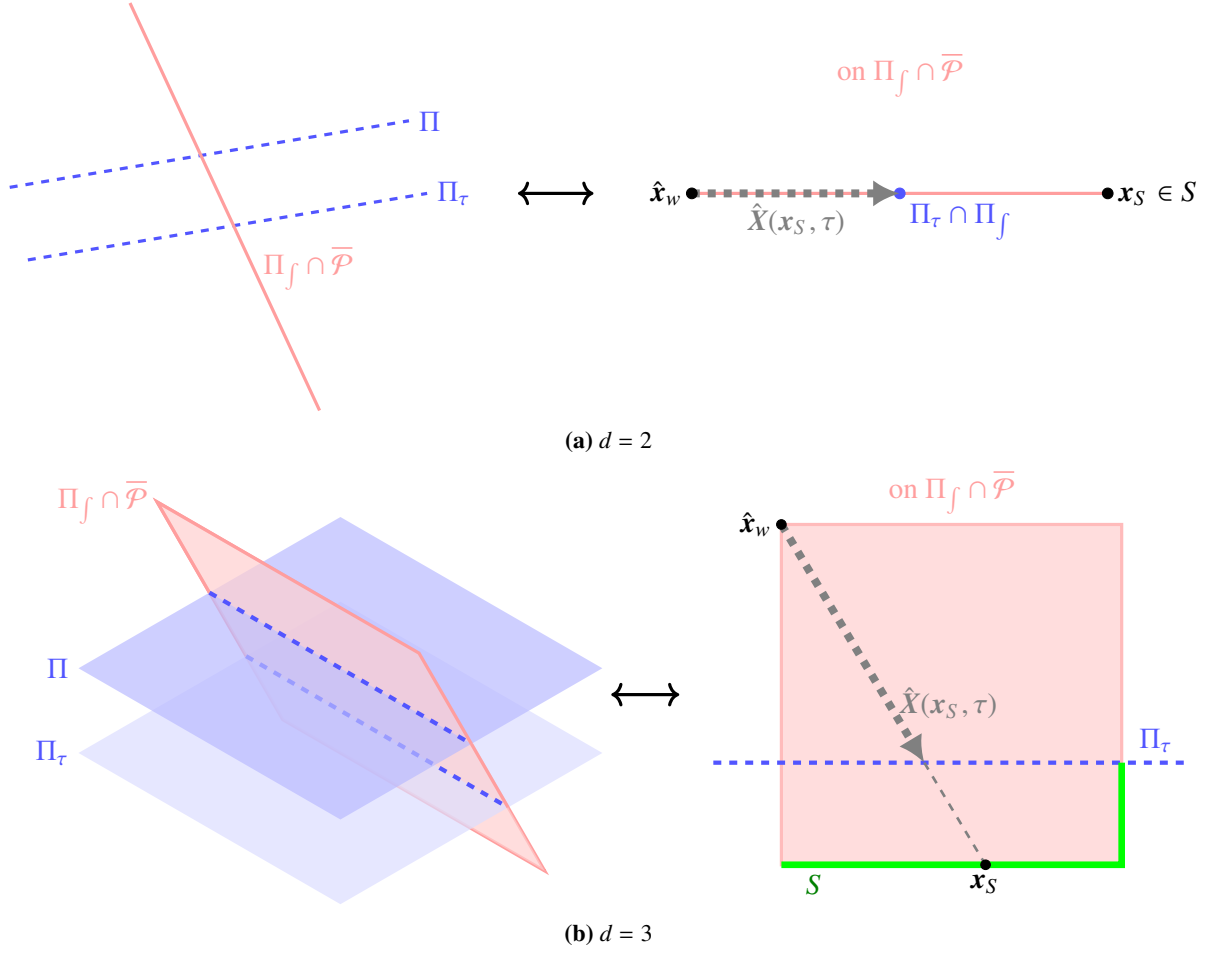


Figure 6 A conceptual picture of the parametrisation $\hat{X}$ from Lemma 3.12 in two and three dimensions. Note that $\hat{X}$ is defined only on $\Pi_f \cap \bar{\mathcal{P}}$.

Lemma 3.13. Consider a perturbation $\phi_t(\mathbf{x}) = \phi(\mathbf{x}) + tw(\mathbf{x})$ for some $t \in (0, t_{\max}]$, according to Assumption 3.5, and where w is a linear Lagrangian basis function. Let $\mathcal{P} \subseteq K \in \mathcal{T}_h$ be a convex polytope where K is in the support of w , Π_t be a moving plane parametrised locally by $\Pi_t|_K = \{\mathbf{x} \in K : \phi_t(\mathbf{x}) = 0\}$, and Π_f be a plane that has a non-empty intersection with $\Pi_t \cap \bar{\mathcal{P}}$ for all t . For $g \in C^0(\overline{\mathcal{E}_t \cap \Pi_f})$ where $\mathcal{E}_t = (H^- \setminus \bar{H}_t^-) \cap \mathcal{P}$, it holds that

$$\int_{\mathcal{E}_t \cap \Pi_f} g \, dS = \int_0^t \int_{\Pi_\tau \cap \Pi_f \cap \bar{\mathcal{P}}} g \frac{w}{|\partial_{N_\tau} \phi_\tau|} \, d\gamma \, d\tau. \quad (50)$$

where N_τ is the unit normal to Π_τ inside Π_f , outward with respect to H_τ^- .

Proof. Suppose $\hat{X}$ is the mapping from Lemma 3.12 and $\mathcal{U} \subset \mathbb{R}^{d-2}$ is the domain of a piecewise smooth parametrisation of S , again from Lemma 3.12. We define a parametrisation of $\mathcal{E}_t \cap \Pi_f$ by $X : \mathcal{U} \times (0, t) \rightarrow \mathcal{E}_t \cap \Pi_f$ as

$$X(\mathbf{u}, \tau) = \hat{X}(\mathbf{x}_S(\mathbf{u}), \tau), \quad (51)$$

where $\tau \in (0, t)$, and the mapping $\mathbf{u} \mapsto \mathbf{x}_S(\mathbf{u})$ is piecewise affine such that $X(\mathcal{U}, \tau) = \Pi_\tau \cap \Pi_f$. Note that the

parametrisation is defined only on $\Pi_f \cap \overline{\mathcal{P}}$. Furthermore, in the case $d = 2$, S consists of a single point $\mathbf{x}_S$, so the above parametrisation reduces to vectors $\mathbf{X}(\tau) = \hat{\mathbf{X}}(\mathbf{x}_S, \tau)$ of one spatial dimension, that is, effectively a real number. However, for simplicity, we use notation (51) for all dimensions d .

Similarly to Lemma 3.8, it can be shown that the Jacobian is given by

$$\det D\mathbf{X} = \pm \frac{\partial \mathbf{X}}{\partial \tau} \cdot \mathbf{N}_\tau \sigma, \quad (52)$$

where, in this case,

$$\mathbf{N}_\tau \sigma = \frac{\partial \mathbf{X}}{\partial u_1} \times \cdots \times \frac{\partial \mathbf{X}}{\partial u_{d-2}}, \quad \sigma = \left\| \frac{\partial \mathbf{X}}{\partial u_1} \times \cdots \times \frac{\partial \mathbf{X}}{\partial u_{d-2}} \right\|. \quad (53)$$

For $d = 3$, we again have

$$\mathbf{N}_\tau \sigma = \left(-\frac{\partial \mathbf{X}_2}{\partial u_1}, \frac{\partial \mathbf{X}_1}{\partial u_1} \right)^\top. \quad (54)$$

In the case of $d = 2$, we can define $\mathbf{N}_\tau \sigma = (1)^\top$, which can be thought of as a unit tangent along Π_τ . By these definitions, $\mathbf{N}_\tau$ will be the unit normal vector to Π_τ inside Π_f and outward with respect to H_τ^- .

Following very similar steps to the proof of Lemma 3.8 leads to the expression

$$|\det D\mathbf{X}| = \frac{w \circ \mathbf{X}}{\|\nabla \phi_\tau \circ \mathbf{X}\|} \sigma, \quad (55)$$

where ∇ is the gradient on $\Pi_f \cap \overline{\mathcal{P}}$ and w and ϕ_τ are restricted to $\Pi_f \cap \overline{\mathcal{P}}$. (50) then follows by similar arguments to those given at the end of the proof of Lemma 3.8. $\square$

Remark 3.14. The $\mathbf{N}_\tau$ can be written extrinsically in terms of the normal $\mathbf{n}_\tau$ of Π_τ and the normal $\mathbf{n}_f$ of Π_f . In particular, $\mathbf{N}_\tau = \pm \tilde{\mathbf{N}}_\tau / \|\tilde{\mathbf{N}}_\tau\|$ where $\tilde{\mathbf{N}}_\tau = \mathbf{n}_\tau - (\mathbf{n}_\tau \cdot \mathbf{n}_f) \mathbf{n}_f$ and the sign is chosen such that $\mathbf{N}_\tau$ points outwards from H_τ^- .

Using Lemma 3.13, we establish the following result.

Theorem 3.15. *Given a perturbation $\phi_{k,t}(\mathbf{x}) = \phi_k(\mathbf{x}) + t w(\mathbf{x})$, where w is a linear Lagrangian basis function, the directional semiderivative of boundary integral*

$$J(\phi_k) = \int_{\partial D_{\phi_i} \cap F_i} g \, dS, \quad (56)$$

for $i \neq k$ and $D_{\phi_k} \in \mathcal{D}_p$ satisfying Assumption 3.6 and $g \in C^0(\overline{\mathcal{T}}_h)$, satisfies

$$dJ(\phi_k)(w) = \lim_{t \rightarrow 0^+} \frac{1}{t} (J(\phi_{k,t}) - J(\phi_k)) = \int_{\partial D_{\phi_i} \cap \partial D_{\phi_k} \cap F_{ik}} g \frac{w}{|\partial_{\mathbf{N}^i} \phi_k|} \, d\gamma, \quad (57)$$

where $\mathbf{N}^i$ is the unit normal to ∂D_{ϕ_k} inside ∂D_{ϕ_i} , outward with respect to D_{ϕ_k} (see Remark 3.14). If $D_{\phi_k} \in \mathcal{D}_c$, the right-hand side is of opposite sign.

- (i) If Assumption 3.2 is violated, then g and $\partial_{\mathbf{N}^i} \phi$ are the limits of these functions from the interior of $\partial D_{\phi_i} \cap F_i$ whenever these quantities possess jump discontinuities on $\partial D_{\phi_i} \cap \partial D_{\phi_k} \cap F_{ik}$.
- (ii) If Assumption 3.2 is satisfied, the semiderivatives $t \rightarrow 0^-$ and $t \rightarrow 0^+$ agree.

Proof. Suppose $D_{\phi_k} \in \mathcal{D}_p$ and let $t \in (0, t_{\max}]$ according to Assumption 3.5. Then for $E_t = (D_{\phi_k} \setminus \overline{D_{\phi_{k,t}}}) \cap F_{ik}$, we have

$$\frac{1}{t}(J(\phi_{k,t}) - J(\phi_k)) = \frac{1}{t} \left(\int_{\partial D_{\phi_i} \cap D_{\phi_{k,t}} \cap F_{ik}} g \, d\mathbf{x} - \int_{\partial D_{\phi_i} \cap D_{\phi_k} \cap F_{ik}} g \, d\mathbf{x} \right) = -\frac{1}{t} \sum_{K \in \mathcal{T}_h} \int_{\partial D_{\phi_i} \cap E_t \cap K} g \, dS. \quad (58)$$

To apply Lemma (3.13) to the right side of expression (58), we will first need to identify its integration domain $\partial D_{\phi_i} \cap E_t \cap K$ with the $\mathcal{E}_t \cap \Pi_{\mathcal{J}}$ of formula (50). For this, select $\mathcal{E}_t = E_t \cap K$ and $\Pi_{\mathcal{J}} \equiv \partial D_{\phi_i} \cap K$, which means that, indeed, $\mathcal{E}_t \cap \Pi_{\mathcal{J}} = \partial D_{\phi_i} \cap E_t \cap K$. Note that the identification $\mathcal{E}_t = E_t \cap K$ implies, by the definitions of $\mathcal{E}_t$ and E_t , that

$$\mathcal{E}_t \equiv (H^- \setminus \overline{H_t^-}) \cap \mathcal{P} = E_t \cap K \equiv (D_{\phi_k} \setminus \overline{D_{\phi_{k,t}}}) \cap F_{ik} \cap K, \quad (59)$$

which means that we can identify $H^- \setminus \overline{H_t^-} = (D_{\phi_k} \setminus \overline{D_{\phi_{k,t}}}) \cap K$ and $\mathcal{P} = F_{ik} \cap K$.

Next, for the integration domain on the right side of formula (50), that is $\Pi_{\tau} \cap \Pi_{\mathcal{J}} \cap \overline{\mathcal{P}}$, note that since $H^- \setminus \overline{H_t^-} = (D_{\phi_k} \setminus \overline{D_{\phi_{k,t}}}) \cap K$, it follows that $\Pi_{\tau} = \partial D_{\phi_{k,\tau}} \cap K$. All together, these definitions yield that $\Pi_{\tau} \cap \Pi_{\mathcal{J}} \cap \overline{\mathcal{P}} = \partial D_{\phi_{k,\tau}} \cap \partial D_{\phi_i} \cap F_{ik} \cap K$.

Then, applying Lemma 3.13, we find

$$\frac{1}{t}(J(\phi_{k,t}) - J(\phi_k)) = -\frac{1}{t} \sum_{K \in \mathcal{T}_h} \int_0^t \int_{\partial D_{\phi_i} \cap \partial D_{\phi_{k,\tau}} \cap F_{ik} \cap K} g \frac{w}{|\partial_{N^i} \phi_{\tau}|} \, d\gamma \, d\tau, \quad (60)$$

where we use Assumption 3.6 to ensure that the intersection $\partial D_{\phi_i} \cap \partial D_{\phi_{k,\tau}}$ does not change to become empty or non-empty for some t depending on whether we take $t \rightarrow 0^+$ or $t \rightarrow 0^-$. Taking the limit gives (57). The case for $D_{\phi_k} \in \mathcal{D}_c$ follows similarly with the addition of a chain-rule computation because $D_{\phi_k}^c = D_{-\phi_k}$.

The cases of (i) and (ii) follow by similar arguments to those in the proof of Theorem 3.10. $\square$

Remark 3.16. For $d = 2$, 57 is evaluated pointwise. Namely, for $D_{\phi_k} \in \mathcal{D}_p$,

$$dJ(\phi_k)(w) = \sum_{K \in \mathcal{T}_h} \left(g \frac{w}{|\partial_{N^i} \phi_k|} \right) \Big|_{\partial D_{\phi_i} \cap \partial D_{\phi_k} \cap F_{ik} \cap K}, \quad (61)$$

where $\partial D_{\phi_i} \cap \partial D_{\phi_k} \cap F_{ik}$ is the set of intersection points of $\partial D_{\phi_i} \cap \partial D_{\phi_k}$ in F_{ik} .

3.5. Boundary integrals

The final case considered is the directional derivative of functionals of the form

$$J(\phi_k) = \int_{\partial D_{\phi_k} \cap F_k} g \, dS. \quad (62)$$

Note that this case is different from (56), because now ϕ_k appears in ∂D_{ϕ_k} rather than implicitly inside F_i . An example of this case is when integrating on Γ_{23} in Figure 1 and computing the derivative of

$$F(\phi_1) = \int_{\partial D_{\phi_1} \cap D_{\phi_2}} f \, d\mathbf{x} \quad (63)$$

under a perturbation of the level-set function ϕ_1 .

We begin by establishing the following integration by parts result.

Lemma 3.17. Consider a perturbation $\phi_{k,t}(\mathbf{x}) = \phi_k(\mathbf{x}) + tw(\mathbf{x})$ for some $t \in (0, t_{max}]$, according to Assumption 3.5, and where w is a linear Lagrangian basis function. For $g \in C^1(\mathcal{T}_h)$ and $\Theta \in L^\infty(E_t)^d$ such that $\Theta|_{E_t \cap K} \in C^1(\overline{E_t \cap K})^d$ for $K \in \mathcal{T}_h$ where $E_t = (D_{\phi_k} \setminus \overline{D_{\phi_k,t}}) \cap F_k$. Then, it holds that

$$\begin{aligned} \sum_{K \in \mathcal{T}_h} \int_{E_t \cap K} [g \nabla \cdot \Theta + \Theta \cdot \nabla g] d\mathbf{x} &= \int_{\partial E_t \setminus (F_k^\partial \cup \partial D)} \mathbf{n} \cdot \Theta g dS \\ &+ \int_0^t \int_{\partial D \cap \partial D_{\phi_k,\tau} \cap F_k} \mathbf{n}_{\partial D} \cdot \Theta g \frac{w}{|\partial_{N_\tau^D} \phi_{k,\tau}|} d\gamma d\tau \\ &+ \sum_{D_{\phi_i} \in \mathcal{D} \setminus \{D_{\phi_k}\}} \int_0^t \int_{\partial D_{\phi_i} \cap \partial D_{\phi_k,\tau} \cap F_{ik}} \pm \mathbf{n}_{\phi_i} \cdot \Theta g \frac{w}{|\partial_{N_\tau^i} \phi_{k,\tau}|} d\gamma d\tau \\ &+ \sum_{S \in \mathcal{F}_h \setminus \partial D} \int_0^t \int_{S \cap F_k \cap \partial D_{\phi_k,\tau}} \llbracket \Theta g \rrbracket \frac{w}{|\partial_{N_\tau^S} \phi_{k,\tau}|} d\gamma d\tau, \end{aligned} \quad (64)$$

where

$$\llbracket \Theta g \rrbracket = \mathbf{n}_{S,1} \cdot \Theta_1 g_1 + \mathbf{n}_{S,2} \cdot \Theta_2 g_2, \quad (65)$$

in which $\mathbf{n}_{S,1}$ and $\mathbf{n}_{S,2} = -\mathbf{n}_{S,1}$ are the unit normals to the plane $S \in \mathcal{F}_h \setminus \partial D$; $\mathbf{n}$, $\mathbf{n}_{\partial D}$, and $\mathbf{n}_{\phi_i}$ are the unit normals to $\partial E_t \setminus (F_k^\partial \cup \partial D)$, ∂D , and ∂D_{ϕ_i} outward with respect to E_t , D and ∂D_{ϕ_i} , respectively; N_τ^D , N_τ^i , and N_τ^S are the unit normals to $\partial D_{\phi_k,\tau}$ inside ∂D , ∂D_{ϕ_i} , and S , respectively, outward with respect to $D_{\phi_k,\tau}$; and $F_k^\partial := \bigcup_{A \in \mathcal{D} \setminus D_{\phi_k}} \partial A$ is the union of all boundaries except ∂D_{ϕ_k} . The sign in the third term is positive if $D_{\phi_i} \in \mathcal{D}_p$ and negative if $D_{\phi_i} \in \mathcal{D}_c$.

Proof. Let $E_t = (D_{\phi_k} \setminus \overline{D_{\phi_k,t}}) \cap F_k$ and $\hat{E}_t = (D_{\phi_k} \setminus \overline{D_{\phi_k,t}}) \cap F_{ik}$. Then, by integration by parts applied to $g \nabla \cdot \Theta$ on each $E_t \cap K$ for $K \in \mathcal{T}_h$, we have

$$\begin{aligned} \sum_{K \in \mathcal{T}_h} \int_{E_t \cap K} [g \nabla \cdot \Theta + \Theta \cdot \nabla g] d\mathbf{x} &= \int_{\partial E_t \setminus (F_k^\partial \cup \partial D)} \mathbf{n} \cdot \Theta g dS + \sum_{K \in \mathcal{T}_h} \int_{\partial D \cap E_t \cap K} \mathbf{n}_{\partial D} \cdot \Theta g dS \\ &+ \sum_{D_{\phi_i} \in \mathcal{D} \setminus \{D_{\phi_k}\}} \sum_{K \in \mathcal{T}_h} \int_{\partial D_{\phi_i} \cap \hat{E}_t \cap K} \pm \mathbf{n}_{\phi_i} \cdot \Theta g dS + \sum_{S \in \mathcal{F}_h \setminus \partial D} \int_{S \cap E_t} \llbracket \Theta g \rrbracket dS, \end{aligned} \quad (66)$$

where the first term on the right-hand side corresponds to the part of ∂E_t that aligns with ∂D_{ϕ_k} and $\partial D_{\phi_k,t}$; the second is the part that aligns with the boundary of the hold-all domain; the third is the part that aligns with any of the boundaries ∂D_{ϕ_i} , $i \neq k$, where the normal is outward with respect to D_{ϕ_i} and has negative sign if $D_{\phi_i} \in \mathcal{D}_c$; and the final term comes from the jump, defined in expression (65), across a mesh facet in $\mathcal{F}_h$. Note that some of the elements of the sum in the third term will vanish because ∂D_{ϕ_i} will have an empty intersection with $\hat{E}_t$.

Identifying $E_t \cap K \equiv \mathcal{E}_t$ and $\partial D \cap K \equiv \Pi_\mathcal{J}$ and $\mathcal{P} \equiv F_k \cap K$ and using the fact that ∂D_{ϕ_i} is planar in K , we can apply Lemma 3.13 to the second term on the right-hand side of (66) to obtain

$$\int_{\partial D \cap E_t \cap K} \mathbf{n}_{\partial D} \cdot \Theta g dS = \int_0^t \int_{\partial D \cap \partial D_{\phi_k,\tau} \cap F_k \cap K} \mathbf{n}_{\partial D} \cdot \Theta g \frac{w}{|\partial_{N_\tau^D} \phi_{k,\tau}|} d\gamma d\tau, \quad (67)$$

where N_τ^D is the normal to $\partial D_{\phi_k,\tau}$ inside ∂D , outward with respect to $D_{\phi_k,\tau}$.

Next, identifying $\partial D_{\phi_i} \cap K \equiv \Pi_\mathcal{J}$ and $\mathcal{P} \equiv F_{ik} \cap K$, we can apply Lemma 3.13 to the third term on the right-hand side of (66) to obtain

$$\int_{\partial D_{\phi_i} \cap \hat{E}_t \cap K} \mathbf{n}_{\phi_i} \cdot \Theta g dS = \int_0^t \int_{\partial D_{\phi_i} \cap \partial D_{\phi_k,\tau} \cap F_{ik} \cap K} \mathbf{n}_{\phi_i} \cdot \Theta g \frac{w}{|\partial_{N_\tau^i} \phi_{k,\tau}|} d\gamma d\tau, \quad (68)$$

where $\mathbf{N}_\tau^i$ is the normal to $\partial D_{\phi_k, \tau}$ inside ∂D_{ϕ_i} , outward with respect to $D_{\phi_k, \tau}$.

Finally, identifying $S \equiv \Pi_\mathcal{F}$ and $\mathcal{P} \equiv F_k \cap K$, we may also apply Lemma 3.13 to the final term on the right-hand side of (66) to obtain

$$\begin{aligned} \int_{S \cap E_t} \llbracket \Theta g \rrbracket dS &= \int_{S \cap E_t \cap \bar{K}_1} \mathbf{n}_1 \cdot \Theta_1 g_1 dS + \int_{S \cap E_t \cap \bar{K}_2} \mathbf{n}_2 \cdot \Theta_2 g_2 dS \\ &= \int_0^t \int_{S \cap \partial D_{\phi_k, \tau} \cap F_k \cap \bar{K}_1} \mathbf{n}_1 \cdot \Theta_1 g_1 \frac{w}{|\partial_{N_\tau^S} \phi_{k, \tau}|} d\gamma d\tau + \int_0^t \int_{S \cap \partial D_{\phi_k, \tau} \cap F_k \cap \bar{K}_2} \mathbf{n}_2 \cdot \Theta_2 g_2 \frac{w}{|\partial_{N_\tau^S} \phi_{k, \tau}|} d\gamma d\tau \\ &= \int_0^t \int_{S \cap \partial D_{\phi_k, \tau} \cap F_k} \llbracket \Theta g \rrbracket \frac{w}{|\partial_{N_\tau^S} \phi_{k, \tau}|} d\gamma d\tau, \end{aligned} \quad (69)$$

where N_τ^S is the normal to $\partial D_{\phi_k, \tau}$ inside S , outward with respect to $D_{\phi_k, \tau}$. In the first equality, we split $S \cap E_t$ into $S \cap E_t \cap \bar{K}_1$ and $S \cap E_t \cap \bar{K}_2$ where K_1 and K_2 are the two cells that share the facet S . In the second equality, we apply Lemma 3.13 to each of these integrals, identifying S as $\Pi_\mathcal{F}$ and $\partial D_{\phi_k, \tau}$ as Π_τ . In the final equality, we combine the two integrals into a single integral on $S \cap \partial D_{\phi_k, \tau}$ using the fact that N_τ^S is the same on $S \cap \bar{K}_1$ or $S \cap \bar{K}_2$ by continuity of $\partial D_{\phi_k, \tau}$.

Substituting (67), (68), and (69) into (66) and rewriting the sums over $K \in \mathcal{T}_h$ completes the proof. $\square$

Thanks to the machinery developed above and in the previous sections, we may state the following result, whose proof follows steps similar to the proof of Theorem 6.11 in Berggren [27].

Theorem 3.18. *Given a perturbation $\phi_{k,t}(\mathbf{x}) = \phi_k(\mathbf{x}) + tw(\mathbf{x})$, where w is a linear Lagrangian basis function, the directional semiderivative of a boundary integral*

$$J(\phi_k) = \int_{\partial D_{\phi_k} \cap F_k} g dS, \quad (70)$$

for D_{ϕ_k} satisfying Assumption 3.2 and Assumption 3.6 and $g \in C^1(\bar{\mathcal{T}}_h)$, satisfies

$$\begin{aligned} dJ(\phi_k)(w) &= \lim_{t \rightarrow 0^+} \frac{1}{t} (J(\phi_{k,t}) - J(\phi_k)) \\ &= - \int_{\partial D_{\phi_k} \cap F_k} \frac{\partial g}{\partial \mathbf{n}_{\phi_k}} \frac{w}{|\partial_{\mathbf{n}_{\phi_k}} \phi_k|} dS + \sum_{S \in \mathcal{F}_h \setminus \partial D} \int_{S \cap F_k \cap \partial D_{\phi_k}} \llbracket \mathbf{n}_{\phi_k} g \rrbracket \frac{w}{|\partial_{N^S} \phi_k|} d\gamma \\ &\quad + \int_{\partial D \cap \partial D_{\phi_k} \cap F_k} \mathbf{n}_{\partial D} \cdot \mathbf{n}_{\phi_k} g \frac{w}{|\partial_{N^D} \phi_k|} d\gamma + \sum_{D_{\phi_i} \in \mathcal{D} \setminus \{D_{\phi_k}\}} \int_{\partial D_{\phi_i} \cap \partial D_{\phi_k} \cap F_{ik}} \pm \mathbf{n}_{\phi_i} \cdot \mathbf{n}_{\phi_k} g \frac{w}{|\partial_{N^i} \phi_k|} d\gamma, \end{aligned} \quad (71)$$

where $\mathbf{n}_{\phi_k}$ is the outward normal to ∂D_{ϕ_k} ; $\llbracket \mathbf{n}_{\phi_k} g \rrbracket = \mathbf{n}_{S,1} \cdot \mathbf{n}_{\phi_k,1} g_1 + \mathbf{n}_{S,2} \cdot \mathbf{n}_{\phi_k,2} g_2$ in which $\mathbf{n}_{S,j}$ are the outward normals to the plane $S \in \mathcal{F}_h$ and $\mathbf{n}_{\phi_k,j}$ are the outward normals on ∂D_{ϕ_k} on either side of S ; $\mathbf{n}_{\partial D}$ and $\mathbf{n}_{\phi_i}$ are the unit normals to ∂D and ∂D_{ϕ_i} outward with respect to D and ∂D_{ϕ_i} , respectively; N^D , N^i , and N^S are the outward unit normals to ∂D_{ϕ_k} inside ∂D , ∂D_{ϕ_i} , and S , respectively. The sign in the final term is positive if $D_{\phi_i} \in \mathcal{D}_p$ and negative if $D_{\phi_i} \in \mathcal{D}_c$. In addition, the index i in the final term corresponds to $D_{\phi_i} \in \mathcal{D} \setminus D_{\phi_k}$ under the sum.

Proof. Suppose $\phi_t(\mathbf{x}) = \phi(\mathbf{x}) + tw(\mathbf{x})$ for some $t \in (0, t_{max}]$, according to Assumption 3.5 and suppose that we have $E_t = (D_{\phi_k} \setminus \overline{D_{\phi_{k,t}}}) \cap F_k$. Owing to Assumption 3.2, almost everywhere on $\partial D_{\phi_k, \tau}$, for $\tau \in [0, t]$, the outwards normal can be written as

$$\mathbf{n}_{\phi_k, \tau} = \frac{\nabla \phi_{k, \tau}}{\|\nabla \phi_{k, \tau}\|} \Big|_{\partial D_{\phi_k, \tau}}. \quad (72)$$

Suppose $K \in \mathcal{T}_h$ has a non-empty intersection with E_t . Identifying $E_t \cap K \equiv \mathcal{E}_t$, we may apply Lemma 3.7 to obtain a diffeomorphism $\hat{X}$ on $\mathcal{E}_t$, or equivalently on $E_t \cap K$. Using this, we can define a function $\Psi_K^{(t)} : E_t \cap K \rightarrow \mathbb{R}^d$ such that

$$\Psi_K^{(t)}(\hat{X}(\mathbf{x}_S, \tau)) = \frac{\nabla \phi_{k,\tau}(\hat{X}(\mathbf{x}_S, \tau))}{\|\nabla \phi_{k,\tau}(\hat{X}(\mathbf{x}_S, \tau))\|}, \quad \forall (\mathbf{x}_S, \tau) \in S \times (0, t). \quad (73)$$

We may then define $\Psi^{(t)} \in L^\infty(E_t)^d$ such that $\Psi^{(t)}|_{E_t \cap K} = \Psi_K^{(t)}$. By construction, we then have

$$\Psi^{(t)}|_{\partial D_{\phi_k,\tau} \cap K} = \mathbf{n}_{\phi_k,\tau}|_{\partial D_{\phi_k,\tau} \cap K}, \quad \forall \tau \in (0, t). \quad (74)$$

The function $\Psi^{(t)}$ now fulfils the conditions for Θ in Lemma 3.17, which implies that for $g \in \prod_{K \in \mathcal{T}_h} C^1(\bar{K})$,

$$\begin{aligned} \sum_{K \in \mathcal{T}_h} \int_{E_t \cap K} [g \nabla \cdot \Psi^{(t)} + \Psi^{(t)} \cdot \nabla g] \, d\mathbf{x} &= \int_{\partial E_t \setminus (F_k^\partial \cup \partial D)} \mathbf{n} \cdot \Psi^{(t)} g \, dS \\ &+ \int_0^t \int_{\partial D \cap \partial D_{\phi_k,\tau} \cap F_k} \mathbf{n}_{\partial D} \cdot \Psi^{(t)} g \frac{w}{|\partial_{N_\tau^D} \phi_{k,\tau}|} \, d\gamma \, d\tau \\ &+ \sum_{D_{\phi_i} \in \mathcal{D} \setminus \{D_{\phi_k}\}} \int_0^t \int_{\partial D_{\phi_i} \cap \partial D_{\phi_k,\tau} \cap F_{ik}} \pm \mathbf{n}_{\phi_i} \cdot \Psi^{(t)} g \frac{w}{|\partial_{N_\tau^S} \phi_{k,\tau}|} \, d\gamma \, d\tau \\ &+ \sum_{S \in \mathcal{F}_h \setminus \partial D} \int_0^t \int_{S \cap F_k \cap \partial D_{\phi_k,\tau}} \llbracket \Psi^{(t)} g \rrbracket \frac{w}{|\partial_{N_\tau^S} \phi_{k,\tau}|} \, d\gamma \, d\tau, \end{aligned} \quad (75)$$

where we use Assumption 3.6 to ensure that the intersection $\partial D_{\phi_i} \cap \partial D_{\phi_k,\tau}$ does not change to become empty or non-empty for some i depending on whether we take $t \rightarrow 0^+$ or $t \rightarrow 0^-$.

Using Lemma 3.8 and (74), we can rewrite each term in the sum in left-hand side of (75) as

$$\begin{aligned} \int_{E_t \cap K} [g \nabla \cdot \Psi^{(t)} + \Psi^{(t)} \cdot \nabla g] \, d\mathbf{x} &= \int_0^t \int_{\partial D_{\phi_k,\tau} \cap F_k \cap K} [g \nabla \cdot \mathbf{n}_{\phi_k,\tau} + \mathbf{n}_{\phi_k,\tau} \cdot \nabla g] \frac{w}{|\partial_{\mathbf{n}} \phi_{k,\tau}|} \, dS \, d\tau \\ &= \int_0^t \int_{\partial D_{\phi_k,\tau} \cap F_k \cap K} \frac{\partial g}{\partial \mathbf{n}_{\phi_k,\tau}} \frac{w}{|\partial_{\mathbf{n}} \phi_{k,\tau}|} \, dS \, d\tau, \end{aligned} \quad (76)$$

where we have identified $E_t \cap K \equiv \mathcal{E}_t$, $\Pi_\tau \equiv \partial D_{\phi_k,\tau} \cap K$, and $\mathcal{P} \equiv F_k \cap K$, and used that $\nabla \cdot \mathbf{n}_{\phi_k,\tau}$ is the mean curvature that vanishes because $\partial D_{\phi_k,\tau} \cap K$ is planar.

By construction, the first term on the right-hand side of (75) consists only of the boundaries ∂D_{ϕ_k} and $\partial D_{\phi_k,t}$ and the integrand satisfies

$$\mathbf{n} \cdot \Psi^{(t)} g|_{\partial E_t \setminus (F_k^\partial \cup \partial D)} = \begin{cases} g & \text{on } \partial D_{\phi_k} \cap E_t \\ -g & \text{on } \partial D_{\phi_k,t} \cap E_t \end{cases}, \quad (77)$$

where the minus sign comes from the fact that $\mathbf{n}$ is the outward normal to E_t , which aligns with $\Psi^{(t)}$ on ∂D_{ϕ_k} but points in the opposite direction on $\partial D_{\phi_k,t}$. Using this, we can write the first term on the right-hand side of (75) as

$$\int_{\partial E_t \setminus (F_k^\partial \cup \partial D)} \mathbf{n} \cdot \Psi^{(t)} g \, dS = \int_{\partial D_{\phi_k} \cap E_t} g \, dS - \int_{\partial D_{\phi_k,t} \cap E_t} g \, dS = \int_{\partial D_{\phi_k} \cap F_k} g \, dS - \int_{\partial D_{\phi_k,t} \cap F_k} g \, dS, \quad (78)$$

by definition of E_t .

Substituting (76) and (78) into (75) and rearranging, we obtain

$$\begin{aligned}
& \int_{\partial D_{\phi_k} \cap F_k} g \, dS - \int_{\partial D_{\phi_k, t} \cap F_k} g \, dS = \\
& = \int_0^t \int_{\partial D_{\phi_k, \tau} \cap F_k} \frac{\partial g}{\partial \mathbf{n}_{\phi_k, \tau}} \frac{w}{|\partial \mathbf{n}_{\phi_k, \tau} \phi_{k, \tau}|} \, dS \, d\tau - \sum_{S \in \mathcal{F}_h \setminus \partial D} \int_0^t \int_{S \cap F_k \cap \partial D_{\phi_k, \tau}} \llbracket \mathbf{n}_{\phi_k, \tau} g \rrbracket \frac{w}{|\partial_{N_\tau^S} \phi_{k, \tau}|} \, d\gamma \, d\tau \\
& \quad - \int_0^t \int_{\partial D \cap \partial D_{\phi_k, \tau} \cap F_k} \mathbf{n}_{\partial D} \cdot \mathbf{n}_{\phi_k, \tau} g \frac{w}{|\partial_{N_\tau^D} \phi_{k, \tau}|} \, d\gamma \, d\tau \\
& \quad - \sum_{D_{\phi_i} \in \mathcal{D} \setminus \{D_{\phi_k}\}} \int_0^t \int_{\partial D_{\phi_i} \cap \partial D_{\phi_k, \tau} \cap F_{ik}} \pm \mathbf{n}_{\phi_i} \cdot \mathbf{n}_{\phi_k, \tau} g \frac{w}{|\partial_{N_\tau^i} \phi_{k, \tau}|} \, d\gamma \, d\tau.
\end{aligned} \tag{79}$$

Dividing by t and taking the limit completes the proof. $\square$

Remark 3.19. As discussed in Remark 3.16, when $d = 2$ the expressions on the intersections of boundaries become pointwise evaluations.

Remark 3.20. To avoid introducing additional geometric objects that may lose meaning in a general $\mathbb{R}^d$ setting, we do not use the triple product as in Lemma 6.10 and Theorem 6.11 of Berggren [27]. Note that if the triple product were used, the last three terms in (71) would have a factor of -1 , and in the case of a single level-set function we recover Theorem 3 of Wegert et al. [25].

Remark 3.21. Theorem 3.18 was proven under the assumption that $g \in C^1(\overline{\mathcal{T}_h})$, independent of ϕ_k . For the derivation of one of directional derivatives in Section 3.6 below, we will, however, encounter a case when the integrand also is a function of ϕ_k , that is, when the objective function reads

$$\hat{J}(\phi_k) = \int_{\partial D_{\phi_k} \cap F_k} g(\phi_k) \, dS. \tag{80}$$

Denote by $g(t)$ the function $t \mapsto g(\phi_{k, t})$. Provided that $t \mapsto g(t)$ and $t \mapsto g'(t)$ are continuous and $g(t), g'(t) \in C^1(\overline{\mathcal{T}_h})$ in the vicinity of $t = 0$, we may use the product rule of differentiation to conclude that

$$d\hat{J}(\phi_k)(w) = \int_{\partial D_{\phi_k} \cap F_k} g' \, dS + dJ(\phi_k)(w), \tag{81}$$

where $dJ(\phi_k)(w)$ is given by expression (71).

3.6. Shape calculus example

To demonstrate how these derivatives can be used in the context of shape and topology optimisation, we return to the example discussed in Section 2. In particular, consider minimising the total thermal dissipation for both phases, that is, the quantity

$$F(U, U; \phi_1, \phi_2) = (\mathbf{q}_1(u_1), \nabla u_1)_{\Omega_1} + (\mathbf{q}_2(u_2), \nabla u_2)_{\Omega_2}, \tag{82}$$

quadratic in $U = [u_1, u_2]$. For the optimisation, we introduce the *reduced objective function*

$$J(\phi_1, \phi_2) = F(U(\phi), U(\phi)(\phi_1, \phi_2); \phi_1, \phi_2), \tag{83}$$

where $\phi = [\phi_1, \phi_2]$ and

$$\begin{aligned} U &\in \mathcal{V}_{1,h} \times \mathcal{V}_{2,h} \quad \text{such that} \\ A(U, V; \phi_1, \phi_2) &= l(V; \phi_1, \phi_2) \quad \forall V \in \mathcal{V}_{1,h} \times \mathcal{V}_{2,h}, \end{aligned} \quad (84)$$

and consider the optimisation problem

$$\min_{(\phi_1, \phi_2) \in \mathcal{V}_h^2} J(\phi_1, \phi_2). \quad (85)$$

For this example, assume that the flux is given by $\mathbf{q}_i = \mathbf{A}_i \nabla u_i$, $i = 1, 2$, where matrices $\mathbf{A}_i$ are symmetric, positive definite, and constant on $\overline{D}_{\phi_i}$. For the weighting parameters κ_1 and κ_2 , we take inspiration from Claus and Kerfriden [22] and define these on Γ_{ij} to be

$$\kappa_1 = \frac{c_j}{c_i + c_j}, \quad \kappa_2 = \frac{c_i}{c_i + c_j}, \quad (86)$$

where $c_i = \mathbf{A}_i : (\mathbf{n} \otimes \mathbf{n}) = \mathbf{n} \cdot \mathbf{A}_i \mathbf{n}$, which accounts for the possible material anisotropy.

Next, recall that $\Omega_1(\phi_1, \phi_2) = D_{\phi_1} \cap \overline{D}_{\phi_2}^c$, $\Omega_2(\phi_1, \phi_2) = D_{\phi_1} \cap D_{\phi_2}$, and $\Gamma_{12}(\phi_1, \phi_2) = D_{\phi_1} \cap \partial D_{\phi_2}$ (see Fig. 1). As a result, F , A and l can be written as

$$F(U, U; \phi_1, \phi_2) = (\mathbf{A}_1 \nabla u_1, \nabla u_1)_{D_{\phi_1} \cap \overline{D}_{\phi_2}^c} + (\mathbf{A}_2 \nabla u_2, \nabla u_2)_{D_{\phi_1} \cap D_{\phi_2}}, \quad (87)$$

$$\begin{aligned} A(U, V; \phi_1, \phi_2) &= (\mathbf{A}_1 \nabla v_1, \nabla v_1)_{D_{\phi_1} \cap \overline{D}_{\phi_2}^c} + (\mathbf{A}_2 \nabla u_2, \nabla v_2)_{D_{\phi_1} \cap D_{\phi_2}} + j([u_1, u_2], [v_1, v_2]) \\ &\quad - (\{\mathbf{A} \nabla u\}, \llbracket v \rrbracket)_{D_{\phi_1} \cap \partial D_{\phi_2}} - (\{\mathbf{A} \nabla v\}, \llbracket u \rrbracket)_{D_{\phi_1} \cap \partial D_{\phi_2}} \\ &\quad + (\mu \llbracket u \rrbracket, \llbracket v \rrbracket)_{D_{\phi_1} \cap \partial D_{\phi_2}}, \end{aligned} \quad (88)$$

and

$$l(V; \phi_1, \phi_2) = \sum_i (g, v_i)_{\Gamma_N}. \quad (89)$$

To differentiate J with respect to ϕ_1 and ϕ_2 , we use C  a's formal method [38], which is a version of the method of Lagrange multipliers applied to shape optimisation problems. Since this method also is encoded in our later use of forward mode automatic differentiation for shape optimisation, we will give a brief review of the approach for the current optimisation problem (85).

The method is based on a Lagrangian functional constructed by subtracting the residual of the state equation from the the objective function, that is,

$$\mathcal{L}(P, \Lambda; \phi_1, \phi_2) = F(P, P; \phi_1, \phi_2) - A(P, \Lambda; \phi_1, \phi_2) + l(\Lambda; \phi_1, \phi_2), \quad (90)$$

where $P = [p_1, p_2]$ and $\Lambda = [\lambda_1, \lambda_2]$.

Since the Lagrangian is linear in Λ (the test function in the variational form), differentiation of the Lagrangian with respect to Λ in the direction W generates the state equation residual, that is,

$$d_\Lambda \mathcal{L}(P, \Lambda; \phi_1, \phi_2)(W) \equiv \frac{d}{dt} \mathcal{L}(P, \Lambda + tW; \phi_1, \phi_2) \Big|_{t=0} = -A(P, W; \phi_1, \phi_2) + l(W; \phi_1, \phi), \quad (91)$$

which vanishes for each $W \in \mathcal{V}_{1,h} \times \mathcal{V}_{2,h}$ when $P = U := [u_1, u_2]$ due to state equation (84).

Moreover, differentiation of the Lagrangian with respect to P in the direction R generates a residual of the so-called adjoint equation, that is,

$$\begin{aligned} d_P \mathcal{L}(P, \Lambda; \phi_1, \phi_2)(R) &\equiv \frac{d}{dt} \mathcal{L}(P + tR, \Lambda; \phi_1, \phi_2) \Big|_{t=0} = d_P F(P, P; \phi_1, \phi_2)(R) - d_P A(P, \Lambda; \phi_1, \phi_2)(R) \\ &= 2F(P, R; \phi_1, \phi_2) - A(R, \Lambda; \phi_1, \phi_2). \end{aligned} \quad (92)$$

The adjoint state $\hat{\Lambda}$ is the Λ for which the residual (92), evaluated for $P = U$, vanishes for each $R \in \mathcal{V}_{1,h} \times \mathcal{V}_{2,h}$, that is, the solution to the following problem: find $\hat{\Lambda} \in \mathcal{V}_{1,h} \times \mathcal{V}_{2,h}$ such that

$$A(R, \hat{\Lambda}; \phi_1, \phi_2) = 2F(U, R; \phi_1, \phi_2) \quad \forall R \in \mathcal{V}_{1,h} \times \mathcal{V}_{2,h}. \quad (93)$$

Finally, the directional derivative of J in the direction w is obtained by differentiating the Lagrangian with respect to ϕ_i in the direction w and evaluating the resulting expression for $P = U$ and $\Lambda = \hat{\Lambda}$, that is,

$$\begin{aligned} d_{\phi_i} J(\phi_1, \phi_2)(w) &= d_{\phi_i} \mathcal{L}(P, \Lambda; \phi_1, \phi_2) \Big|_{P=U, \Lambda=\hat{\Lambda}}(w) \\ &= d_{\phi_i} F(U, U; \phi_1, \phi_2)(w) - d_{\phi_i} A(U, \hat{\Lambda}; \phi_1, \phi_2)(w) + d_{\phi_i} l(\hat{\Lambda}; \phi_1, \phi_2)(w). \end{aligned} \quad (94)$$

Remark 3.22. It may not be entirely obvious why the directional derivative of the reduced objective function is obtained according to expression (94), which is why we here give a short justification. This explanation also highlights the main reason why the procedure is merely formal. According to definitions (83) and (90), the reduced objective satisfies

$$J(\phi_1, \phi_2) = \mathcal{L}(U(\phi), \Lambda; \phi_1, \phi_2) \quad \forall \Lambda \in \mathcal{V}_{1,h} \times \mathcal{V}_{2,h}. \quad (95)$$

Differentiating with respect to ϕ_i in direction w and applying the chain rule yields

$$d_{\phi_i} J(\phi_1, \phi_2)(w) = d_P \mathcal{L}(U(\phi), \Lambda; \phi_1, \phi_2)(dU) + d_{\phi_i} \mathcal{L}(U(\phi), \Lambda; \phi_1, \phi_2)(w) \quad \forall \Lambda \in \mathcal{V}_{1,h} \times \mathcal{V}_{2,h}, \quad (96)$$

where $\phi = [\phi_1, \phi_2]$, and where

$$dU = \frac{d}{dt} U(\phi + tw) \Big|_{t=0} \quad (97)$$

is the *shape derivative* of the state U . Now note that, due to adjoint equation (93), the left side of expression (92) vanishes when $P = U$ and $\Lambda = \hat{\Lambda}$, that is,

$$d_P \mathcal{L}(U(\phi), \hat{\Lambda}; \phi_1, \phi_2)(R) = 0 \quad \forall R \in \mathcal{V}_{1,h} \times \mathcal{V}_{2,h}. \quad (98)$$

Thus, if the shape derivative dU exists and resides in $\mathcal{V}_{1,h} \times \mathcal{V}_{2,h}$, we may choose $R = dU$ in equation (98), which makes the first term on the right side of expression (96) to vanish for $\Lambda = \hat{\Lambda}$, leaving

$$d_{\phi_i} J(\phi_1, \phi_2)(w) = d_{\phi_i} \mathcal{L}(U(\phi), \hat{\Lambda}; \phi_1, \phi_2)(w), \quad (99)$$

which was the claim in expression (94). Hence, the main assumption in this scheme is that *the shape derivative dU exists and resides in $\mathcal{V}_{1,h} \times \mathcal{V}_{2,h}$* . The existence and regularity of shape derivatives of state variables are generally delicate issues. For instance, if body-fitted discretisations are used, and mesh deformations are utilised to keep the mesh conformed to changes in the level-set function, the shape derivative of a continuous state variable, if it exists, will typically contain jump discontinuities [27, § 4.2]. That is, the shape derivative of the state is then less regular than the state itself. In contrast, for the unfitted discretisation used here, the shape derivative, if it exists, will stay in the same space as the state variable itself [27, § 5.4].

Using the procedure above, we arrive at the following result for the directional derivative of J . Note that for brevity, we only state the derivative with respect to ϕ_1 .

Suppose D_{ϕ_1} and D_{ϕ_2} satisfy Assumption 3.2. Then, given a perturbation $\phi_{1,t}(\mathbf{x}) = \phi_1(\mathbf{x}) + tw(\mathbf{x})$, where w is a Lagrangian basis function, the directional derivative of objective function (83) is

$$\begin{aligned} d_{\phi_1} J(\phi_1, \phi_2)(w) = & \int_{\partial D_{\phi_1} \cap \overline{D}_{\phi_2}^c} (\mathbf{A}_1 \nabla u_1) \cdot (\nabla \hat{\lambda}_1 - \nabla u_1) \frac{w}{|\partial_{\mathbf{n}_{\phi_1}} \phi_1|} \, dS \\ & + \int_{\partial D_{\phi_1} \cap D_{\phi_2}} (\mathbf{A}_2 \nabla u_2) \cdot (\nabla \hat{\lambda}_2 - \nabla u_2) \frac{w}{|\partial_{\mathbf{n}_{\phi_1}} \phi_1|} \, dS \\ & + \int_{\partial D_{\phi_1} \cap \partial D_{\phi_2}} \left(\{\mathbf{A} \nabla u\} \cdot \llbracket \hat{\lambda} \rrbracket + \{\mathbf{A} \nabla \hat{\lambda}\} \cdot \llbracket u \rrbracket - \mu \llbracket u \rrbracket \cdot \llbracket \hat{\lambda} \rrbracket \right) \frac{w}{|\partial_{\mathbf{N}^2} \phi_1|} \, d\gamma, \end{aligned} \quad (100)$$

where $\hat{\lambda}_1$ and $\hat{\lambda}_2$ satisfy the adjoint problem: find $[\hat{\lambda}_1, \hat{\lambda}_2] \in \mathcal{V}_{1,h} \times \mathcal{V}_{2,h}$ such that

$$\begin{aligned} & (\mathbf{A}_1 \nabla r_1, \nabla \hat{\lambda}_1)_{D_{\phi_1} \cap \overline{D}_{\phi_2}^c} + (\mathbf{A}_2 \nabla r_2, \nabla \hat{\lambda}_2)_{D_{\phi_1} \cap D_{\phi_2}} + j([r_1, r_2], [\hat{\lambda}_1, \hat{\lambda}_2]) + (\mu \llbracket r \rrbracket, \llbracket \hat{\lambda} \rrbracket)_{D_{\phi_1} \cap \partial D_{\phi_2}} \\ & - (\{\mathbf{A} \nabla \hat{\lambda}\}, \llbracket r \rrbracket)_{D_{\phi_1} \cap \partial D_{\phi_2}} - (\{\mathbf{A} \nabla r\}, \llbracket \hat{\lambda} \rrbracket)_{D_{\phi_1} \cap \partial D_{\phi_2}} = 2F(U, [r_1, r_2]; \phi_1, \phi_2), \end{aligned} \quad (101)$$

for all $[r_1, r_2] \in \mathcal{V}_{1,h} \times \mathcal{V}_{2,h}$. The derivation of this result is given in the Supplementary Material.

Remark 3.23. The differentiation method outlined above constitutes a general procedure to devise expressions for reduced gradients, a procedure that is applicable to a wide range of shape and topology optimisation problems. However, in some particular cases, the optimisation problem can be formulated to possess symmetries such that no separate adjoint state is needed. In fact, the thermal dissipation example we consider here can be reformulated to provide such a symmetry by minimising $j(\phi_1, \phi_2) = l(U; \phi_1, \phi_2) = \sum_i (g, u_i)_{\Gamma_N}$ instead of $J(\phi_1, \phi_2) = F(U, U; \phi_1, \phi_2)$. After making this change and carrying out the above differentiation scheme, it turns out that the adjoint state for the reformulated problem satisfies $\tilde{\lambda}_i = u_i$; that is, a separate adjoint equation will not be needed. The gradient expression for the reformulated problem will then be

$$\begin{aligned} d_{\phi_1} j(\phi_1, \phi_2)(w) = & \int_{\partial D_{\phi_1} \cap \overline{D}_{\phi_2}^c} (\mathbf{A}_1 \nabla u_1) \cdot \nabla u_1 \frac{w}{|\partial_{\mathbf{n}_{\phi_1}} \phi_1|} \, dS + \int_{\partial D_{\phi_1} \cap D_{\phi_2}} (\mathbf{A}_2 \nabla u_2) \cdot \nabla u_2 \frac{w}{|\partial_{\mathbf{n}_{\phi_1}} \phi_1|} \, dS \\ & + \int_{\partial D_{\phi_1} \cap \partial D_{\phi_2}} \left(2\{\mathbf{A} \nabla u\} \cdot \llbracket u \rrbracket - \mu \llbracket u \rrbracket \cdot \llbracket u \rrbracket \right) \frac{w}{|\partial_{\mathbf{N}^2} \phi_1|} \, d\gamma, \end{aligned} \quad (102)$$

The quantity $l(U; \phi_1, \phi_2)$ is the measure of heat power supplied to the system through the boundary Γ_N , whereas $F(U, U; \phi_1, \phi_2)$ is the heat power dissipated in the domain. Due to energy conservation, these quantities are the same for the problem before discretisation, and the difference between these problem formulations is only due to discretisation effects — namely the Nitsche interface terms — that will vanish as $h \rightarrow 0$.

This example serves to show how level-set shape derivatives *can* be computed by hand for a multi-phase problem. However, not only are the resulting expressions complicated, particularly for the case given in the Supplementary Material, their implementation in source code is tedious, error prone, and narrowly specific, making it an arduous task to update for new cases. To this end, in the next section, we develop automatic shape differentiation techniques to provide an alternate way to compute these quantities.

4. Automatic shape differentiation

In this section, we discuss how the automatic differentiation techniques developed by Wegert et al. [25, 30] can be extended as an alternative to using the formulas devised in Theorems 3.10, 3.15, and 3.18.

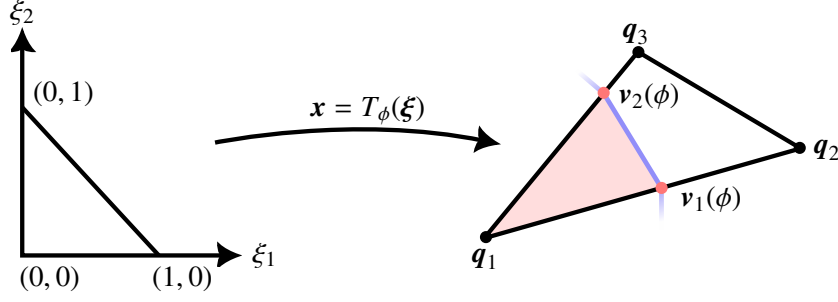


Figure 7 An illustration of the geometric map $T_\phi : \hat{K} \rightarrow K_{\text{sub}}(\phi)$ from the reference cell $\hat{K}$ to a cut subcell $K_{\text{sub}}(\phi)$ (pink). The intersection points $v_1(\phi)$ and $v_2(\phi)$ of the cut subcell are computed using $\phi(q_1)$, $\phi(q_2)$, and $\phi(q_3)$ via interpolation. Figure recreated from Wegert et al. [25].

4.1. Preliminaries

There are two important concepts that we discuss to motivate the rest of this section. The first is forward-mode automatic differentiation via *dual numbers*. This approach involves computing the derivative of a composition of functions by propagating dual numbers from input to output [39]. A dual number is defined as $a + b\varepsilon$ with the property $\varepsilon^2 = 0$ where $a \in \mathbb{R}$ is the real component and $b \in \mathbb{R}$ is the dual component. The Taylor series of a differentiable function $f(a + b\varepsilon)$ about a is

$$f(a + b\varepsilon) = f(a) + bf'(a)\varepsilon \quad (103)$$

where the higher order terms vanish because $\varepsilon^2 = 0$. The derivative of f can therefore be computed exactly by evaluating the dual component of $f(a + b\varepsilon)$. It is also possible to define N -dimensional dual numbers ε with the property that $\varepsilon_i \varepsilon_j = 0$ [40]. Using this, one can show for a function $g : \mathbb{R}^N \rightarrow \mathbb{R}$,

$$g(\mathbf{x} + \varepsilon) = g(\mathbf{x}) + \sum_{i=1}^N \frac{\partial g}{\partial x_i} \varepsilon_i. \quad (104)$$

As a result, all the partial derivatives of g at $\mathbf{x}$ can be computed with a single evaluation of $g(\mathbf{x} + \varepsilon)$. We call this a *dualised* function. We refer to Revels et al. [40] and the references therein for further discussion of vectorised forward-mode automatic differentiation.

In Wegert et al. [25], dual numbers were utilised to compute level-set shape derivatives for unfitted discretisations defined by a single level-set function ϕ for a fixed integrand. As an example, assume that we wish to compute the shape derivative of an integral over a domain in which $\phi < 0$. That is, we will compute the derivative of the integral with respect to the nodal values of the level-set function. Fundamentally, this approach relies on propagating dual numbers through the *local* geometric maps from the reference background cell to the physical cut subcell, illustrated in Figure 7. In particular, given a cut subcell, the derivative can be evaluated by: 1) propagating dual numbers through the cell-wise degrees of freedom of the level-set function; 2) computing the *dualised* nodal coordinates of the intersection of the interface defined by the level set and the boundary of the background cell; 3) constructing the geometric map from the reference background cell to the physical subcell; 4) evaluating the cell contribution to the integral via numerical quadrature and extracting the dual components. The local derivatives can then be assembled into a global gradient using routine finite element assembly. When the integrand f is not fixed (that is, it depends on the solution to a PDE that in turn depends on ϕ), an additional step is added in which the adjoint problem is constructed and solved. Thus, the shape derivative can be computed in tandem with the assembly of the integral.

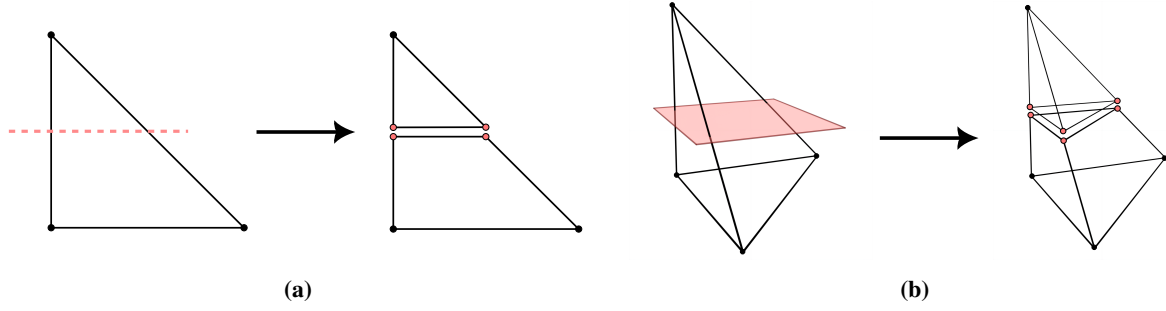


Figure 8 Visualisation of the polytope splitting operation `Split` in (a) two dimensions and (b) three dimensions.

This approach assumes that there is a single level-set function that is used to cut the background mesh [25]. This means that intersections of the interface and the boundary of a background cell can be computed a priori using a precomputed non-conforming triangulation of the interface. It is therefore possible to implement forward-mode automatic differentiation without propagating dual numbers through the cutting algorithm. For the case of several level-set functions, more complex intersections between the zero level sets can occur within a cell. Therefore, propagation of dual numbers through the cutting algorithm is necessary and requires special care for the sake of efficiency. We outline our approach below.

4.2. Cell-wise polytope cutter

In the following, we develop a polytope cutter to extend the automatic shape differentiation approach of Wegert et al. [25] to multiple level-set functions, ϕ_i for $i = 1, \dots, L$. We make the following design choices for the cutter:

- (i) Cell-wise: the cutter should operate on a cell-wise basis so that derivatives can be computed locally.
- (ii) Polytopal splitting: the cutter should be able to split an arbitrary polytope contained inside a cell K with a local level-set finite element function $\phi^{(K)}(\mathbf{x})$. We denote this operation `Split`.
- (iii) Recursive: to support several level-set functions $\phi_i^{(K)}(\mathbf{x})$, the subsequent split polytopes from (ii) should themselves be splittable by the cutter with another level-set function.
- (iv) Simplex polytopes: once (iii) has been completed for all level-set functions, the resulting polytopes should be simplexified for the purpose of numerical quadrature. We denote this operation `Simplexify`.

Figure 8 and 9 show examples of cell-wise `Split` and `Simplexify` operations in two and three dimensions. For `Split`, we reuse the methods from Wegert et al. [Sec. 4.3.1, 25]. In particular, we adapt the framework and algorithms in `STLCutters.jl` [41, 42] for the case where cuts are determined using a local level-set finite element function $\phi^{(K)}(\mathbf{x})$ inside a given cell K . For `Simplexify`, we adapt the algorithms from Gridap [32]. We refer the reader to Badia et al. [41], Martorell and Badia [42], Verdugo and Badia [32] for further discussion of the `Split` and `Simplexify` algorithms.

Using the `Split` and `Simplexify` operations, we can develop the `RecursiveCut` operation. Here, we utilise a binary tree with nodes that hold data related to a polytope $\mathcal{P}$ and a state vector $\mathbf{S}$ with components S_j , $j = 1, \dots, L$. When $\mathcal{P}$ is a cell, S_j indicates whether that cell is in or out with respect to each ϕ_j . When $\mathcal{P}$ is a facet, S_j indicates whether that facet is in, out, or is a cut generated by the zero level-set of ϕ_j . We assume the following functions implemented for the binary tree, where $\mathcal{N}$ refers to a node within the tree:

- `Polytope( $\mathcal{N}$ )`: Return the polytope stored in node $\mathcal{N}$.

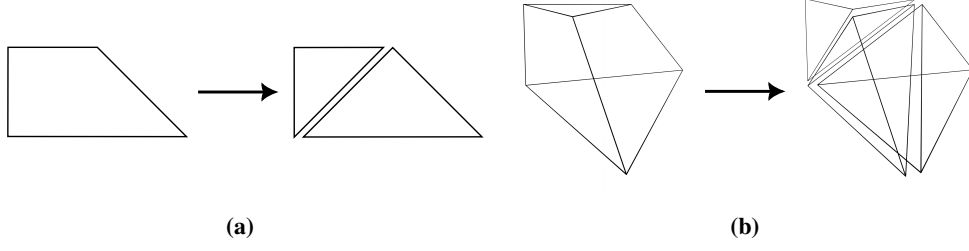


Figure 9 Visualisation of the polytope simplexify operation `Simplexify` in (a) two dimensions and (b) three dimensions.

- `State( $\mathcal{N}$ )`: Return $\mathcal{S}$ which gives the in, out, or cut state stored in node $\mathcal{N}$.
- `Children( $\mathcal{N}$ )`: Return the children, $\mathcal{N}_{\text{Left}}$ and $\mathcal{N}_{\text{Right}}$, of node $\mathcal{N}$.
- `Leaves( $\mathcal{N}$ )`: Return a list of polytopes and their respective states at the end points, also known as leaves, of the branch associated with node $\mathcal{N}$ of the binary tree. When $\mathcal{N}$ is the root node, this returns all leaves of the binary tree.
- `Root( $\mathcal{P}, L$ )`: Initialise the root node of a binary tree where $\mathcal{P}$ is the root cell or facet and L is the total number of level-set functions.

Finally, the states in $\mathcal{S}$ are denoted to either be $\text{IN} = -1$, $\text{OUT} = 1$, or $\text{CUT} = 0$. The recursive cutting algorithm is then as follows.

Algorithm 1 `RecursiveCut( $\mathcal{N}, \phi^{(K)}, L, s$ )`

```

1: if IsEmpty( $\phi^{(K)}$ ) then
2:   return
3: end if
4:  $\mathcal{P} \leftarrow \text{Polytope}(\mathcal{N}), \quad \mathcal{S} \leftarrow \text{State}(\mathcal{N}), \quad n \leftarrow L - \text{Length}(\phi^{(K)}) + 1$ 
5: if  $s = n$  then
6:    $\mathcal{S}_n \leftarrow \text{CUT}$ 
7:   RecursiveCut( $\mathcal{N}, [\phi_2^{(K)}, \dots, \phi_N^{(K)}], L, s$ )
8:   return
9: end if
10:  $(\text{state}, \mathcal{P}_{\text{in}}, \mathcal{P}_{\text{out}}) \leftarrow \text{Split}(\mathcal{P}, \phi_1^{(K)})$ 
11: if  $\text{state} = \text{IN}$  or  $\text{state} = \text{OUT}$  then
12:    $\mathcal{S}_n \leftarrow \text{state}$ 
13:   RecursiveCut( $\mathcal{N}, [\phi_2^{(K)}, \dots, \phi_N^{(K)}], L, s$ )
14:   return
15: end if
16:  $(\mathcal{N}_{\text{Left}}, \mathcal{N}_{\text{Right}}) \leftarrow \text{Children}(\mathcal{N})$ 
17:  $\text{Polytope}(\mathcal{N}_{\text{Left}}) \leftarrow \mathcal{P}_{\text{in}}, \quad \text{Polytope}(\mathcal{N}_{\text{Right}}) \leftarrow \mathcal{P}_{\text{out}}$ 
18: RecursiveCut( $\mathcal{N}_{\text{Left}}, [\phi_2^{(K)}, \dots, \phi_N^{(K)}], L, s$ )
19: RecursiveCut( $\mathcal{N}_{\text{Right}}, [\phi_2^{(K)}, \dots, \phi_N^{(K)}], L, s$ )
20: return

```

In the above, we denote $\phi^{(K)}$ to be the vector of remaining local level-set finite element functions, L is

the total number of level-set functions, and s is an integer specifying whether to skip a particular level-set function when cutting facets. The integer s specifies which level-set was used to generate an initial facet and is only non-zero when cutting facets. In line 10, we split the polytope $\mathcal{P}$ with a local level-set finite element function. If the level-set function does not split $\mathcal{P}$, then we specify whether that polytope is IN or OUT with respect to the level-set function in ‘state’. In the case that $\mathcal{P}$ is split, we take ‘state = nothing’ and return the two resulting polytopes from splitting $\mathcal{P}$. Note that Algorithm 1 returns nothing and instead modifies the binary tree.

Using the algorithm and operations described above, we now describe `CutAndSimplexifyCell` whereby a cell $K \in \mathcal{T}_h$ is cut recursively using a vector of level-set functions and then the resulting polytopes are simplexified. The resulting simplices make up $[\cup_i(D_{\phi_i} \cap K)] \cup [\cup_i(D_{\phi_i}^c \cap K)]$ and cover K .

Algorithm 2 `CutAndSimplexifyCell( $K, \phi^{(K)}$ )`

```

1:  $L \leftarrow \text{Length}(\phi^{(K)})$ 
2:  $\mathcal{N} \leftarrow \text{Root}(K, L)$ 
3: RecursiveCut( $\mathcal{N}, \phi^{(K)}, L, 0$ )
4:  $\mathcal{L} \leftarrow \text{Leaves}(\mathcal{N})$ 
5: return Simplexify( $\mathcal{L}$ )

```

Note that `Simplexify` is applied element-wise to the vector of polytopes and returns a list of all simplices and their state $\mathbf{S}$. Figure 10 shows a visualisation of this algorithm. Note that the first three rows of this figure show the polytopes associated with the nodes of the binary tree.

Finally consider the `CutAndSimplexifyFacet` algorithm. This algorithm splits a cell $K \in \mathcal{T}_h$ with the i^{th} level-set function, constructs the facet connecting the two subcells, then recursively cuts the facet with the remaining level-set functions, and returns a list of simplices making up the original facet, $\partial D_{\phi_i} \cap K$.

Algorithm 3 `CutAndSimplexifyFacet( $K, \phi^{(K)}, i$ )`

```

1: (state,  $\mathcal{P}_{\text{in}}, \mathcal{P}_{\text{out}}$ )  $\leftarrow \text{Split}(K, \phi_i^{(K)})$ 
2: if state = IN or state = OUT then
3:   continue
4: end if
5:  $\mathcal{F}_0 \leftarrow \partial \mathcal{P}_{\text{in}} \cap \partial \mathcal{P}_{\text{out}}, \quad L \leftarrow \text{Length}(\phi^{(K)})$ 
6:  $\mathcal{N} \leftarrow \text{Root}(\mathcal{F}_0, L)$ 
7: RecursiveCut( $\mathcal{N}, \phi^{(K)}, L, i$ )
8:  $\mathcal{L} \leftarrow \text{Leaves}(\mathcal{N})$ 
9: return Simplexify( $\mathcal{L}$ )

```

All of the facets $(\cup_i \partial D_{\phi_i}) \cap K$ can then be constructed by looping over the index of each level-set function and applying Algorithm 3.

Applying Algorithm 2 and 3 over the entire mesh constructs a non-conforming triangulation for $(\cup_i D_{\phi_i}) \cup (\cup_i D_{\phi_i}^c)$ that covers the entire computational domain D . Figure 12a shows an example. Using the states $\mathbf{S}$ for each simplex, we can isolate a particular domain D_{ϕ_j} or apply more complicated set operations to several domains. Figure 12b shows an example of the resulting triangulation of $D_{\phi_1} \cap \overline{D_{\phi_2}^c} \cap \overline{D_{\phi_3}^c}$.

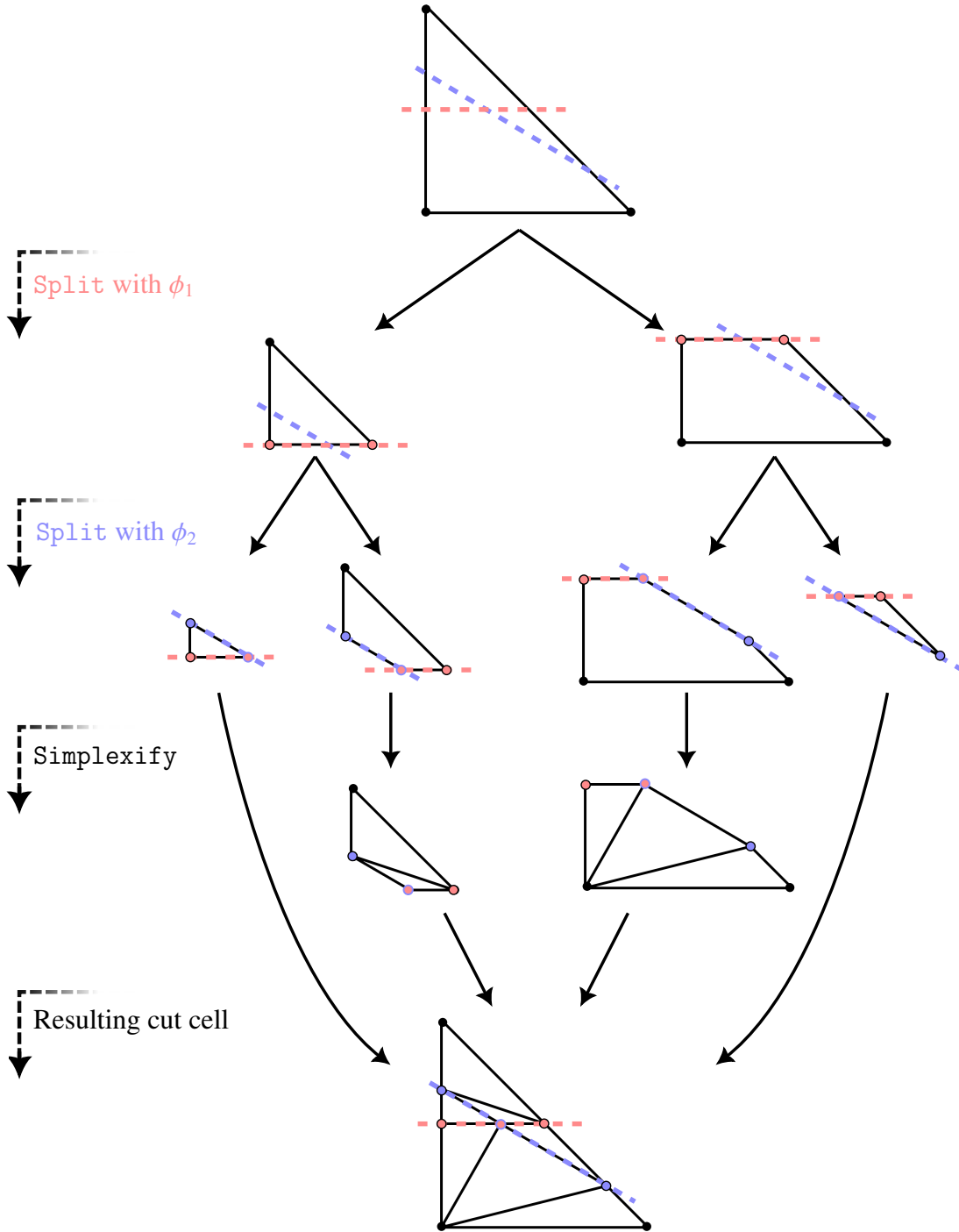


Figure 10 Visualisation of Algorithm 2 in which a two-dimensional simplex is recursively cut and simplexified with two level-set functions. The dashed blue and red lines are the cuts defined by ϕ_1 and ϕ_2 at the cell degrees of freedom, respectively. The coloured vertices represent the level-set functions used for splitting, for example, a blue and red dot means that both ϕ_1 and ϕ_2 contributed to the splitting process.

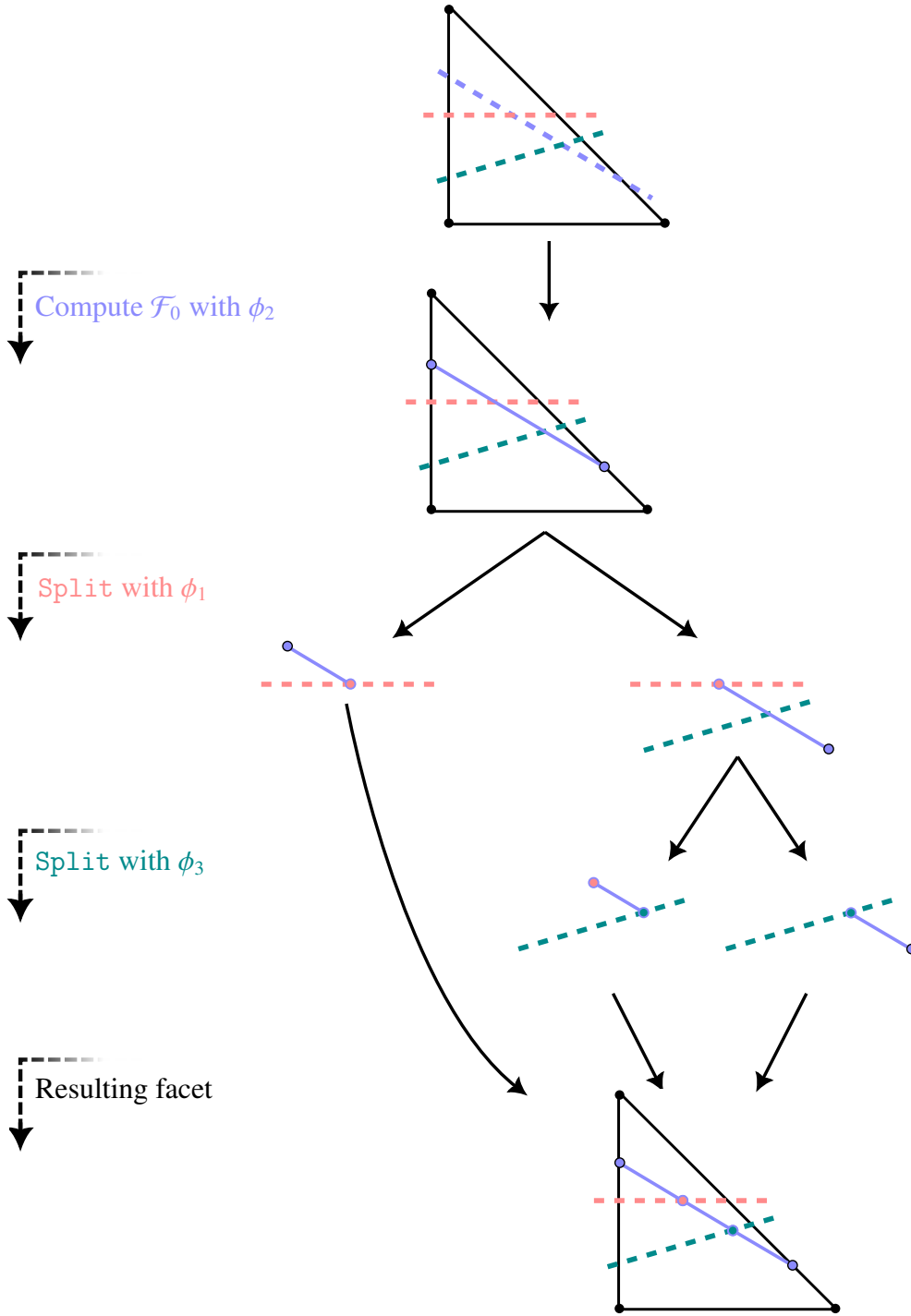


Figure 11 Visualisation of Algorithm 3 in which a one-dimensional facet is computed from a level-set function, recursively cut with the remaining level-set functions, then simplexified. Here we start with a facet $\mathcal{F}_0$ computed by splitting the polytope with ϕ_2 whose cut is shown by the dashed blue line. The resulting facet is then cut with ϕ_1 and ϕ_3 whose cuts are shown by the red and green dashed lines, respectively. The coloured vertices represent the level-set functions used for splitting. Note that in this example, facets are line segments and so Simplexify is simply the identity.

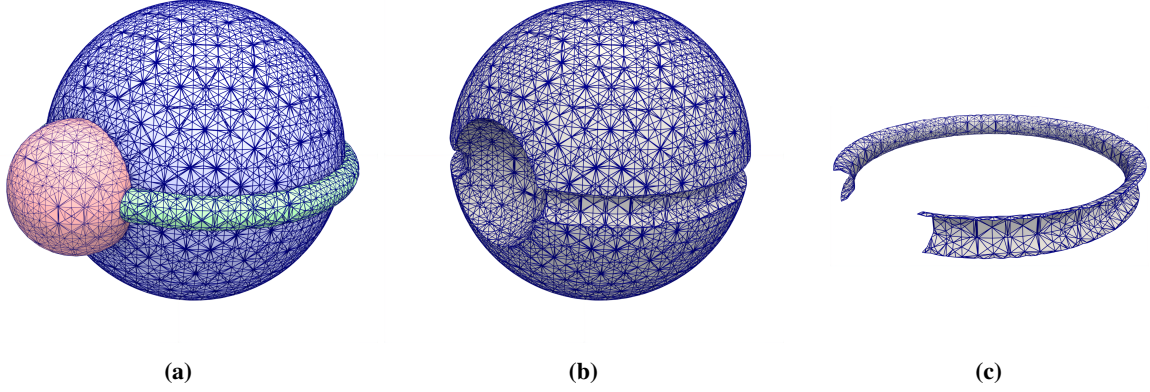


Figure 12 (a) Three non-conforming triangulations of D_{ϕ_i} ($i = 1, 2, 3$) constructed from level-set functions ϕ_i via Algorithm 2. (b) The triangulation of a domain $\Omega = D_{\phi_1} \cap \overline{D_{\phi_2}} \cap \overline{D_{\phi_3}}$ that is constructed by masking with the vectors $\mathcal{S}$ for each simplex that result from cutting. (c) The triangulation of the interface $\partial\Omega \cap \partial D_{\phi_3}$ resulting from Algorithm 3 and masking. The code to generate these is given in Listing 1.

4.3. Forward-mode automatic differentiation

We now consider how to utilise forward-mode automatic differentiation through Algorithms 1, 2, and 3. These algorithms are cell-wise operations and rely on the local level-set finite element functions $\phi_i^{(K)}(\mathbf{x})$. To compute directional derivatives in ϕ_i using dual numbers, we can construct a *dualised* version of $\phi_i^{(K)}(\mathbf{x})$ whereby the basis functions η_i are assigned a vector of duals ε_i , where i denotes that these are related to the i^{th} level set. These dual numbers then append the corresponding values of $\phi_i^{(K)}$ at each basis node. The following Dualise operation constructs a vector of level-set functions in which the k^{th} level set is dualised.

Algorithm 4 Dualise($\phi^{(K)}, k$)

- 1: $\eta_k \leftarrow \text{Basis}(\phi_k^{(K)})$
 - 2: $\phi_{\varepsilon, k}^{(K)}(\mathbf{x}) \leftarrow \sum_{n=1}^{N_j} (\phi_{n, k}^{(K)} + \varepsilon_n) \eta_{n, k}(\mathbf{x}),$
 - 3: $\phi_{\varepsilon, j}^{(K)} \leftarrow \phi_j^{(K)}$ for $j \neq k$
 - 4: **return** $\phi_{\varepsilon}^{(K)}(\mathbf{x})$
-

In the above, $\eta_{n, j}$ denotes the n^{th} basis of $\phi_j^{(K)}$, $\phi_{n, j}^{(K)}$ is the value of $\phi_j^{(K)}$ at the node where the n^{th} basis is unity, and δ_{jk} is the Kronecker delta. Note that only the k^{th} index of $\phi_{\varepsilon}^{(K)}(\mathbf{x})$ that relates to level-set function $\phi_k^{(K)}$ is dualised.

We now consider how a shape derivative under a perturbation of ϕ_k can be computed using forward-mode automatic differentiation. In the following, assume that $\Omega \subset \mathbb{R}^d$ is constructed using set operations on the domains D_{ϕ_i} . The level-set shape derivative of a functional $J(\phi) = \int_{\Omega(\phi)} f \, d\mathbf{x}$, where $f \in C^0(\overline{\mathcal{T}_h})$, with respect to ϕ_k can be computed as follows:

Algorithm 5 ShapeDerivative($f, \Omega, \mathcal{T}_h, \phi, k$)

```
1:  $\mathcal{T}_{h,\text{cut}} \leftarrow \{K \in \mathcal{T}_h : \cup_i [K \cap \partial D_{\phi_i}] \neq \emptyset\}$ 
2: for  $K \in \mathcal{T}_{h,\text{cut}}$  do
3:    $\phi^{(K)} \leftarrow \phi|_K$ 
4:    $\phi_\varepsilon^{(K)} \leftarrow \text{Dualise}(\phi^{(K)}, k)$ 
5:    $(\mathcal{P}_{\text{subcells}}, \mathbf{S}) \leftarrow \text{CutAndSimplexifyCell}(K, \phi_\varepsilon^{(K)})$ 
6:   for  $\mathcal{P} \in \text{Mask}(\mathcal{P}_{\text{subcells}}, \mathbf{S}, \Omega)$  do
7:      $(T, J_T) \leftarrow \text{CellMap}(K, \mathcal{P})$ 
8:      $\mathbf{dJ}_K \leftarrow \mathbf{dJ}_K + \text{Du}(\int_{T^{-1}(\mathcal{P})} (f \circ T) J_T \, d\xi)$ 
9:   end for
10: end for
11: return  $\mathbf{dJ}_{\text{glob}} \leftarrow \text{Assemble}(\mathbf{dJ})$ 
```

In the above, `Mask` should return the simplices that partition Ω as defined by the state vector $\mathbf{S}$; `CellMap`($K, \mathcal{P}$) should return the map T from the reference cell to the simplex $\mathcal{P}$ cell, as well as its Jacobian determinant J_T ; `Assemble` should take a vector of local contributions and assemble them into a global vector using standard finite element assembly; `Du` returns the dual part of a dual number or vector; and $\mathbf{dJ}_K$ is a vector of local contributions in a cell K with length equal to the number of basis functions of $\phi_k^{(K)}$. An equivalent algorithm can be used to compute the shape derivative of a functional defined on an interface by replacing the cell-wise cutting with facet-wise cutting and modifying the integrand accordingly.

Remark 4.1. It is possible to compute all derivatives of J in a single pass of Algorithm 5. This is achieved by modifying Algorithm 4 to dualise all level-set functions by replacing ε_n with dual numbers ε_{nj} that satisfy $\varepsilon_{nj}\varepsilon_{ml} = 0$ for all n, j, m, l .

4.4. Implementation

We extend the `GridapTopOpt` package [30] to include the polytope cutting and forward-mode automatic differentiation algorithms discussed above. These are compatible with the wider `Gridap` package ecosystem [31, 32]. Our code depends on and extends `GridapEmbedded` [43], which is the `Gridap` package that implements unfitted discretisations. As a result, much of the functionality available in `GridapEmbedded` (for instance the cell aggregation technique `AgFEM` [16]) could be used in our context, although we do not explore that in this work. Our implementation utilises `Gridap`'s `LazyArray` functionality to cache and defer the execution of operations until the specific elements of an array are explicitly accessed. Furthermore, we interface with `GridapDistributed` [33] so that our implementation can be leveraged in both serial and distributed CPU computing frameworks. The distributed computing implementation readily follows from the serial implementation because all computations are cell-wise and can be mapped over each local mesh partition.

Like `Gridap`, our implementation attempts to provide a clear and intuitive application programming interface (API). We demonstrate this in Listing 1, whereby we construct the domain Ω and interface Γ shown in Figures 12b and 12c, respectively, and computes the derivatives of a functional $J(\phi_i) = \int_{\Omega(\phi_i)} 1 \, dx + \int_{\Gamma(\phi_i)} 1 \, dS$ with respect to the three level-set functions ϕ_i for $i = 1, \dots, 3$.

Our forward-mode automatic differentiation implementation is fully compatible with `GridapTopOpt`'s adjoint methods for arbitrary PDE-constrained maps. At the lowest level, `GridapTopOpt` uses C ea's method to compute derivatives for PDE-constrained functionals. For the case of a variational form similar as the one in (85), the method proceeds analogously as described in Section 3.6. In particular, the adjoint

Listing 1 An example script demonstrating how the polytopal cutter and forward-mode automatic differentiation techniques can be leveraged in the wider Gridap ecosystem. The domain Ω and interface Γ are shown in Figures 12b and 12c.

```

1  using Gridap, Gridap.Adaptivity, Gridap.Geometry
2  using GridapEmbedded, GridapEmbedded.LevelSetCutters
3  using GridapTopOpt
4  # Background mesh
5  base_model = CartesianDiscreteModel((0,1,0,1,0,1),(41,41,41))
6  ref_model = refine(UnstructuredDiscreteModel(base_model), refinement_method = "barycentric")
7  model = get_model(ref_model)
8  # Level-set functions and geometries  $D\phi_i$ 
9  order = 1
10 reffe = ReferenceFE(lagrangian,Float64,order)
11  $V\phi_i$  = MultiFieldFESpace([TestFESpace(model,reffe) for i in 1:3])
12  $\phi_1$  = x->sqrt((x[1]-0.5)^2+(x[2]-0.5)^2+(x[3]-0.5)^2)-0.25
13  $\phi_2$  = x->sqrt((x[1]-0.75)^2+(x[2]-0.5)^2+(x[3]-0.5)^2)-0.1
14  $\phi_3$  = x->sqrt((0.25-sqrt((x[1]-0.5)^2+(x[2]-0.5)^2))^2 + (x[3]-0.5)^2) - 0.025
15  $\phi_{hi}$  = interpolate([ $\phi_1, \phi_2, \phi_3$ ],  $V\phi_i$ )
16  $D\phi_1$  = DiscreteGeometryFromFEFunction( $\phi_{hi}[1]$ , model)
17  $D\phi_2$  = DiscreteGeometryFromFEFunction( $\phi_{hi}[2]$ , model)
18  $D\phi_3$  = DiscreteGeometryFromFEFunction( $\phi_{hi}[3]$ , model)
19 # Triangulation of  $\Omega$  and  $\Gamma$ 
20  $\Omega_{\text{geo}}$  =  $D\phi_1 \cap !D\phi_2 \cap !D\phi_3$ 
21 cutgeo = cut(PolytopalLevelSetCutter(), model,  $\Omega_{\text{geo}}$ )
22  $\Omega$  = DifferentiableTriangulation(cutgeo,  $\Omega_{\text{geo}}$ )
23  $\Gamma$  = DifferentiableEmbeddedBoundary(cutgeo,  $\Omega_{\text{geo}}, D\phi_3$ )
24  $d\Omega$  = Measure( $\Omega$ , 2*order)
25  $d\Gamma$  = Measure( $\Gamma$ , 2*order)
26 # Functional and gradient
27  $J(\phi_{hi})$  =  $\int(1)d\Omega + \int(1)d\Gamma$ 
28  $dJ$  = gradient(J,  $\phi_{hi}$ )
29 vec_dJ = assemble_vector( $dJ, V\phi_i$ )

```

equation is obtained using Gridap’s element-wise, forward-mode automatic differentiation to compute $d_P \mathcal{L}(P, \Lambda; \phi_1, \phi_2)(R)|_{P=U}$. For this computation, R and Λ are, in each element, replaced with the local finite element basis functions, which assembled together produces the adjoint equation, whose solution $\hat{\Lambda}$ satisfies $d_P \mathcal{L}(U, \hat{\Lambda}; \phi_1, \phi_2) = 0$. The computation of the adjoint equation through this method exhibits the same computational complexity as a regular assembly of a finite-element matrix. The terms $d_{\phi_i} F$, $d_{\phi_i} A$, and $d_{\phi_i} l$ appearing in (94) are then computed using the forward-mode automatic differentiation techniques proposed in Section 4.3 instead of via Theorem 3.10, 3.15 and 3.18. For more complicated PDE-constrained maps (for instance ϕ mapping to $J(\phi)$ through several variational problems), the highest layer of GridapTopOpt’s automatic differentiation system enables reverse-mode automatic differentiation tools such as Zygote.jl [44] to efficiently accumulate the derivative using the adjoint methods available in GridapTopOpt. For further discussion of this, we refer to the GridapTopOpt documentation.

4.5. Verification and benchmarks

In Table 1 we verify our implementation of the forward-mode automatic differentiation implementation against the derivative expressions given in Theorems 3.10, 3.15, and 3.18 and finite differences for the geometries shown in Figure 13. Our implementation of the derivative formulas is not robust enough to handle

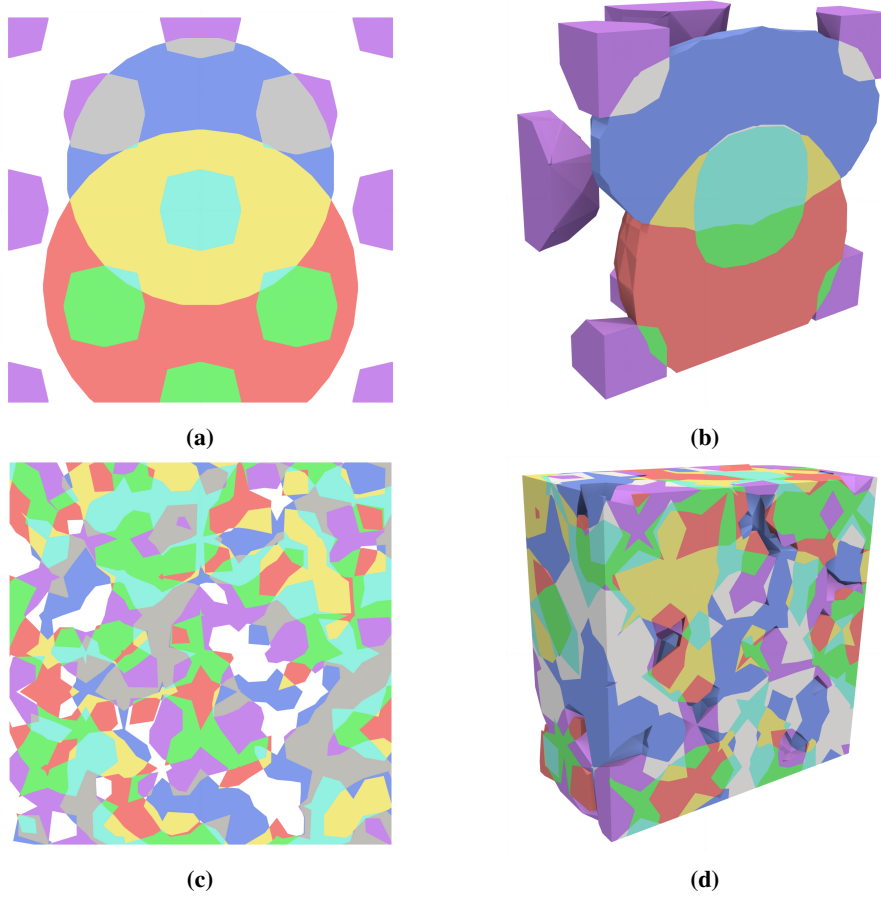


Figure 13 Geometries generated by three level-set functions for comparing the forward-mode automatic differentiation implementation with the exact derivatives provided in Section 3 and finite differences. (a) and (c) are in two dimensions. (b) and (d) are in three dimensions showing a visualisation of half of the background domain and with one phase removed to reveal internal structure. (c) and (d) are generated using random seeded values for the level-set functions.

Table 1 Verification of the forward-mode automatic differentiation implementation (dF_{AD}) against finite differences (dF_{FDM}) and the exact derivatives provided in Section 3 (dF) for the functional $\phi \mapsto \int_{\Omega} \sin(x) \sin(y) d\Omega$ with respect to all three level-set functions.

Ω \ Case	$\ dF_{AD} - dF_{FDM}\ _{\infty} / \ dF_{AD}\ _{\infty}$				$\ dF_{AD} - dF\ _{\infty} / \ dF\ _{\infty}$	
	Fig. 13a	Fig. 13b	Fig. 13c	Fig. 13d	Fig. 13a	Fig. 13b
$D_{\phi_1} \cap D_{\phi_2} \cap D_{\phi_3}$	4.46e-8	1.34e-8	1.86e-9	2.82e-9	1.02e-14	5.24e-16
$\overline{D_{\phi_1}} \cap \overline{D_{\phi_2}} \cap \overline{D_{\phi_3}}$	3.53e-8	1.51e-8	7.69e-9	2.69e-9	9.12e-15	2.50e-15
$\partial D_{\phi_1} \cap D_{\phi_2} \cap D_{\phi_3}$	7.28e-8	4.31e-8	9.27e-10	1.08e-8	5.17e-15	1.93e-14
$D_{\phi_1} \cap \partial D_{\phi_2} \cap \overline{D_{\phi_3}}$	5.94e-8	4.63e-8	8.88e-10	2.00e-9	6.74e-15	8.49e-15
$D_{\phi_1} \cap \overline{D_{\phi_2}} \cap \partial D_{\phi_3}$	8.89e-9	2.91e-8	1.78e-9	3.00e-9	3.62e-15	5.26e-15

randomly generated geometries (Fig. 13c & 13d), we therefore only use these geometries for validating the robustness of the automatic differentiation implementation against finite differences. For brevity, we only report relative errors in Table 1 for a few domain combinations. Our implementation in GridapTopOpt includes an extensive suite of unit tests that cover all derivative cases for one, two and three level-set functions

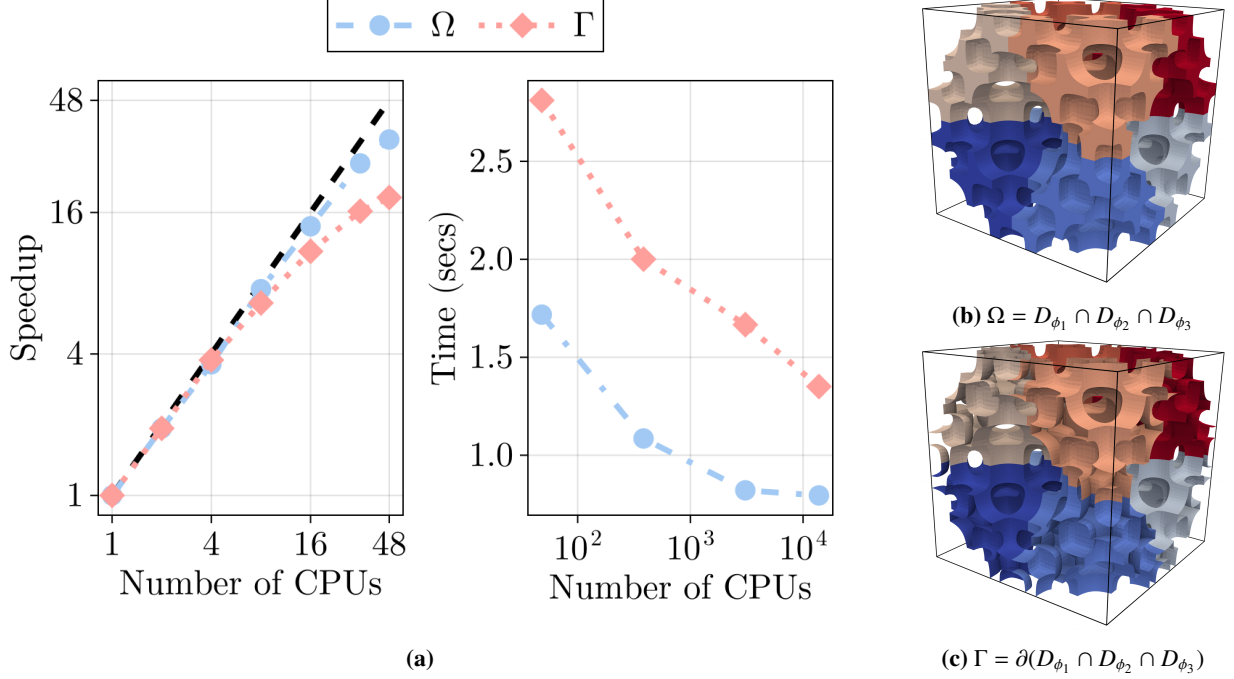


Figure 14 (a) Scalability benchmarks for computing all derivatives of $(\phi_1, \phi_2, \phi_3) \mapsto \int_M \cos(x) \sin(y) dM$ where $M = \Omega$ or Γ . The strong scaling benchmark is given in the left plot and weak scaling in the right plot. (b) and (c) Visualisations of the domains Ω and Γ , respectively, that are generated by $\phi_i(x, y, z) = \cos(a_i(x - b_i)) \cos(a_i y) \cos(a_i z) - c$ for $i = 1, 2, 3$ with $a_i = (2, 4, 6)$, $b_i = (0, 1/4, 1/6)$, and $c = \sqrt{2}/10$. The colours show the partitioning over eight CPUs for the strong scaling benchmark.

as well as tests for solving PDEs on the unfitted discretisations generated by the polytopal cutter.

Next, we consider strong and weak scaling benchmarks of the automatic differentiation functionality for the geometries shown in Figure 14. We utilise the Gadi supercomputing cluster at the NCI (Canberra, Australia). Each node used has 192 GB of RAM and an Intel Xenon Platinum 8274 (Cascade Lake) processor. We use OpenMPI 5.0.8 and Julia 1.12.6 for all computation. For strong scaling benchmarks, we use a $\sim 5M$ cell tetrahedral background mesh constructed from a Cartesian mesh with 62^3 cells that undergo one round of barycentric refinement. We partition this mesh over several CPUs, $P = \{1, 2, 4, 8, 16, 32, 48\}$. For weak scaling benchmarks, we set the number of cells per CPU to be 120,000 and scale up the number of CPUs, $P = \{48, 384, 3072, 13824\}$.

Figure 14a shows the strong- and weak-scaling results. For strong scaling, the performance is suboptimal, which is expected due to load imbalance across the processors. In contrast, the weak-scaling experiments demonstrate nearly ideal scalability up to 1,658,880,000 cells across 13824 CPUs. In fact, the runtime even decreases as the processor count grows, which is expected because the number of cut cells is still decreasing for finer meshes. Finally, across all our numerical experiments, we observe that computing all derivatives in a single pass of Algorithm 5 is more efficient than separately for each ϕ_i .

Finally, it should be noted that the use of automatic differentiation will invariably require more floating-point operations than an assembly of the directional derivatives given in Section 3. As an illustration, we consider taking $J(\phi_1, \phi_2, \phi_3) = \int_{\Omega} \cos(x) \cos(y) \cos(z) dx$ in Listing 1 and benchmarking Line 28–29 against the assembly of the exact derivative. For this case, we observe that assembling the exact derivative takes

approximately 0.44 seconds, whereas automatic differentiation takes approximately 1.46 seconds. As per the scalability results above, the forward-mode automatic differentiation can be substantially accelerated using a distributed CPU framework. In practice, the decision to rely on automatic differentiation should be balanced against the intellectual effort needed to derive and implement the exact expressions. In situations where the explicit formulas are cumbersome (for instance $d_{\phi_2}J(\phi_1, \phi_2)(w)$ in the Supplementary Material), or where the formulation of the optimisation problem is subject to change, automatic differentiation provides a practical route to reduce the time-to-first-result, to avoid errors in sensitivity derivations and implementation, and to make the code easier to maintain and update.

5. Level-set topology optimisation

In the following, we briefly discuss our implementation of multi-phase level-set topology optimisation with unfitted finite elements. We follow the approach of Wegert et al. [25] for each level-set function.

5.1. Boundary evolution

To update the level-set functions we use the approach proposed by Burman et al. [15]. Namely, we evolve the i^{th} level-set function ϕ_i by solving the transport equation

$$\begin{cases} \frac{\partial \phi_i(t, \mathbf{x})}{\partial t} + \boldsymbol{\beta}_i \cdot \nabla \phi_i(t, \mathbf{x}) = 0, \\ \phi_i(0, \mathbf{x}) = \phi_{i,0}(\mathbf{x}), \\ \mathbf{x} \in D, \quad t \in (0, T), \end{cases} \quad (105)$$

where $\boldsymbol{\beta}_i = g_i \mathbf{n}_i$ is the velocity field and $\phi_{i,0}$ is the initial value of the i^{th} level-set function. In this work, g_i is the gradient resulting from the Hilbertian extension approach [45], and the normal $\mathbf{n}_i$ is computed as $\mathbf{n}_i = \nabla \phi_i / \|\nabla \phi_i\|$. The Hilbertian extension approach identifies the directional derivative as a gradient in a Hilbert space H , defined on the background domain D , by solving the variational problem

$$\begin{aligned} &\text{Find } g_i \in H \text{ such that} \\ &\langle g_i, w \rangle_H = \text{d}J(\phi_i; w), \quad \forall w \in H, \end{aligned} \quad (106)$$

where $\langle \cdot, \cdot \rangle_H$ is the inner product on H . Here we use a weighted $H^1(D)$ bilinear form and solve using standard Lagrangian finite elements [25]. This choice provided a smooth gradient whose support is extended throughout the background domain D .

To solve (105) and (106), we utilise the approach of Wegert et al. [25] for each level-set function. In particular, the transport equation in (105) is solved using an interior penalty approach and Crank–Nicolson for the discretisation in time [24, 46, 47].

5.2. Reinitialisation

A smooth evolution of the boundary described above requires that the level-set function ϕ_i has well-behaved gradients. This is usually achieved by regularly reinitialising ϕ_i to be an approximation of the signed distance function [48] for the corresponding domain D_{ϕ_i} . That is, ϕ_i is reinitialised as

$$\phi_i(\mathbf{x}) = \begin{cases} -d(\mathbf{x}, \partial D_{\phi_i}) & \text{if } \mathbf{x} \in D_{\phi_i}, \\ 0 & \text{if } \mathbf{x} \in \partial D_{\phi_i}, \\ d(\mathbf{x}, \partial D_{\phi_i}) & \text{if } \mathbf{x} \in D \setminus \overline{D}_{\phi_i}, \end{cases} \quad (107)$$

where $d(\mathbf{x}, \partial D_{\phi_i}) := \min_{\mathbf{p} \in \partial D_{\phi_i}} |\mathbf{x} - \mathbf{p}|$ is the minimum distance from $\mathbf{x}$ to the boundary ∂D_{ϕ_i} . To find an approximate signed distance function for each level-set function, we utilise the approach of Mallon et al. [49], Wegert et al. [25]. In particular, we solve the reinitialisation problem using Picard iterations on the eikonal equation, stabilising the scheme via an artificial viscosity term, and prevent boundary movement via a surface penalty term. We refer to Mallon et al. [49] for further discussion.

5.3. Isolated volumes

Non-connected domains with isolated volumes may easily appear during the optimisation iterations. In the context of elliptic problems like those in Sections 2 and 6.1, the state equation restricted to an isolated volume will become singular due to the natural boundary condition on the volume's boundary. In this case, there will be a local one-dimensional null space of constant functions in the isolated volume. To deal with this issue in the context of multi-phase unfitted finite elements, we follow the approach of Wegert et al. [25]. In particular, we construct an indicator function χ using isolated volume tagging [25] and add a term to the bilinear form that constrains the average of the solution to be zero in the regions marked by χ . This addition removes the null space from isolated regions appearing in intermediate designs.

6. Examples

In this section, we consider example multi-phase topology optimisation problems using unfitted finite elements to demonstrate the capabilities provided by our approach. All scripts are provided open-source as described in the *Data availability* section.

6.1. Anisotropic diffusion

We first consider minimising the total thermal diffusion in two-dimensional multi-phase anisotropic media. This problem is also known as the *thermal compliance problem*, due to the resemblance with the analogue problem in linear elasticity. The formulation is as discussed in Sections 2 and 3.6, with the addition of volume constraints.

The optimisation problem is given by

$$\begin{aligned} \min_{(\phi_1, \phi_2) \in \mathcal{V}_h^2} & (\mathbf{A}_1 \nabla u_1, \nabla u_1)_{D_{\phi_1} \cap \overline{D}_{\phi_2}^c} + (\mathbf{A}_2 \nabla u_2, \nabla u_2)_{D_{\phi_1} \cap D_{\phi_2}} \\ \text{s.t. } & \text{Vol}(D_{\phi_1} \cap \overline{D}_{\phi_2}^c) = 0.05 \text{Vol}(D), \\ & \text{Vol}(D_{\phi_1} \cap D_{\phi_2}) = 0.05 \text{Vol}(D), \end{aligned} \quad (108)$$

where u_1 and u_2 satisfy $A([u_1, u_2], [v_1, v_2]; \phi_1, \phi_2) = l([v_1, v_2]; \phi_1, \phi_2)$ with A and l given by

$$\begin{aligned} A([u_1, u_2], [v_1, v_2]; \phi_1, \phi_2) &= (\mathbf{A}_1 \nabla v_1, \nabla v_1)_{D_{\phi_1} \cap \overline{D}_{\phi_2}^c} + (\mathbf{A}_2 \nabla v_2, \nabla v_2)_{D_{\phi_1} \cap D_{\phi_2}} \\ &+ \sum_{F \in \mathcal{F}_{G_1}} (\eta_1(h) \llbracket \partial_{\mathbf{n}_F} u_1 \rrbracket, \llbracket \partial_{\mathbf{n}_F} v_1 \rrbracket)_F + \sum_{F \in \mathcal{F}_{G_2}} (\eta_2(h) \llbracket \partial_{\mathbf{n}_F} u_2 \rrbracket, \llbracket \partial_{\mathbf{n}_F} v_2 \rrbracket)_F, \\ &- (\{\mathbf{A} \nabla_i u_i\}, \llbracket v_i \rrbracket)_{D_{\phi_1} \cap \partial D_{\phi_2}} - (\{\mathbf{A} \nabla_i v_i\}, \llbracket u_i \rrbracket)_{D_{\phi_1} \cap \partial D_{\phi_2}} + (\mu(h) \llbracket u_i \rrbracket, \llbracket v_i \rrbracket)_{D_{\phi_1} \cap \partial D_{\phi_2}} \\ &+ (\chi u_1, v_1)_{D_{\phi_1} \cap \overline{D}_{\phi_2}^c} + (\chi u_2, v_2)_{D_{\phi_1} \cap D_{\phi_2}}, \end{aligned} \quad (109)$$

and

$$l(V; \phi_1, \phi_2) = \sum_i (g, v_i)_{\Gamma_N}. \quad (110)$$

The last two terms of (109) constrains u_1 and u_2 to be zero within isolated volumes marked by χ (see Section 5.3). In this example, we take D to be the unit square and choose Γ_N and Γ_D to be given by the dark blue and black regions in Figure 15a, respectively. Furthermore, we take the flux g on Γ_N to be unitary and the anisotropic diffusion tensors to be

$$\mathbf{A}_1 = \begin{pmatrix} 5 & 0 \\ 0 & 1 \end{pmatrix}, \quad \mathbf{A}_2 = \begin{pmatrix} 1 & 0 \\ 0 & 5 \end{pmatrix}. \quad (111)$$

For the weighting parameters κ_1 and κ_2 , we use expressions (86). Finally, the ghost penalty parameters $\eta_i(h)$ and the Nitsche penalty parameter $\mu(h)$ are set to

$$\eta_i(h) = 0.1\alpha_i h, \quad \mu(h) = \frac{10^2 \max(\alpha_1, \alpha_2)}{h}, \quad (112)$$

where $\alpha_i = \|\mathbf{A}_i\|_\infty$. Finally, for the numerical experiments, we fix the geometry near the Dirichlet region to be a small ball. This is done by creating a level-set function for the region with domain D_{Diri} and modifying the descriptions of the phases to be $(D_{\phi_1} \cap \overline{D_{\phi_2}^c}) \setminus \overline{D_{\text{Diri}}}$ and $(D_{\phi_1} \cap D_{\phi_2}) \cup D_{\text{Diri}}$.

We discretise the above problem over a structured background mesh made of 40,000 triangles and solve the constrained optimisation problem in (108) using a projection method [50]. Automatic shape differentiation computes all sensitivities.

Figure 15 visualises the material phases at some iterations of the optimisation algorithm and shows the iteration history of the objective and volume constraints. The final optimised design has a cross-like structure, with vertical struts comprised of the material phase that has a large vertical diffusion coefficient, and horizontal struts comprised of the material phase that has a large horizontal diffusion coefficient. The design therefore gives efficient transfer of heat from the sources (Neumann boundary conditions) to the sink (Dirichlet boundary condition). The increase of the objective function over the optimisation that we see in Figure 15 is expected due to the high volume fraction of each phase in the initial design.

We can include obstacles using the same machinery that is used to fix the geometry around the Dirichlet region. In the example in Figure 16, we solve the anisotropic minimum thermal compliance problem with four square obstacles blocking the straight paths from the Neumann boundaries to the Dirichlet region, prescribing zero Neumann boundary conditions on the obstacle boundaries. For this example, the volume fraction constraint is 0.1 rather than 0.05 for each phase.

We also minimise the thermal compliance of a three-dimensional multi-phase anisotropic media. This example uses add a third phase that has an anisotropic diffusion tensor with a large z component. We constrain the three material phases to each have a volume fraction of 0.02. The boundary conditions are analogous to the two dimensional case; that is, the Dirichlet region is now a small spherical inclusion at the centre of the domain and the Neumann boundaries are square regions in the middle of each face. We solve this three-dimensional problem on the Gadi supercomputing cluster using 16 CPUs. The Hilbertian extension problem (106) is solved using an algebraic multigrid preconditioned conjugate gradient method as outlined in [30], whereas the direct parallel solver MUMPS [51, 52] is employed for the state equation, the transport equation, and the reinitialisation equation. Finally, we use $1/8^{\text{th}}$ symmetry of the problem to further reduce the computational burden. We note that the investigation of iterative methods is important for solving these problems at large scales; however this is outside the scope of the current contribution.

In Figure 17 we show results for this three-dimensional anisotropic thermal diffusion topology optimisation problem. Similar to the two-dimensional case, the final design has a cross-like structure where each member has a large diffusion coefficient in the most appropriate direction for efficient transfer of heat from the sources to the sink. In Figure 17a we see that the green phase is separated from the void phase by a thin

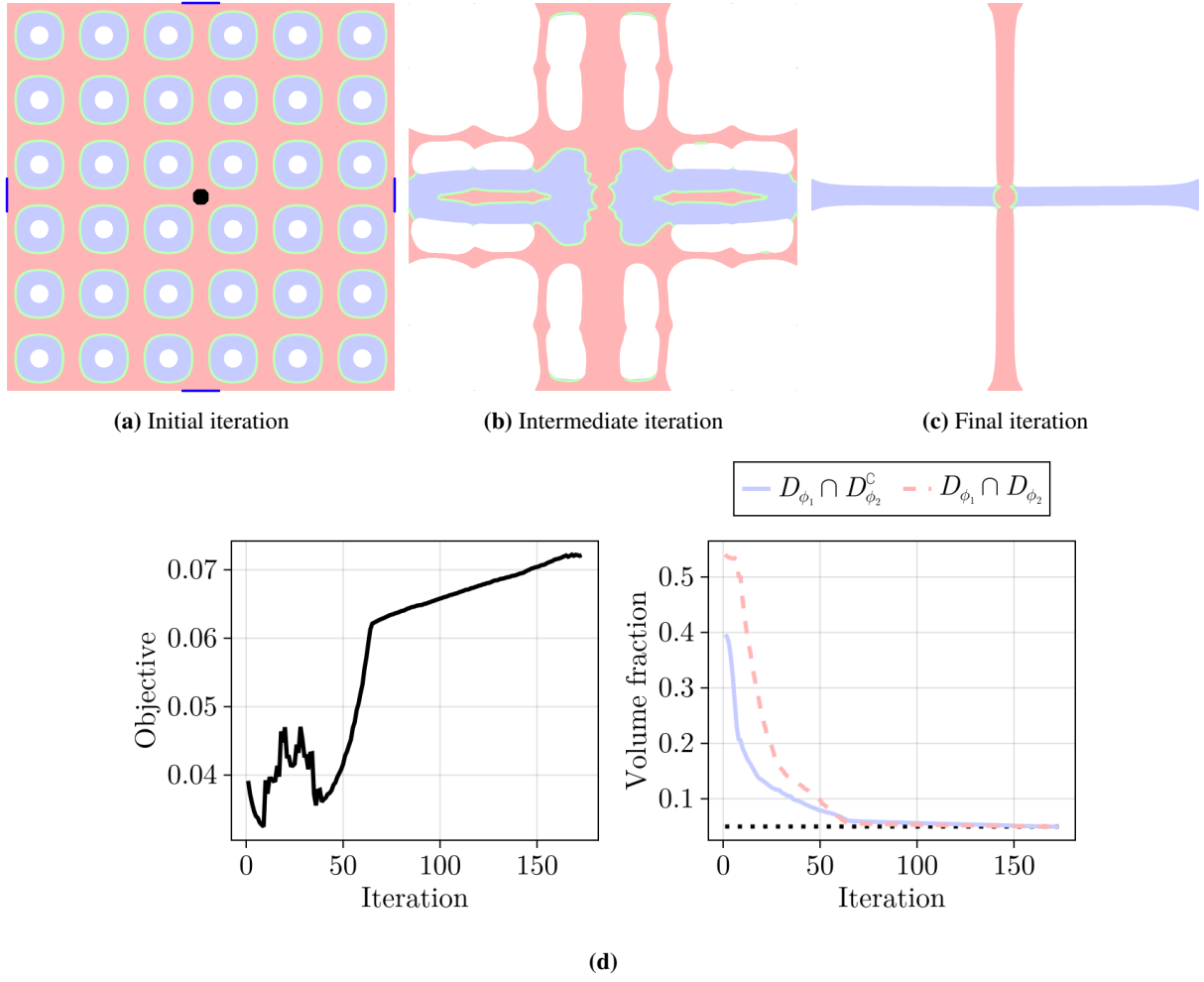


Figure 15 (a)–(c) Visualisations of the material phases for the two-dimensional anisotropic minimum thermal compliance problem. (d) Iteration history for the objective and volume fractions of each phase compared to the required volume fraction of 0.05. The blue and red phases correspond to materials with large diffusion coefficients in the horizontal and vertical directions, respectively. In green we visualise the interface between the two material phases. In (a), we show the Neumann and Dirichlet regions in dark blue and black, respectively.

layer consisting of the other material phases. This occurs because we have used two level-set functions to describe the three material phases and the void phase. The interface connecting the green phase $D_{\phi_1} \cap D_{\phi_2}$ to the void phase $\overline{D}_{\phi_1}^c \cap \overline{D}_{\phi_2}^c$ can therefore only ever have measure zero (see Fig. 1). To allow an interface between these phases, another level-set function could be added [e.g., 53]); however, further exploration of this is outside the scope of this work.

6.2. Cantilever

The next example that we consider is minimising the compliance of a multi-phase linear elastic cantilever subject to a volume constraint. This is a classical problem in the literature, and we extend the formulation described by Wegert et al. [25] and references therein.

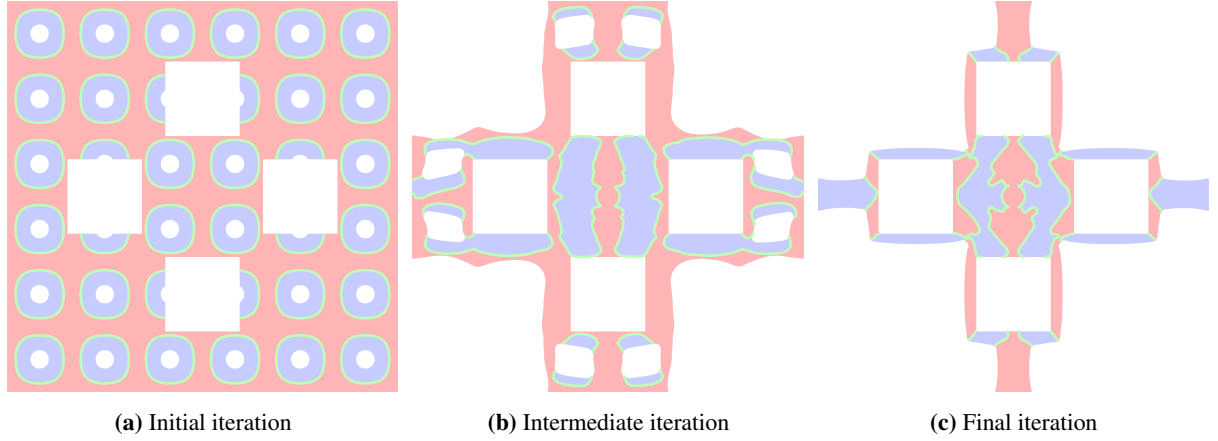


Figure 16 Visualisations of the material phases for the two-dimensional anisotropic minimum thermal compliance problem with the inclusion of four obstacles. The blue and red phases corresponds to materials with large diffusion coefficients in the horizontal and vertical directions, respectively. In green we visualise the interface between the two material phases.

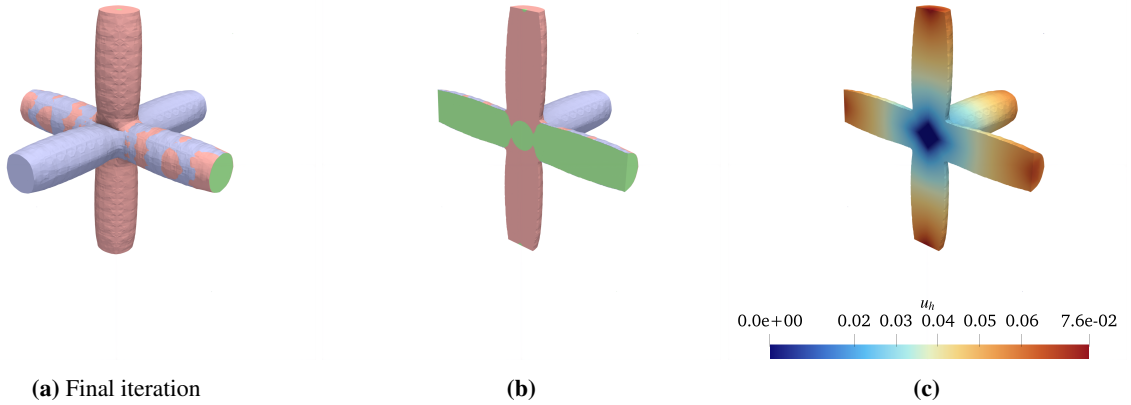


Figure 17 (a) Visualisation of the optimised material phases for the three-dimensional anisotropic minimum thermal compliance problem with three phases. (b) The internal structure of the phases. (c) Visualisation of the temperature field across all phases.

The optimisation problem is given by

$$\begin{aligned}
 \min_{(\phi_1, \phi_2) \in \mathcal{V}_h^2} & \quad (\boldsymbol{\varepsilon}(\mathbf{u}_1), \boldsymbol{\sigma}_1(\mathbf{u}_1))_{\overline{D}_{\phi_1}^c \cap D_{\phi_2}} + (\boldsymbol{\varepsilon}(\mathbf{u}_2), \boldsymbol{\sigma}_2(\mathbf{u}_2))_{D_{\phi_1} \cap D_{\phi_2}} \\
 \text{s.t.} & \quad \text{Vol}(\overline{D}_{\phi_1}^c \cap D_{\phi_2}) = 0.15 \text{Vol}(D), \\
 & \quad \text{Vol}(D_{\phi_1} \cap D_{\phi_2}) = 0.15 \text{Vol}(D),
 \end{aligned} \tag{113}$$

where the displacement fields $\mathbf{u}_1$ and $\mathbf{u}_2$ satisfy the variational form of the two-phase linear elasticity equations:

$$\begin{aligned}
 & \text{Find } [\mathbf{u}_1, \mathbf{u}_2] \in \mathcal{W}_{1,h} \times \mathcal{W}_{2,h} \text{ such that} \\
 & B([\mathbf{u}_1, \mathbf{u}_2], [\mathbf{v}_1, \mathbf{v}_2]; \phi_1, \phi_2) = l_s([\mathbf{v}_1, \mathbf{v}_2]; \phi_1, \phi_2) \quad \forall [\mathbf{v}_1, \mathbf{v}_2] \in \mathcal{W}_{1,h} \times \mathcal{W}_{2,h},
 \end{aligned} \tag{114}$$

where

$$\begin{aligned}
B([\mathbf{u}_1, \mathbf{u}_2], [\mathbf{v}_1, \mathbf{v}_2]; \phi_1, \phi_2) &= (\boldsymbol{\varepsilon}(\mathbf{v}_1), \boldsymbol{\sigma}_1(\mathbf{u}_1))_{\overline{D}_{\phi_1} \cap D_{\phi_2}} + (\boldsymbol{\varepsilon}(\mathbf{v}_2), \boldsymbol{\sigma}_2(\mathbf{u}_2))_{D_{\phi_1} \cap D_{\phi_2}} \\
&+ \sum_{F \in \mathcal{F}_{G_2}} (\eta_1(h) \llbracket \partial_{\mathbf{n}_F} \mathbf{u}_1 \rrbracket, \llbracket \partial_{\mathbf{n}_F} \mathbf{v}_1 \rrbracket)_F + \sum_{F \in \mathcal{F}_{G_3}} (\eta_2(h) \llbracket \partial_{\mathbf{n}_F} \mathbf{u}_2 \rrbracket, \llbracket \partial_{\mathbf{n}_F} \mathbf{v}_2 \rrbracket)_F, \\
&- (\{\boldsymbol{\sigma}_i(\mathbf{u}_i)\}, \llbracket \mathbf{v}_i \rrbracket)_{\partial D_{\phi_1} \cap D_{\phi_2}} - (\{\boldsymbol{\sigma}_i(\mathbf{v}_i)\}, \llbracket \mathbf{u}_i \rrbracket)_{\partial D_{\phi_1} \cap D_{\phi_2}} + (\mu(h) \llbracket \mathbf{u}_i \rrbracket, \llbracket \mathbf{v}_i \rrbracket)_{\partial D_{\phi_1} \cap D_{\phi_2}} \\
&+ (\chi \mathbf{u}_1, \mathbf{v}_1)_{\overline{D}_{\phi_1} \cap D_{\phi_2}} + (\chi \mathbf{u}_2, \mathbf{v}_2)_{D_{\phi_1} \cap D_{\phi_2}},
\end{aligned} \tag{115}$$

and

$$l_s([\mathbf{v}_1, \mathbf{v}_2]; \phi_1, \phi_2) = \sum_i (\mathbf{g}, \mathbf{v}_i)_{\Gamma_N}. \tag{116}$$

Here, $\mathcal{W}_{i,h} = \{\mathbf{v} \in C^0(\mathcal{T}_{i,h})^d : \mathbf{v}|_K \in \mathbb{P}_1(K)^d \ \forall K \in \mathcal{T}_{i,h}, \ \mathbf{v}|_{\Gamma_D} = \mathbf{0}\}$ and $\mathcal{T}_{1,h}$ and $\mathcal{T}_{2,h}$ are the active elements for $\overline{D}_{\phi_1} \cap D_{\phi_2}$ and $D_{\phi_1} \cap D_{\phi_2}$, respectively; $\boldsymbol{\sigma}_i$ is the stress tensor in each phase given by

$$\boldsymbol{\sigma}_i(\mathbf{u}_i) = \lambda_i \operatorname{tr}(\boldsymbol{\varepsilon}(\mathbf{u}_i)) \mathbf{I} + 2\mu_i \boldsymbol{\varepsilon}(\mathbf{u}_i), \quad i = 1, 2 \tag{117}$$

where $\boldsymbol{\varepsilon}(\mathbf{u}) = \frac{1}{2}(\nabla \mathbf{u} + (\nabla \mathbf{u})^\top)$ is the strain tensor, and λ_i and μ_i are the Lamé parameters corresponding to the Young's modulus E_i and Poisson's ratio ν_i of each material phase; $\mathbf{g}$ is the loading on Γ_N given by $\mathbf{g} = (0, -1)$; and Γ_N and Γ_D are given by the dark blue and black regions in Figure 18a, respectively. For the weighting parameters κ_1 and κ_2 , we take

$$\kappa_1 = \frac{E_2}{E_1 + E_2}, \quad \kappa_2 = \frac{E_1}{E_1 + E_2}. \tag{118}$$

Finally, the ghost penalty parameters $\eta_i(h)$ and Nitsche penalty parameter $\mu(h)$ are set to

$$\eta_i(h) = 10^{-7} \alpha_i h^3, \quad \mu(h) = \frac{10^2 \max(\alpha_1, \alpha_2)}{h}, \tag{119}$$

where we scale by $\alpha_i = \lambda_i + \mu_i$, based on Burman et al. [24].

We discretise the above problem over a structured background mesh made of 80,000 simplices and solve the constrained optimisation problem in (113) using a projection method [50], relying on automatic shape differentiation to compute all derivatives.

Figure 18b shows the optimisation result for $E_1 = 1.0$, $E_2 = 0.5$ and $\nu_1 = \nu_2 = 0.3$. As expected from previous results in the literature [e.g., 10], the material with a larger Young's modulus is concentrated at high-stress regions. It is important to note that although this problem is classical, the inclusion of a sharp interface between the phases enables several interesting extensions, such as more complicated interface conditions [e.g., 15].

6.3. Fluid–structure interaction

The final example is a fluid–structure interaction problem, in which the solid structure is made of two material phases. To our knowledge, this is the first time topology optimisation for such a setup has been considered in the literature. This problem exemplifies the type of multi-physics and multi-phase topology optimisation problems with complex interface conditions that the methods we investigate here are able to address.

The formulation of this problem is similar to the example considered by Wegert et al. [25], with the addition of multiple phases in the structural component. We assume that the structure is linear elastic and

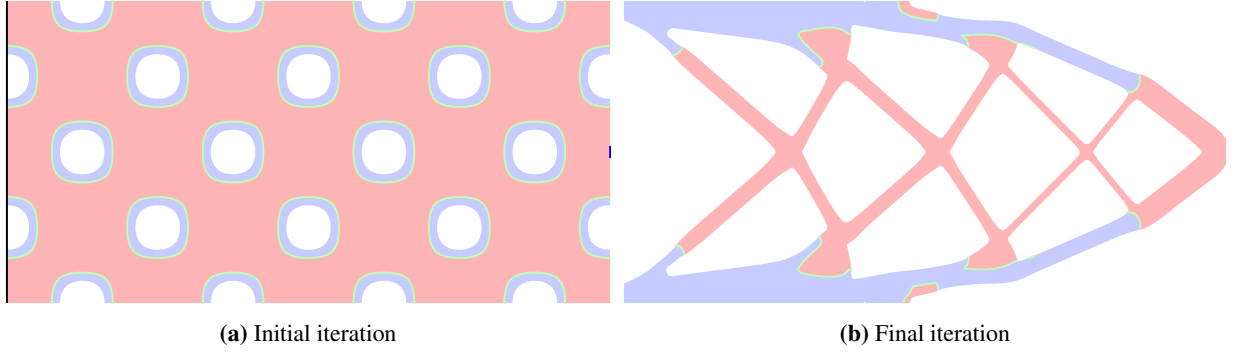


Figure 18 Visualisation of the material phases for the multi-phase minimum compliance cantilever problem. The blue and red phases corresponds to linear elastic materials with a Young's modulus of 1 and 1/2, respectively. In (a), we show the Neumann and Dirichlet regions in dark blue and black, respectively.

the fluid is governed by Stokes flow. We consider displacements small enough such that the coupling in the fluid–structure problem is one way. This allows the fluid and elastic problems to be decoupled, so that they can be solved in a sequential manner. For the Stokes flow problem, we use the stabilised Nitsche fictitious domain method and face ghost-penalty stabilisation [15] with continuous piecewise linear elements for the velocities and stabilised piecewise constant elements for the pressures.

The optimisation problem is given by

$$\begin{aligned}
& \min_{(\phi_1, \phi_2) \in \mathcal{V}_h^2} (\boldsymbol{\varepsilon}(\mathbf{d}_1), \boldsymbol{\sigma}_1(\mathbf{d}_1))_{\overline{D}_{\phi_1} \cap D_{\phi_2}} + (\boldsymbol{\varepsilon}(\mathbf{d}_2), \boldsymbol{\sigma}_2(\mathbf{d}_2))_{D_{\phi_1} \cap D_{\phi_2}} \\
& \text{s.t. } \text{Vol}(\overline{D}_{\phi_1} \cap D_{\phi_2}) = 0.035 \text{Vol}(D), \\
& \quad \text{Vol}(D_{\phi_1} \cap D_{\phi_2}) = 0.035 \text{Vol}(D),
\end{aligned} \tag{120}$$

where the displacement fields $\mathbf{d}_1$ and $\mathbf{d}_2$ satisfy the variational form of the two-phase linear elasticity equations:

$$\begin{aligned}
& \text{Find } [\mathbf{d}_1, \mathbf{d}_2] \in \mathcal{W}_{1,h} \times \mathcal{W}_{2,h} \text{ such that} \\
& B([\mathbf{d}_1, \mathbf{d}_2], [s_1, s_2]; \phi_1, \phi_2) = l_f([s_1, s_2]; [\mathbf{u}, p], \phi_1, \phi_2) \quad \forall [s_1, s_2] \in \mathcal{W}_{1,h} \times \mathcal{W}_{2,h},
\end{aligned} \tag{121}$$

where B is as in (115) and l_f is given by

$$l_f([s_1, s_2]; [\mathbf{u}, p], \phi_1, \phi_2) = \left((1 - \chi_s) s_1, (\boldsymbol{\sigma}_f(\mathbf{u}, p) \cdot \mathbf{n}) \right)_{\Gamma_{f,1}} + \left((1 - \chi_s) s_2, (\boldsymbol{\sigma}_f(\mathbf{u}, p) \cdot \mathbf{n}) \right)_{\Gamma_{f,2}}. \tag{122}$$

Here, the structure is subject to stress $\boldsymbol{\sigma}_f$ due to Stokes flow, where the velocity $\mathbf{u}$ and pressure p satisfy the variational form of Stokes flow:

$$\begin{aligned}
& \text{Find } [\mathbf{u}, p] \in \mathcal{U}_{h, \mathbf{u}_{in}} \times \mathcal{Q}_h \text{ such that} \\
& R([\mathbf{u}, p], [\mathbf{v}, q]; \phi_1, \phi_2) = 0 \quad \forall [\mathbf{v}, q] \in \mathcal{U}_{h, \mathbf{0}} \times \mathcal{Q}_h,
\end{aligned} \tag{123}$$

where

$$\begin{aligned}
R([\mathbf{u}, p], [\mathbf{v}, q]; \phi_1, \phi_2) = & (\nabla \mathbf{u}, \nabla \mathbf{v})_{\Omega_f} - (\partial_n \mathbf{u}, \mathbf{v})_{\Gamma} - (\partial_n \mathbf{v}, \mathbf{u})_{\Gamma} + (\gamma_{\Gamma} \mathbf{u}, \mathbf{v})_{\Gamma} - (p, \nabla \cdot \mathbf{v})_{\Omega_f} - (q, \nabla \cdot \mathbf{u})_{\Omega_f} \\
& - (p \mathbf{n}, \mathbf{v})_{\Gamma} - (q \mathbf{n}, \mathbf{u})_{\Gamma} + \sum_{F \in \mathcal{F}_{G,f}} (\gamma_u \llbracket \partial_{\mathbf{n}_F} \mathbf{u} \rrbracket, \llbracket \partial_{\mathbf{n}_F} \mathbf{v} \rrbracket)_F - \sum_{F \in \mathcal{F}_f} (\gamma_p \llbracket \mathbf{u} \rrbracket, \llbracket \mathbf{v} \rrbracket)_F + (\chi_f p, q)_{\Omega_f}.
\end{aligned} \tag{124}$$

Here, $\mathcal{W}_{i,h} = \{ \mathbf{v} \in C^0(\mathcal{T}_{i,h})^d : \mathbf{v}|_K \in \mathbb{P}_1(K)^d \ \forall K \in \mathcal{T}_{i,h}, \ \mathbf{v}|_{\Gamma_{\text{bottom}}} = \mathbf{0} \}$ are the spaces of the displacements, where $\mathcal{T}_{1,h}$ and $\mathcal{T}_{2,h}$ are the active elements for $\overline{D}_{\phi_1}^{\text{G}} \cap D_{\phi_2}$ and $D_{\phi_1} \cap \overline{D}_{\phi_2}^{\text{G}}$, respectively; $\mathcal{U}_{h,\mathbf{w}} = \{ \mathbf{v} \in C^0(\mathcal{T}_{f,h})^d : \mathbf{v}|_K \in \mathbb{P}_1(K)^d \ \forall K \in \mathcal{T}_{i,h}, \ \mathbf{v}|_{\Gamma_{\text{inlet}}} = \mathbf{w}, \ \mathbf{v}|_{\Gamma_{\text{bottom}} \cup \Gamma_{\text{top}}} = \mathbf{0} \}$ and $\mathcal{Q}_h = \{ p \in L^2(\mathcal{T}_{f,h}) : p|_K \in \mathbb{P}_0(K) \ \forall K \in \mathcal{T}_{f,h} \}$ are the velocity and pressure spaces, respectively, where $\mathcal{T}_{f,h}$ are the active elements for the fluid phase $\Omega_f = (D_{\phi_1} \cap \overline{D}_{\phi_2}^{\text{G}}) \cup (\overline{D}_{\phi_1}^{\text{G}} \cap D_{\phi_2})$; $\Gamma_{f,i}$ is the interface between the fluid phase and the i^{th} structural phase with $\Gamma = \Gamma_{f,1} \cup \Gamma_{f,2}$; $\mathcal{F}_{G,f}$ is the ghost skeleton for the fluid domain; $\mathcal{F}_f$ is the set of facets for the active domain defined by Ω_f ; χ_f is the indicator function for isolated volumes for the fluid phase; $\boldsymbol{\sigma}_f(\mathbf{u}, p) = 2\boldsymbol{\varepsilon}(\mathbf{u}) - p\mathbf{I}$ is the Stokes stress tensor; χ_s is the indicator function for isolated volumes in the structural problem; $\gamma_\Gamma = 100/h$ is the Nitsche parameter; $\gamma_u = 0.1h$ is the ghost penalty parameter; and $\gamma_p = 0.25h$ is the symmetric pressure stabilisation parameter. Note that following Wegert et al. [25], we include $1 - \psi_s$ in l_s to ensure that isolated volumes of the solid phase have zero displacement.

The background domain D has a length and height of 1 and 1/2, respectively. For the boundary conditions on the fluid, we prescribe no-slip boundary conditions on the top and bottom of the bounding domain D , a parabolic inlet velocity of $\mathbf{u}_{\text{in}} = (16y(0.5 - y)\mathbf{e}_1)$, a vanishing natural boundary condition for the Stokes variational form on the outlet, and a no-slip boundary condition on the fluid–structure interface. For the structural component, in addition to the forcing caused by the fluid, we have zero-displacement conditions on the bottom of the bounding domain. We include a non-designable support region for the structure shown in Figure 19a. Finally, the structural phases have a Young’s modulus of 1 and 0.1, as well as a Poisson’s ratio of 0.3.

We discretise the above problem over an unstructured background mesh with refinement around the non-designable region to improve accuracy of the solutions near the interface of the elastic structure. The non-refined region has a maximum element size of 0.025, while the refined region has a maximum element size of 0.0025. We solve the constrained optimisation problem in (120) using a projection method [50]. To ensure that the optimiser does not favour isolated volumes that disrupt the fluid flow by, a term is added to the objective to penalise isolated volumes in the solid phases [25]. Here, we take this penalisation term to be $I(\phi_1, \phi_2) = \int_{(\overline{D}_{\phi_1}^{\text{G}} \cap D_{\phi_2}) \cup (D_{\phi_1} \cap \overline{D}_{\phi_2}^{\text{G}})} h^{-2} \chi_s \, dx$, which vanishes when no isolated regions are present. Finally, automatic shape differentiation is used to compute all derivatives.

Figure 19 shows the material phases at the initial and final optimisation iterations. As in the case of the cantilever problem, the material with a larger Young’s modulus is concentrated at high-stress regions. These results demonstrate that multi-phase unfitted discretisations in conjunction with automatic shape differentiation can be used to effectively solve multi-phase and multi-physics topology optimisation problems.

7. Summary, conclusions, and outlook

The focus of this contribution has been on discrete shape calculus techniques and automatic shape differentiation for the case of functionals over domains that are defined by the intersections of multiple level-set functions. For the shape calculus, aimed at discretised cases, we used concepts from convex geometry to generalise the mathematical results of Berggren [27] and Wegert et al. [25] to arbitrary convex polytopes in $\mathbb{R}^d$. We constructed equivalence between level-set and polytopal representations in order to then construct and prove Theorems 3.10, 3.15, and 3.18. These results establish directional semiderivatives for the three types of functionals: (i) domain integrals (Sec. 3.3); (ii) boundary integrals truncated by a perturbed domain (Sec. 3.4); and (iii) boundary integrals (Sec. 3.5). For the case of a single level-set function, our results for (i) and (iii) reduce to those of Berggren [27] and Wegert et al. [25]. Case (ii) is new and appears only in the case of two or more intersecting level-set functions. The new results are general and establish a discrete shape calculus enabling multi-phase level-set topology optimisation with unfitted discretisations.

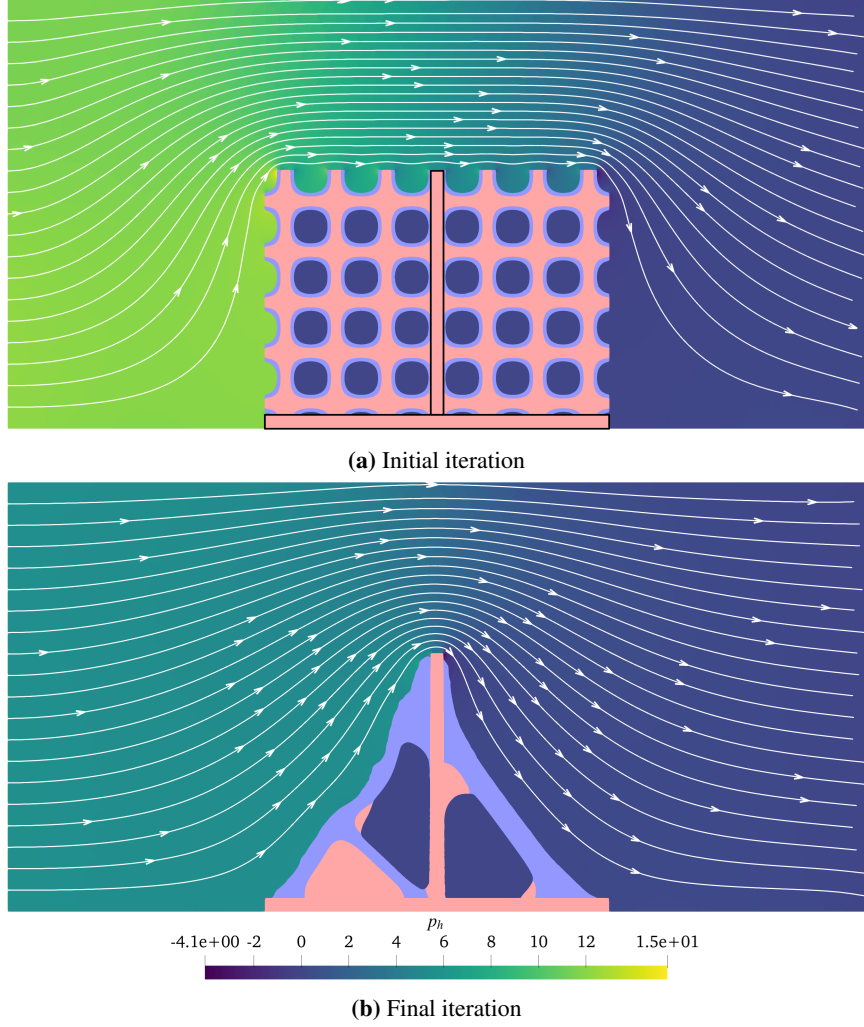


Figure 19 Visualisation of material phases for the minimum compliance fluid–structure interaction problem. The blue and red phases corresponds to linear elastic materials with a Young’s modulus of 1 and 0.1, respectively. The pressure field is visualised by the colour map while the direction of flow is indicated by the arrows on the streamlines of the velocity field. The region outlined by black lines in (a) is non-designable.

The shape calculus results are complemented by our proposed automatic shape differentiation approach for multi-phase problems. In particular, we developed a polytope cutter that provides an extension of the automatic shape differentiation approach of Wegert et al. [25] to multiple level-set functions. We validated the implementation against finite differences as well as the formulas devised in Theorems 3.10, 3.15, and 3.18. As expected, the automatic shape differentiation matched the exact derivatives to near machine precision (see Table 1). Our implementation can be leveraged in both serial and distributed CPU computing frameworks, and we demonstrate the scalability of the multi-phase automatic shape differentiation up to 1,658,880,000 cells over 13824 CPUs. The implementation aims for a clear and intuitive application programming interface (API), and is fully compatible with GridapTopOpt’s automatic differentiation of arbitrary PDE-constrained maps. Releasing this open-source software will facilitate the solution of complex multi-physics and multi-phase topology optimisation problems.

We demonstrated our framework for multi-phase unfitted level-set topology optimisation by solving topology optimisation problems for multi-phase anisotropic diffusion, multi-phase linear elasticity, and multi-phase fluid–structure interaction. The fluid–structure interaction example especially well demonstrates the power of our approach for solving topology optimisation problems involving multi-phase and multi-physics systems with complex boundary conditions. In particular, interface conditions between structural phases as well as the no-slip boundary condition at the fluid–structure boundaries are applied using unfitted finite elements and Nitsche’s method.

In the future, we plan to investigate the use of other unfitted finite element methods such as AgFEM, investigate matrix-free methods to improve the scalability of solvers, and to apply the approach to other complex design problems involving multiple physics and multiple phases.

CRedit authorship contribution statement

Zachary J Wegert: Writing – original draft, Writing – review and editing, Conceptualisation, Data curation, Mathematical analysis, Investigation, Methodology, Software, Validation, Visualisation. **Martin Berggren:** Writing – review and editing, Mathematical analysis, Investigation, Methodology, Validation. **Vivien J Challis:** Supervision, Conceptualisation, Writing – review & editing, Validation, Project administration, Funding acquisition, Resources.

Declaration of Competing Interest

The authors have no competing interests to declare that are relevant to the content of this article.

Data availability

The source code is available at <https://github.com/zjwegert/GridapTopOpt.jl>. To recreate the results, we provide all scripts at https://github.com/zjwegert/Wegert_et_al_2026_MP.

Acknowledgement

This research was funded (partially or fully) by the Australian Government through the Australian Research Council (ARC), Discovery Grant DP220102759. This research used computational resources provided by the Queensland Cyber Infrastructure Foundation (QCIF) and the National Computational Infrastructure (NCI) Australia. The first author would also like to thank Jordi Manyer for many insightful discussions regarding cutting algorithms.

References

- [1] X. Yu, J. Griffis, G. Manogharan, A. Panesar, Multi-material additive manufacturing: a computational design perspective, *Virtual and Physical Prototyping* 20 (2025) e2546671. doi:10.1080/17452759.2025.2546671.
- [2] B. Blakey-Milner, P. Gradl, G. Snedden, M. Brooks, J. Pitot, E. Lopez, M. Leary, F. Berto, A. Du Plessis, Metal additive manufacturing in aerospace: A review, *Materials & Design* 209 (2021) 110008. doi:10.1016/j.matdes.2021.110008.
- [3] L. Meng, W. Zhang, D. Quan, G. Shi, L. Tang, Y. Hou, P. Breitkopf, J. Zhu, T. Gao, From topology optimization design to additive manufacturing: Today’s success and tomorrow’s roadmap, *Archives of Computational Methods in Engineering* 27 (2020) 805–830. doi:10.1007/s11831-019-09331-1.
- [4] L. Peng, S. Xiuli, D. Shaojing, F. Yufan, Topology optimization methods and its applications in aerospace: a review, *Structural and Multidisciplinary Optimization* 68 (2025) 105. doi:10.1007/s00158-025-04030-x.

- [5] M. P. Bendsøe, Optimal shape design as a material distribution problem, *Structural Optimization* 1 (1989) 193–202. doi:10.1007/BF01650949.
- [6] G. I. N. Rozvany, M. Zhou, T. Birker, Generalized shape optimization without homogenization, *Structural Optimization* 4 (1992) 250–252. doi:10.1007/BF01742754.
- [7] M. Y. Wang, X. Wang, D. Guo, A level set method for structural topology optimization, *Computer Methods in Applied Mechanics and Engineering* 192 (2003) 227–246. doi:10.1016/S0045-7825(02)00559-5.
- [8] G. Allaire, F. Jouve, A.-M. Toader, Structural optimization using sensitivity analysis and a level-set method, *Journal of Computational Physics* 194 (2004) 363–393. doi:10.1016/j.jcp.2003.09.032.
- [9] A. Faure, G. Michailidis, G. Parry, N. Vermaak, R. Estevez, Design of thermoelastic multi-material structures with graded interfaces using topology optimization, *Structural and Multidisciplinary Optimization* 56 (2017) 823–837. doi:10.1007/s00158-017-1688-2.
- [10] G. Allaire, C. Dapogny, G. Delgado, G. Michailidis, Multi-phase structural optimization via a level set method, *ESAIM: Control, Optimisation and Calculus of Variations* 20 (2014) 576–611. doi:10.1051/cocv/2013076.
- [11] P. Liu, L. Shi, Z. Kang, Multi-material structural topology optimization considering material interfacial stress constraints, *Computer Methods in Applied Mechanics and Engineering* 363 (2020) 112887. doi:10.1016/j.cma.2020.112887.
- [12] B. Nguyen-Tran, D. Phan, T. Nguyen-Thoi, S. H. Nguyen, Multi-material structural topology optimization via dynamic body-fitted multi-trimmed mesh evolution and reaction–diffusion-based level-set method, *International Journal for Numerical Methods in Engineering* 126 (2025) e70167. doi:10.1002/nme.70167.
- [13] M. Kosta, O. Amir, Shape derivatives in bi-material level-set optimization with a precise interface: a comparative study, *Structural and Multidisciplinary Optimization* 68 (2025) 155. doi:10.1007/s00158-025-04095-8.
- [14] M. Kosta, E. Shakur, O. Amir, Multi-material topology optimization with interfacial stress constraints using body-fitted IGA, *Structural and Multidisciplinary Optimization* 69 (2026) 106. doi:10.1007/s00158-026-04307-9.
- [15] E. Burman, S. Claus, P. Hansbo, M. G. Larson, A. Massing, CutFEM: Discretizing geometry and partial differential equations, *International Journal for Numerical Methods in Engineering* 104 (2015) 472–501. doi:10.1002/nme.4823.
- [16] S. Badia, F. Verdugo, A. F. Martín, The aggregated unfitted finite element method for elliptic problems, *Computer Methods in Applied Mechanics and Engineering* 336 (2018) 533–553. doi:10.1016/j.cma.2018.03.022.
- [17] F. de Prenter, C. V. Verhoosel, E. H. van Brummelen, M. G. Larson, S. Badia, Stability and conditioning of immersed finite element methods: Analysis and remedies, *Archives of Computational Methods in Engineering* 30 (2023) 3617–3656. doi:10.1007/s11831-023-09913-0.
- [18] B. Schott, C. Ager, W. A. Wall, Monolithic cut finite element–based approaches for fluid–structure interaction, *International Journal for Numerical Methods in Engineering* 119 (2019) 757–796. doi:10.1002/nme.6072.
- [19] A. Bernland, E. Wadbro, M. Berggren, Acoustic shape optimization using cut finite elements: Acoustic shape optimization using cut finite elements, *International Journal for Numerical Methods in Engineering* 113 (2018) 432–449. doi:10.1002/nme.5621.
- [20] N. Berre, M. E. Rognes, A. Massing, Cut finite element discretizations of cell-by-cell emi electrophysiology models, *SIAM Journal on Scientific Computing* 46 (2024) B527–B553. doi:10.1137/23M1580632.
- [21] S. Claus, P. Kerfriden, A stable and optimally convergent LaTin-CutFEM algorithm for multiple unilateral contact problems, *International Journal for Numerical Methods in Engineering* 113 (2018) 938–966. doi:10.1002/nme.5694.
- [22] S. Claus, P. Kerfriden, A CutFEM method for two-phase flow problems, *Computer Methods in Applied Mechanics and Engineering* 348 (2019) 185–206. doi:10.1016/j.cma.2019.01.009.
- [23] H. Gammanpila, E. Aulisa, A. Chierici, Stabilized Nitsche-Type CIP/GP CutFEM for Two-Phase Flow Applications, *Mathematics* 13 (2025) 2853. doi:10.3390/math13172853.
- [24] E. Burman, D. Elfverson, P. Hansbo, M. G. Larson, K. Larsson, Shape optimization using the cut finite element method, *Computer Methods in Applied Mechanics and Engineering* 328 (2018) 242–261. doi:10.1016/j.cma.2017.09.005.
- [25] Z. J. Wegert, J. Manyer, C. N. Mallon, S. Badia, V. J. Challis, Level-set topology optimisation with unfitted finite elements and automatic shape differentiation, *Computer Methods in Applied Mechanics and Engineering* 445 (2025) 118203. doi:https://doi.org/10.1016/j.cma.2025.118203.
- [26] C. H. Villanueva, K. Maute, CutFEM topology optimization of 3d laminar incompressible flow problems, *Computer Methods in Applied Mechanics and Engineering* 320 (2017) 444–473. doi:10.1016/j.cma.2017.03.007.
- [27] M. Berggren, Shape calculus for fitted and unfitted discretizations: Domain transformations vs. boundary-face dilations, *Communications in Optimization Theory* 2023 (2023). URL: https://cot.mathres.org/archives/1568. doi:10.23952/cot.2023.27.
- [28] M. Delfour, Topological derivative: A semidifferential via the Minkowski content, *Journal of Convex Analysis* (2018) 957–982.
- [29] A. Griewank, A. Walther, Evaluating derivatives: principles and techniques of algorithmic differentiation, 2. ed ed., SIAM, Philadelphia, 2008.
- [30] Z. J. Wegert, J. Manyer, C. N. Mallon, S. Badia, V. J. Challis, GridapTopOpt.jl: a scalable Julia toolbox for level set-based

- topology optimisation, *Structural and Multidisciplinary Optimization* 68 (2025) 22. doi:10.1007/s00158-024-03927-3.
- [31] S. Badia, F. Verdugo, Gridap: an extensible finite element toolbox in Julia, *Journal of Open Source Software* 5 (2020) 2520. doi:10.21105/joss.02520.
 - [32] F. Verdugo, S. Badia, The software design of Gridap: A finite element package based on the Julia JIT compiler, *Computer Physics Communications* 276 (2022) 108341. doi:10.1016/j.cpc.2022.108341.
 - [33] S. Badia, A. F. Martín, F. Verdugo, GridapDistributed: a massively parallel finite element toolbox in Julia, *Journal of Open Source Software* 7 (2022) 4157. doi:10.21105/joss.04157.
 - [34] D. N. Arnold, F. Brezzi, B. Cockburn, L. D. Marini, Unified analysis of discontinuous Galerkin methods for elliptic problems, *SIAM J. Numer. Anal.* 39 (2002) 1749–1779.
 - [35] E. Burman, Ghost penalty, *C. R. Acad. Sci. Paris Sér. I Math.* 348 (2010) 1217–1220.
 - [36] P. M. Gruber, *Convex and Discrete Geometry*, volume 336 of *A Series of Comprehensive Studies in Mathematics*, Springer, Berlin, Heidelberg, 2007. doi:10.1007/978-3-540-71133-9.
 - [37] M. Spivak, *Calculus on manifolds: a modern approach to classical theorems of advanced calculus*, Mathematics monograph series, CRC Press, Taylor & Francis Group, Boca Raton London New York, 2018.
 - [38] J. Céa, Conception optimale ou identification de formes, calcul rapide de la dérivée directionnelle de la fonction coût, *ESAIM: Mathematical Modelling and Numerical Analysis* 20 (1986) 371–402. doi:10.1051/m2an/1986200303711.
 - [39] A. G. Baydin, B. A. Pearlmutter, A. A. Radul, J. M. Siskind, Automatic Differentiation in Machine Learning: a Survey, *Journal of Machine Learning Research* 18 (2018) 1–43.
 - [40] J. Revels, M. Lubin, T. Papamarkou, Forward-mode automatic differentiation in Julia (2016). doi:10.48550/arXiv.1607.07892, arXiv:1607.07892 [cs].
 - [41] S. Badia, P. A. Martorell, F. Verdugo, Geometrical discretisations for unfitted finite elements on explicit boundary representations, *Journal of Computational Physics* 460 (2022) 111162. doi:10.1016/j.jcp.2022.111162.
 - [42] P. A. Martorell, S. Badia, Stlcutters.jl: A scalable geometrical framework library for unfitted finite element discretisations, *Computer Physics Communications* 309 (2025) 109479. doi:10.1016/j.cpc.2024.109479.
 - [43] F. Verdugo, E. Neiva, P. A. Martorell, S. Badia, GridapEmbedded, <https://github.com/gridap/GridapEmbedded.jl>, 2020.
 - [44] M. Innes, Don't unroll adjoint: Differentiating SSA-form programs, 2019. URL: <https://arxiv.org/abs/1810.07951>. arXiv:1810.07951.
 - [45] G. Allaire, C. Dapogny, F. Jouve, Shape and topology optimization, volume 22, Elsevier, 2021, p. 1–132. doi:10.1016/bs.hna.2020.10.004.
 - [46] E. Burman, D. Elfverson, P. Hansbo, M. G. Larson, K. Larsson, A cut finite element method for the Bernoulli free boundary value problem, *Computer Methods in Applied Mechanics and Engineering* 317 (2017) 598–618. doi:10.1016/j.cma.2016.12.021.
 - [47] E. Burman, M. A. Fernández, Finite element methods with symmetric stabilization for the transient convection–diffusion–reaction equation, *Computer Methods in Applied Mechanics and Engineering* 198 (2009) 2508–2519. doi:10.1016/j.cma.2009.02.011.
 - [48] S. Osher, R. Fedkiw, *Level Set Methods and Dynamic Implicit Surfaces*, Applied Mathematical Sciences, 1 ed., Springer Science & Business Media, 2006.
 - [49] C. N. Mallon, A. W. Thornton, M. R. Hill, S. Badia, Neural Level Set Topology Optimization Using Unfitted Finite Elements, *International Journal for Numerical Methods in Engineering* 126 (2025) e70004. doi:10.1002/nme.70004.
 - [50] Z. J. Wegert, A. P. Roberts, V. J. Challis, A Hilbertian projection method for constrained level set-based topology optimisation, *Structural and Multidisciplinary Optimization* 66 (2023) 204. doi:10.1007/s00158-023-03663-0.
 - [51] P. Amestoy, I. S. Duff, J. Koster, J.-Y. L'Excellent, A fully asynchronous multifrontal solver using distributed dynamic scheduling, *SIAM Journal on Matrix Analysis and Applications* 23 (2001) 15–41.
 - [52] P. Amestoy, A. Buttari, J.-Y. L'Excellent, T. Mary, Performance and Scalability of the Block Low-Rank Multifrontal Factorization on Multicore Architectures, *ACM Transactions on Mathematical Software* 45 (2019) 2:1–2:26.
 - [53] Y. Wang, Z. Luo, Z. Kang, N. Zhang, A multi-material level set-based topology and shape optimization method, *Computer Methods in Applied Mechanics and Engineering* 283 (2015) 1570–1586. doi:10.1016/j.cma.2014.11.002.